



SCHOOL OF COMPUTATION, INFORMATION AND
TECHNOLOGY

TECHNICAL UNIVERSITY OF MUNICH

Bachelor's Thesis

Automated Equivalence Checking of Stabilizer Codes

Maxie Helen Bichmann





SCHOOL OF COMPUTATION, INFORMATION AND
TECHNOLOGY

TECHNICAL UNIVERSITY OF MUNICH

Bachelor's Thesis

Automatisierte Äquivalenzprüfung von Stabilizer
Codes

**Automated Equivalence Checking of
Stabilizer Codes**

Author:	Maxie Helen Bichmann
Supervisor:	Prof. Dr. Robert Wille
Advisor:	Ludwig Schmid
Submission Date:	27 August 2026

I confirm that this bachelor's thesis is my own work and I have documented all sources and materials used.

Ich versichere, dass ich diese Bachelor's Thesis selbständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet habe.

Ort, Datum

Maxie Helen Bichmann

Acknowledgments

I want to thank Tom Peham for proposing this topic, introducing me to the field, and supporting me both before and during the first stages of this thesis. I am also grateful to Ludwig Schmid for taking over my supervision and for his thoughtful guidance and valuable feedback throughout the later stages of this work.

Finally, I would like to express my appreciation to my family and friends for their encouragement and understanding during the course of this work.

Abstract

Determining whether two quantum error-correcting codes represent fundamentally different constructions or merely differ by admissible transformations is crucial for the practical comparison and analysis of codes. This thesis investigates the automated equivalence checking of stabilizer codes under permutations and local Clifford operations on their physical qubits. Four corresponding decision problems for general stabilizer codes and the subclass of CSS codes are formulated using the binary symplectic representation. To solve them, transformation invariants and decision procedures are adapted and implemented, utilizing tools such as Boolean satisfiability, classical-code equivalence, matroid and graph isomorphism, and graph-state transformations.

The experimental evaluation shows that the most suitable method depends on the code class, equivalence relation, and code parameters. SAT-based procedures provide the best overall performance for permutation and local Clifford equivalence of general stabilizer codes. For CSS-code permutation equivalence, a matroid-based approach successfully covers a broad parameter range, whereas this code class poses greater difficulty for a SAT-based method. A signature-based preprocessing step can be beneficial in deciding permutation equivalence for both general stabilizer and CSS codes.

The investigation further demonstrates that local-Clifford-equivalence procedures for stabilizer states do not generally extend to stabilizer codes with multiple logical qubits, due to the freedom in choosing logical operators.

Based on the observed performance characteristics, parameter-dependent hybrid decision procedures are designed, evaluated, and integrated into MQT-QECC. Across the evaluated instances, the resulting procedures reliably decide permutation equivalence for general stabilizer codes with up to 47 physical qubits. In particular, CSS-code inputs are practically feasible up to approximately 30 physical qubits, while preprocessing resolves some negative instances with up to 144 physical qubits. Local-Clifford-equivalence problems remain practically decidable up to 72 physical qubits, while inequivalence can often be established within seconds for instances with up to 144 physical qubits.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Research Questions	2
1.3	Contributions	2
1.4	Outline	3
2	Background	4
2.1	Quantum Error Correction	4
2.2	Stabilizer Formalism	5
2.3	Representation of Stabilizer Codes	7
2.3.1	Pauli Strings	7
2.3.2	Binary Symplectic Representation	8
2.3.3	Quaternary Representation	9
2.3.4	Quantum Circuits	10
2.3.5	ZX Calculus	11
2.4	CSS Codes	13
2.5	Clifford Operations	13
2.5.1	Clifford Group	13
2.5.2	Local Clifford Operations	14
2.6	Equivalence Notions	15
2.6.1	Row Equivalence	15
2.6.2	Permutation Equivalence	16
2.6.3	Local Clifford Equivalence	16
2.7	Algorithmic Tools	17
2.7.1	Graphs	17
2.7.2	Graph States	18
2.7.3	Matroids	18
2.7.4	Boolean Satisfiability	19
3	Related Work	20
4	Problem Formulation	21
4.1	Input Representation	21
4.2	Equivalence Problems	22
4.3	Quality Criteria	22
5	Algorithmic Approaches	24
5.1	Preprocessing	24
5.1.1	Invariants under Qubit Permutations	24

5.1.2	Invariants under Local Clifford Operations	25
5.1.3	Shared Assumptions	26
5.2	Common Subroutines	26
5.3	Brute-Force Search	27
5.3.1	Permutation Equivalence	27
5.3.2	Local Clifford Equivalence	27
5.4	SAT-Based Approaches	28
5.5	Classical-Code Approaches	33
5.5.1	Sendrier’s Support Splitting Algorithm	33
5.5.2	Feulner’s Canonicalization Algorithm	35
5.5.3	Adaptation to Quantum Codes	36
5.5.4	Automorphism Group	40
5.6	Isomorphism-Based Approaches	41
5.6.1	Matroid Isomorphism	41
5.6.2	Signature-Based Graph Construction	43
5.6.3	Full Stabilizer Group Mapping	44
5.7	Graph-State Machinery	47
5.7.1	From Stabilizer Code to Stabilizer State	48
5.7.2	From Stabilizer State to Graph State	52
5.7.3	Alternative Graph Constructions	54
5.7.4	Local Clifford Equivalence of Graph States	56
5.8	Comparison of Approaches	61
6	Implementation	63
6.1	Software Architecture	63
6.2	Integration Design	63
6.3	Implementation Choices	64
6.3.1	Runtime	64
6.3.2	Memory	64
7	Experimental Evaluation	65
7.1	Analytical Objectives	65
7.2	Experimental Setup	65
7.2.1	Input Instance Generation	65
7.2.2	Resource Restrictions	66
7.3	Methodology	67
7.3.1	Statistical Variation across Randomized Instances	67
7.3.2	Selection of Parameter Settings	67
7.4	Results	67
7.4.1	Figure Conventions	67
7.4.2	Invariant Benchmarks	68
7.4.3	Performance Comparison of Algorithmic Approaches	69
7.4.4	Performance on Small Codes	72
7.4.5	Performance on Large Codes	73

8	Integration	76
8.1	Permutation Equivalence of Stabilizer Codes	76
8.2	Permutation Equivalence of CSS Codes	77
8.3	Local Clifford Equivalence of Stabilizer Codes	79
8.4	Local Clifford Equivalence of Stabilizer Codes to CSS Codes	79
9	Discussion	81
9.1	Answers to the Research Questions	81
9.2	Broader Implications	81
9.3	Methodological Limitations	82
10	Conclusion	84
10.1	Summary of Contributions	84
10.2	Future Work	84
A	Appendix	86
	Notation	94
	Acronyms	96
	Bibliography	97

1 Introduction

1.1 Motivation

Quantum computing offers the potential to solve problems that are intractable for classical computers. However, quantum systems encode and process information differently from classical systems, introducing additional challenges. Quantum information is inherently fragile because the quantum states encoding it are highly susceptible to noise and errors.

The need to protect quantum information and ensure reliable computation has motivated the development of Quantum Error Correction (QEC). Although both share their basic principle of introducing redundancy to detect and correct errors, QEC faces additional challenges compared to its classical counterpart. In particular, the no-cloning theorem prohibits the direct copying of quantum information, while the continuous nature of quantum errors demands mechanisms to reduce them to a discrete set of correctable errors. Furthermore, the act of measuring a quantum state can destroy the information it encodes, necessitating indirect methods for error diagnosis. Consequently, Quantum Error-Correction Codes (QECCs) must be designed to operate within these inherently quantum-mechanical constraints [36, 39].

As the study of QECC remains a comparatively young field, many of the proposed constructions and protocols have not yet been standardized. Consequently, several frameworks for describing QECCs coexist, making their comparison and evaluation challenging. In particular, it may be difficult to determine whether two representations describe equivalent codes or correspond to inherently distinct constructions.

A general and efficient method for determining whether two codes are equivalent under a specified set of transformations is therefore of practical importance. Such a method could support the comparison of codes expressed in different representations, reveal redundancies in the literature, and aid the development of new codes by providing a systematic means of assessing their novelty.

This thesis investigates the automated equivalence checking of a subclass of QECCs under permutations of the physical qubits and local Clifford operations. These transformations are particularly relevant because different construction methods or circuit synthesizers may produce equivalent codes whose equivalence is not immediately apparent. Alternatively, establishing the existence of a more favorable equivalent code could enable the selection of a less costly construction process. The thesis focuses on stabilizer codes and, in particular, Calderbank-Shor-Steane (CSS) codes, which constitute widely studied classes of quantum codes due to their structured representations and their importance in fault-tolerant quantum computation.

1.2 Research Questions

This thesis addresses four distinct equivalence problems, guided by the following Research Questions (RQs), which are illustrated in Figure 1.1:

- **RQ1: Permutation Equivalence of Stabilizer Codes**
To what extent can deciding permutation equivalence of two given stabilizer codes be made practically feasible?
- **RQ2: Permutation Equivalence of CSS Codes**
To what extent can deciding permutation equivalence of two given CSS codes be made practically feasible?
- **RQ3: Local Clifford Equivalence of Stabilizer Codes**
To what extent can deciding local Clifford equivalence of two given stabilizer codes be made practically feasible?
- **RQ4: Local Clifford Equivalence of Stabilizer Codes to CSS Codes**
To what extent can deciding local Clifford equivalence of a given stabilizer code to an arbitrary CSS code be made practically feasible?

Here, practical feasibility refers to the ability to solve instances of relevant sizes within reasonable computational resources; it does not imply polynomial-time complexity.

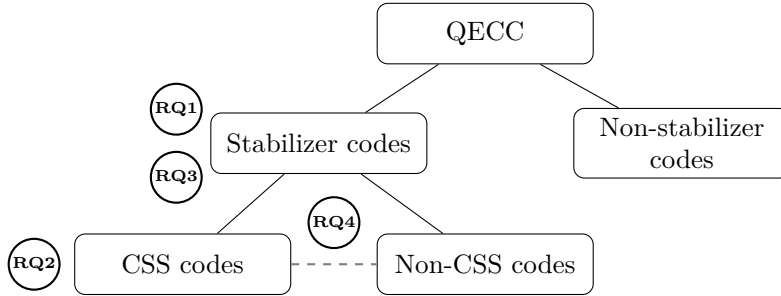


Figure 1.1: Overview of code classes and associated RQs.

1.3 Contributions

To address each RQ, the thesis makes the following contributions:

- Formulation of the exact equivalence problem for a suitable code representation
- Compilation and adaptation of algorithmic techniques for solving the problem, considering their computational complexity
- Implementation of different prototype algorithms
- Evaluation of the implemented approaches with respect to their practical feasibility, including the identification of limitations, performance bottlenecks, and promising directions for improvement

- Design and implementation of a hybrid algorithm informed by the evaluation
- Integration of the hybrid algorithm into the toolkit MQT-QECC

1.4 Outline

Chapter 2 introduces the *background* required to follow the equivalence problems and algorithmic techniques. The following Chapter 3 reviews *related work* on equivalence checking for QECC. Chapter 4 formally defines the equivalence *problems* addressed in this thesis. Building on these definitions, Chapter 5 presents candidate *approaches* and analyzes their theoretical complexity, while Chapter 6 focuses on *implementation* choices. Chapter 7 *evaluates* the resulting algorithms experimentally. Based on these results, Chapter 8 combines the most promising techniques into *hybrid* algorithms, whose practical feasibility is *discussed* in Chapter 9. Finally, Chapter 10 *summarizes* the thesis and outlines directions for future work.

2 Background

2.1 Quantum Error Correction

This section establishes the necessary concepts and motivation for QEC, based on the background provided in [24, 36, 39]. Fundamentally, QEC protects quantum information from potential errors by adding redundancy to its representation.

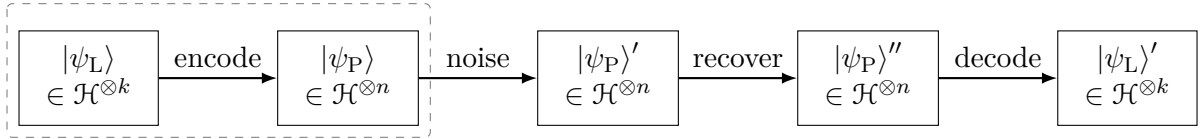


Figure 2.1: *Quantum Error Correction Process*

The process [36] of protecting this information can be divided into several stages, as illustrated in Figure 2.1.

First, a k -qubit quantum state $|\psi_L\rangle$ carrying logical quantum information is *encoded* into an n -qubit physical state $|\psi_P\rangle$, in a manner that does not violate the principles of quantum mechanics. The physical qubits are exposed to a *noisy channel*, which models the effects of potential errors in the form of bit or phase flips on the qubits. Consequently, the resulting state $|\psi_P\rangle'$ may differ from the encoded state. To detect such errors, syndrome information is extracted from the physical qubits, without destroying the transmitted quantum information. According to the syndrome, an appropriate *recovery* operation may be applied to obtain the state $|\psi_P\rangle''$. Lastly, the resulting n -qubit state is *decoded* back to a k -qubit state $|\psi_L\rangle'$, which is identical to the original logical state, if the error correction succeeds.

In this broader context, the thesis focuses primarily on the encoding of logical qubits into physical qubits—the first step shown in Figure 2.1—and on the properties of the employed QECC. Such a Quantum Error-Correction Code, or simply *code*, is commonly described by the parameters $[[n, k, d]]$, adding the code distance d , the minimum weight of a non-trivial undetectable error, to the number of physical qubits n and the number of logical qubits k . Unfortunately, there are multiple definitions of what constitutes a QECC, and thus, what properties are associated with it [24]. Some examples of those definitions are:

- **Def. 1:** A QECC is specified by the encoding isometry between $\mathcal{H}^{\otimes k}$ and $\mathcal{H}^{\otimes n}$, together with the set of all correctable errors.
- **Def. 2:** A QECC is specified by the encoding isometry and its associated code space, which is the set of all valid codewords.
- **Def. 3:** A QECC is specified solely by the code space.

The thesis adopts the second definition, as both the encoding isometry and the code space are relevant for the analysis. Any additional assumptions and properties required by the equivalence notions, which are introduced later, will be stated explicitly.

2.2 Stabilizer Formalism

The stabilizer formalism [23] is a useful framework for representing and manipulating QECC, and it applies to a large class of codes known as *stabilizer codes*. The formalism uses group theory to describe and analyze the code space and the encoding and decoding processes of a code, as in some cases it may be more straightforward to work with the operators in the stabilizer group than with the code space itself.

Central in this framework are the identity and Pauli matrices

$$I = \sigma_0 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad X = \sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Y = \sigma_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \quad Z = \sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

with their relevant properties—especially their commutation and anti-commutation relations—which are summarized by the following equations:

$$\forall i \in \{1, 2, 3\} : \sigma_i^2 = I \quad \text{and} \quad \forall i, j \in \{1, 2, 3\} : \sigma_i \sigma_j = (-1)^{1-\delta_{ij}} \sigma_j \sigma_i$$

With this in place, the n -qubit Pauli group $\mathcal{P}_n$ is defined, which consists of all n -fold tensor products of Pauli matrices and the identity, along with the global phases ± 1 and $\pm i$:

$$\forall P \in \mathcal{P}_n : P = \alpha \bigotimes_{j=1}^n \sigma_{p_j} \quad \text{with} \quad \alpha \in \{1, -1, i, -i\}, \quad p_j \in \{0, 1, 2, 3\}$$

The Pauli operation on the j -th qubit of a Pauli string P is denoted as P_j . The weight

$$\text{wt}(P) = \sum_{j=1}^n (P_j \neq I)$$

of a Pauli string P is the number of non-identity factors in its tensor product decomposition. Considering the mentioned properties of the relevant matrices, the Pauli group constitutes an algebraic group [36] with matrix multiplication as the group operation. As an example, the single-qubit Pauli group [24] is given by:

$$\mathcal{P}_1 = \{\pm I, \pm iI, \pm X, \pm iX, \pm Y, \pm iY, \pm Z, \pm iZ\}$$

Attention now turns to the principal object of the stabilizer formalism, a stabilizer group $\mathcal{S}$. It is an *abelian* subgroup of the *Pauli group* with a *non-trivial* common $+1$ eigenspace of all its elements, which are called *stabilizers*. To fulfill these requirements, a stabilizer group must satisfy the following properties [24, 39]:

- (i) $\forall s \in \mathcal{S} : s \in \mathcal{P}_n \quad \rightarrow$ subgroup of the Pauli group
- (ii) $\forall s_1, s_2 \in \mathcal{S} : s_1 s_2 = s_2 s_1 \quad \rightarrow$ (commutative) abelian group
- (iii) $-I \notin \mathcal{S} \quad \rightarrow$ non-trivial common $+1$ eigenspace

By the closure property of a group, each product of stabilizer elements is again a stabilizer. Thus, the whole stabilizer group can be generated by combining a minimal set of r independent *generators*:

$$\mathcal{S} = \langle g_1, g_2, \dots, g_r \rangle$$

Property (iii) is essential for ensuring the non-triviality of the common +1 eigenspace, which is crucial as a stabilizer group characterizes the code space of a QECC as exactly this vector space. Accordingly, the code space $\mathcal{C}$ consists of all quantum states that the stabilizer group elements stabilize [39]:

$$\mathcal{C} = \{|\psi\rangle \in \mathcal{H}^{\otimes n} \mid \forall s \in \mathcal{S} : s|\psi\rangle = |\psi\rangle\}$$

During the syndrome extraction step of QEC, the stabilizer generators are measured as observables on the quantum state $|\psi_P\rangle'$, and thus project the potentially erroneous state into one of the eigenspaces of that generator. The combined measurement outcomes of all generators form the syndrome information. Thus, the complete projector onto the code space is a combination of +1 eigenspace projectors of each generator [24]:

$$\Pi_{\mathcal{C}} = \frac{1}{2^r} \prod_{i=1}^r (I + g_i) \quad (2.1)$$

Each stabilizer generator imposes a constraint on the code space; the more independent generators $\mathcal{S}$ has, the smaller the stabilized code space $\mathcal{C}$ becomes, and thus the fewer logical qubits can be encoded with a fixed number of physical qubits. In the case of n independent generators, the code space collapses to a single quantum state, and no logical qubits can be encoded. This special case is called a stabilizer state [48].

Not only can the code space of a code be defined using the stabilizer formalism, but also its encoding map can be characterized using related elements. The *centralizer* $\mathcal{C}(\mathcal{S})$ of the stabilizer group is the set of all operators that commute with the stabilizer group [24]:

$$\mathcal{C}(\mathcal{S}) = \{p \in \mathcal{P}_n \mid \forall s \in \mathcal{S} : ps = sp\}$$

Operators in $\mathcal{C}(\mathcal{S})$ preserve the code space. Effectively, stabilizers and their scalar multiples act trivially on encoded states, whereas all other elements in $\mathcal{C}(\mathcal{S})$ induce non-trivial logical operations on the code space. Such operations, mapping one valid codeword to another, are undetectable by stabilizer measurements. In Figure 2.1, this corresponds to the case of a failed error correction, i.e., $|\psi_L\rangle' \neq |\psi_L\rangle$. The distance of a code is the minimum weight of such an operator, or equivalently, the minimum number of qubits that must be affected to cause an undetectable logical error [24]:

$$d = \min \{ \text{wt}(l) \mid l \in \mathcal{C}(\mathcal{S}) \wedge \forall \alpha \in \{1, -1, i, -i\} : \alpha l \notin \mathcal{S} \}$$

Since multiplication by a stabilizer or a global phase does not change the represented logical operation, a set of $2k$ independent representatives in $\mathcal{C}(\mathcal{S}) \setminus \mathcal{S}$ can be chosen, that generate the quotient $\mathcal{C}(\mathcal{S})/\mathcal{S}$, disregarding global phases. It is possible to structure those representatives in ordered pairs, corresponding to a single X or Z error on a single logical qubit, where $\overline{X}_i$ represents an X error on the i -th *logical* qubit. While the labeling of those representatives is arbitrary, they need to satisfy the following commutation relations [23, 24, 39]:

$$\begin{aligned} \text{(i)} \quad & \forall i \in \{1, \dots, k\} : & \overline{X}_i \overline{Z}_i &= -\overline{Z}_i \overline{X}_i \\ \text{(ii)} \quad & \forall i, j \in \{1, \dots, k\}, i \neq j, \quad L, M \in \{X, Z\} : & \overline{L}_i \overline{M}_j &= \overline{M}_j \overline{L}_i \end{aligned} \quad (2.2)$$

In the following, such ordered representatives satisfying those requirements will be called a logical framework, which implicitly defines an encoding map for the code.

2.3 Representation of Stabilizer Codes

For automated equivalence checking, a suitable representation of the input codes is required, as working with abstract mathematical objects is not practicable. Especially in the context of equivalence checking, it is important to have a representation that enables efficient manipulation and comparison of codes, while minimizing irrelevant differences arising from arbitrary representational choices. The *stabilizer formalism* provides a convenient framework for defining stabilizer codes, but there are several ways to represent them within it. The following sections introduce the representations used in this thesis.

2.3.1 Pauli Strings

A straightforward way to represent a stabilizer code is to use the generators of its stabilizer group as Pauli strings:

$$\mathcal{S} = \langle g_1, g_2, \dots, g_r \rangle, \quad \forall i \in \{1, \dots, r\} : g_i = \{\pm 1, \pm i\} \{I, X, Y, Z\}^{\otimes n}$$

From this representation, the number of physical qubits n , the number of logical qubits $k = n - r$, the stabilizer group $\mathcal{S}$, the code space $\mathcal{C}$, and the code distance d can be derived using multiplication and commutation of Pauli operators.

The $2k$ orthogonal logical operators satisfying the commutation relations in Equation (2.2) can be represented by ordered pairs of Pauli strings

$$(\overline{X}_1, \overline{Z}_1), (\overline{X}_2, \overline{Z}_2), \dots, (\overline{X}_k, \overline{Z}_k)$$

fixing the encoding as well.

Throughout this thesis, selected concepts and algorithms are illustrated using explicit examples. Those examples are distinctly depicted in such blocks.

In this section, the different code representations are demonstrated using a $[[4, 1, 2]]$ code $\mathcal{C}$ as a running example. Its stabilizer group $\mathcal{S}$ and logical operators are represented by the following Pauli strings:

$$\begin{aligned} \mathcal{S} &= \langle XXXX, ZZII, IIZZ \rangle \\ (\overline{X}_1, \overline{Z}_1) &= (ZIZI, IIXX) \end{aligned}$$

This representation is not unique for a given code, as it depends on the choice of generators for the stabilizer group and representatives for the logical operators. Different sets of generators can produce the same stabilizer group, and the logical operators can be chosen differently, as long as they satisfy the commutation relations. Consequently, the same code admits multiple valid Pauli-string representations. Furthermore, Pauli strings are less amenable to efficient algebraic manipulation than the other representations introduced below.

2.3.2 Binary Symplectic Representation

A well-established and useful way to describe stabilizer codes is the *binary symplectic representation* [24], which maps Pauli strings to binary vectors. It thereby translates many manipulations and comparisons of stabilizer codes into linear algebra over $\mathbb{F}_2$. Thus, all arithmetic operations in this section are performed over this field.

Fundamentals of the Symplectic Representation

Representation Map This representation uses the fact that there exists an isomorphism between the Pauli group modulo phases $\mathcal{P}_n/\{\pm I, \pm iI\}$ with

$$\forall P \in \mathcal{P}_n/\{\pm I, \pm iI\} : P = \bigotimes_{j=1}^n \sigma_{p_j} \quad \text{with} \quad p_j \in \{0, 1, 2, 3\}$$

and the vector space $\mathbb{F}_2^{2n}$. It is apparent that when considering $\mathcal{P}_n$, the mapping would not be injective—thus no isomorphism—as the information about the global phases is lost. Whenever the signs of stabilizer generators are relevant, they must be stored separately.

In this mapping, each Pauli operator is represented as a *binary symplectic vector* of dimension $2n$, with the first n entries representing the X components and the last n entries representing the Z components of the Pauli operator:

$$\phi : \mathcal{P}_n \rightarrow \mathbb{F}_2^{2n}, \quad P \mapsto (a \mid b) = (a_0 \ a_1 \ \dots \ a_{n-1} \mid b_0 \ b_1 \ \dots \ b_{n-1}), \quad (a_j \mid b_j) = \begin{cases} (0|0) & \text{if } P_j = I \\ (1|0) & \text{if } P_j = X \\ (0|1) & \text{if } P_j = Z \\ (1|1) & \text{if } P_j = Y \end{cases}$$

Operations The operation-preserving nature of the isomorphism is visible in the fact that matrix multiplication of Pauli operators corresponds to the addition of symplectic vectors in $\mathbb{F}_2$:

$$P, R \in \mathcal{P}_n, \quad \phi(PR) = \phi(P) + \phi(R)$$

The other useful, eponymous operation is the *symplectic inner product* of two symplectic vectors

$$\langle (a \mid b), (a' \mid b') \rangle_{\text{sp}} = (a \mid b) \begin{bmatrix} 0 & I \\ I & 0 \end{bmatrix} (a' \mid b')^\top = a \cdot b' + a' \cdot b = \sum_{i=0}^{n-1} (a_i b'_i + a'_i b_i)$$

which captures the commutation relation of the corresponding Pauli operators, namely

$$P, R \in \mathcal{P}_n : PR = (-1)^{\langle \phi(P), \phi(R) \rangle_{\text{sp}}} RP.$$

The symplectic inner product is useful, as many commutation questions, which are essential in the stabilizer formalism, can be easily checked by computing the symplectic inner product of the corresponding symplectic vectors.

Symplectic Representation for Codes

To extend the symplectic representation of Pauli operators to a stabilizer code C , each stabilizer generator is represented as a symplectic vector and stacked vertically to form a tableau, called the *symplectic check matrix* or *stabilizer tableau*. The symplectic representations of the logical X and Z operators are ordered according to their pairs and likewise stacked to obtain two logical symplectic matrices:

$$S(C) := \begin{bmatrix} \phi(g_1) \\ \vdots \\ \phi(g_r) \end{bmatrix} \in \mathbb{F}_2^{r \times 2n} \quad L_x(C) := \begin{bmatrix} \phi(\bar{X}_1) \\ \vdots \\ \phi(\bar{X}_k) \end{bmatrix} \in \mathbb{F}_2^{k \times 2n} \quad L_z(C) := \begin{bmatrix} \phi(\bar{Z}_1) \\ \vdots \\ \phi(\bar{Z}_k) \end{bmatrix} \in \mathbb{F}_2^{k \times 2n}$$

Code parameters n , r , and k can be derived from the dimensions and rank of the stabilizer tableau, and the full stabilizer group can be generated by computing its row space. Similarly, d and the code space can be derived from the stabilizer tableau, provided that the loss of phase information is accounted for, whereas the logical symplectic matrices fix the encoding.

The rest of the thesis will primarily utilize this representation. For later use, we introduce the following projection maps, which extract the X - and Z -parts of a symplectic matrix:

$$\forall M = \left[\begin{array}{ccc|ccc} a_{00} & \dots & a_{0,n-1} & b_{00} & \dots & b_{0,n-1} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ a_{m0} & \dots & a_{m,n-1} & b_{m0} & \dots & b_{m,n-1} \end{array} \right] \in \mathbb{F}_2^{m \times 2n} : M^{(x)} = \begin{bmatrix} a_{00} & \dots & a_{0,n-1} \\ \vdots & \ddots & \vdots \\ a_{m0} & \dots & a_{m,n-1} \end{bmatrix}, M^{(z)} = \begin{bmatrix} b_{00} & \dots & b_{0,n-1} \\ \vdots & \ddots & \vdots \\ b_{m0} & \dots & b_{m,n-1} \end{bmatrix} \in \mathbb{F}_2^{m \times n}$$

The following symplectic matrices represent code C from above:

$$S(C) = \left[\begin{array}{cccc|cccc} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \end{array} \right] \quad L_x(C) = [0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0] \quad L_z(C) = [0 \ 0 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0]$$

Similarly to the Pauli string representation, the exact symplectic representation is not unique for a given code, but depends on the choice of generators for the stabilizer group and the choice of representatives for the logical operators.

2.3.3 Quaternary Representation

The *quaternary representation* extends the symplectic representation, allowing for a more compact depiction of Pauli operators and stabilizer codes, but inherits the symplectic representation's limitations regarding phase information.

Fundamentals of the Quaternary Representation

Representation Map Instead of binary vectors, Pauli operators are expressed as vectors in $\mathbb{F}_4^n$. The elements of this field are $\{0, 1, \omega, \omega^2\}$ [3, 12, 24]:

$$\varphi : \mathcal{P}_n \rightarrow \mathbb{F}_4^n, P \mapsto v = (v_0 \ v_1 \ \dots \ v_{n-1}), v_i = \begin{cases} 0 & \text{if } P_i = I \\ 1 & \text{if } P_i = X \\ \omega & \text{if } P_i = Z \\ \omega^2 & \text{if } P_i = Y \end{cases}$$

Operations In this field, the elements 0 and 1 are the additive and multiplicative identities, respectively, with $\omega + 1 = \omega^2$. These identities determine the addition and multiplication rules in $\mathbb{F}_4$. Conjugation is defined by $\bar{v} = v^2$, and the *trace* by $\text{tr}(v) = v + \bar{v}$. The operation-preserving nature of the isomorphism can be found in the correspondence of matrix multiplication of Pauli operators to addition of vectors in $\mathbb{F}_2$. Consequently, the quaternary representation does not utilize the whole field, as it does not preserve multiplication—equivalently, it is only $\mathbb{F}_2$ -additive:

$$P, R \in \mathcal{P}_n, \quad \varphi(PR) = \varphi(P) + \varphi(R)$$

Similar to the binary symplectic product, the commutation relation of two Pauli operators can be expressed in terms of the *trace inner product* in $\mathbb{F}_4$:

$$u * v = \text{tr}(u \cdot \bar{v}) = \sum_{i=0}^{n-1} (u_i \bar{v}_i + \bar{u}_i v_i)$$

Quaternary Representation for Codes

Equivalently to the symplectic representation, a stabilizer group and logical operators can be represented as tableaus in $\mathbb{F}_4^{r \times n}$ using the quaternary representation.

In the quaternary representation, the code C is described as:

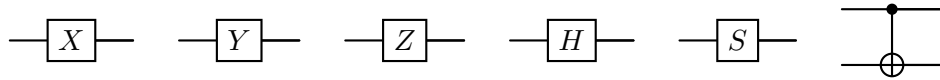
$$S(C) = \begin{bmatrix} 1 & 1 & 1 & 1 \\ \omega & \omega & 0 & 0 \\ 0 & 0 & \omega & \omega \end{bmatrix} \quad L_x(C) = \begin{bmatrix} \omega & 0 & \omega & 0 \end{bmatrix} \quad L_z(C) = \begin{bmatrix} 0 & 0 & 1 & 1 \end{bmatrix}$$

2.3.4 Quantum Circuits

Fundamentals of Quantum Circuits

A *quantum circuit* [36] with n wires describes an n -qubit unitary operation $U \in \mathcal{U}_{2^n}$ mapping an input quantum state to an output state. Similar to classical circuits, quantum circuits are composed of gates and wires. Gates describe elementary unitary operations, while wires represent qubits and connect the gates. Quantum circuits can be described at various levels of abstraction, depending on which aspects of their structure or behavior are relevant.

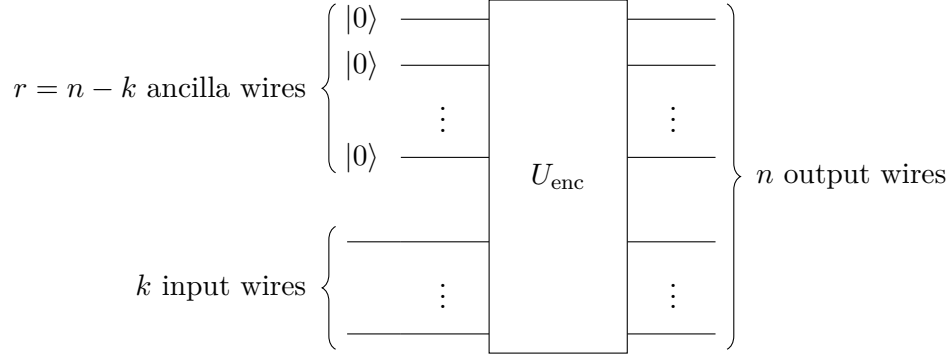
Within the scope of this thesis, only a small set of elementary quantum gates is relevant. The Pauli operators, together with the Hadamard (H), phase (S), and controlled-NOT ($CNOT$) gates, are introduced below, as they play an important role in the following sections:



Quantum Circuits for Codes

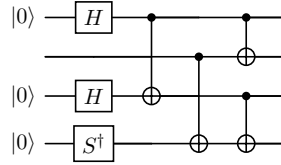
Their interpretation as maps makes quantum circuits a natural representation of the encoding and decoding processes. Here we are primarily interested in *encoding circuits* which implement

the map of a logical state $|\psi_L\rangle$ to the corresponding encoded state $|\psi_P\rangle$:



This isometry $\mathcal{H}^{\otimes k} \rightarrow \mathcal{H}^{\otimes n}$ is a distinct way of describing a code, as the code space $\mathcal{C}$ is the image of the circuit's encoding map. Unlike previous representations, it does not directly specify the stabilizer group.

The following circuit implements an encoding circuit for the code $\mathcal{C}$, obtained by applying a circuit extraction from [52], whereas the second wire is used as input:



It is important to note that the encoding circuit is not unique, as various gate identities may be used to transform one circuit into another without changing the overall unitary operation U_{enc} —thus, preserving the isometry—and preparing the same code. In contrast, precomposing the encoding map with a unitary operation on the logical input qubits does not change its image, since such an operation maps $\mathcal{H}^{\otimes k}$ onto itself. Effectively, this transformation modifies the unitary operation U'_{enc} of the circuit and consequently the isometry, but does not change the represented code space.

2.3.5 ZX Calculus

Fundamentals of ZX Diagrams

The *ZX calculus* [46] is a graphical formalism with application in many areas of quantum computing, including QEC. It provides a diagrammatic language for representing and manipulating quantum states and operations.

Components A *ZX diagram* consists of wires and nodes. Its nodes are either *spiders* representing quantum states and operations, or *Hadamard nodes*. The elementary components of ZX diagrams are defined below, and everything else can be expressed by connecting these components using matrix multiplication and tensor products:

Z-spider with an arbitrary number of in- and outputs, and phase α :

$$\text{Z-spider}(\alpha) := |0 \cdots 0\rangle \langle 0 \cdots 0| + e^{i\alpha} |1 \cdots 1\rangle \langle 1 \cdots 1|$$

X-spider with an arbitrary number of in- and outputs, and phase α :

$$\text{X-spider}(\alpha) := |+\cdots+\rangle \langle +\cdots+| + e^{i\alpha} |-\cdots-\rangle \langle -\cdots-|$$

For both spider types we use the convention that a spider with a phase of $\alpha = 0$ can be left empty, which is referred to as a phase-free spider.

A Hadamard gate translates to a node with only one input and one output. It can be interpreted as property of the wire, thus, is also represented as a dashed line:

$$\text{Hadamard gate} = \text{dashed line} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

The following transformations of the wires themselves are also possible in the ZX calculus, and effectively simulate the *Bell* state and a *swap* gate:

$$\left(\begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix} \right) = \begin{pmatrix} 1 & 0 & 0 & 1 \end{pmatrix} \quad \text{Crossing} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

An important property of ZX diagrams is that only the connectivity of their components matters, not their precise geometric arrangement. This is a key feature of the ZX calculus, which allows for more flexible manipulation compared to quantum circuits.

For the purposes of this thesis, any relevant quantum circuit can be transformed into a ZX diagram [46] by replacing the gates with their corresponding ZX-calculus components, and connecting them according to the structure of the circuit. One necessary component that might not be as straightforward to construct is the *CNOT* gate:

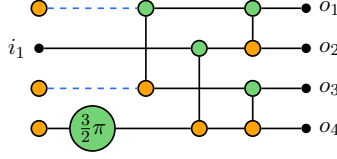
$$\text{CNOT gate} = \text{CNOT gate with spiders}$$

A major strength of the ZX calculus is its collection of rewrite rules, which enable ZX diagrams to be transformed and simplified while preserving their semantics, as can be verified in [46].

ZX Calculus for Codes

ZX diagrams can represent QECC through their encoding maps. Compared to encoding circuits, their graphical structure offers more intuitive ways to manipulate and simplify these representations. Other purposes of ZX diagrams in the context of QEC include the construction of normal forms [32] and the representation of graph states, which will be discussed in more detail later.

The following ZX diagram represents the encoding circuit for the quantum code C:



2.4 CSS Codes

CSS codes form a subclass of stabilizer codes whose stabilizers can be chosen to be exclusively of either X -type or Z -type. In the symplectic representation, this property allows the stabilizer tableau to be written in a block-diagonal form, whose blocks satisfy the orthogonality condition:

$$S(C) = \begin{bmatrix} H_x(C) & 0 \\ 0 & H_z(C) \end{bmatrix} \in \mathbb{F}_2^{r \times 2n} \text{ with } H_x(C) \in \mathbb{F}_2^{r_x \times n}, H_z(C) \in \mathbb{F}_2^{r_z \times n} : H_x(C) \cdot H_z(C)^T = 0$$

where $r = r_x + r_z$. The orthogonality condition ensures the stabilizer group's commutation requirement.

CSS codes are closely related to classical linear error-correction codes, as $H_x(C)$ and $H_z(C)$ can be interpreted as mutually orthogonal parity-check matrices of classical codes. By treating bit-flip and phase-flip errors separately, CSS codes are generally easier to construct, analyze, and implement than general stabilizer codes [36]. Their additional structure also admits simpler graphical representations [32], enabling specialized algorithms that are unavailable for general stabilizer codes. Moreover, several widely used QECC families, such as topological quantum codes [6] or bivariate bicycle codes [10], naturally belong to the CSS class.

2.5 Clifford Operations

2.5.1 Clifford Group

Another class of unitary operations of particular relevance to this thesis is the n -qubit *Clifford group* $\widehat{\mathcal{C}}_n$. It is defined as the *normalizer* of the Pauli group in the unitary group; that is, mapping one element of the Pauli group to another under conjugation:

$$\widehat{\mathcal{C}}_n := \{U \in \mathcal{U}_{2^n} \mid \forall P \in \mathcal{P}_n : UPU^\dagger \in \mathcal{P}_n\}$$

As all elements of the form $e^{i\theta}I$ are included in this group, it is an infinite group. However, since multiplication by a global phase does not affect the action of a unitary operator, we disregard those phases. Therefore, we only consider the restricted Clifford group modulo phases, which we will refer to as *Clifford group* throughout the thesis:

$$\mathcal{C}_n = \widehat{\mathcal{C}}_n / \mathcal{U}_1 \text{ with } \mathcal{U}_1 = \{e^{i\theta}I \mid \theta \in \mathbb{R}\}$$

This quotient is finite and contains $2^{n^2+2n} \prod_{j=1}^n (4^j - 1)$ elements [12].

Up to global phases, the Clifford group is exactly the finite group of unitaries that are generated using the following three *Clifford Operations* [24]:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} \quad CNOT = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Their action by conjugation on the Pauli operators is summarized in Table 2.2. For an n -qubit Pauli operator P , when not clear from context, subscripts indicate the qubit or qubits on which an operation acts, while the identity operation on the remaining qubits is assumed.

P	I	X	Y	Z
$HPH^\dagger$	I	Z	$-Y$	X
$SPS^\dagger$	I	Y	$-X$	Z

(a) Single-qubit H and S .

$c \backslash t$	I	X	Y	Z
I	II	IX	ZY	ZZ
X	XX	XI	YZ	$-YY$
Y	YX	YI	$-XZ$	XY
Z	ZI	ZX	IY	IZ

(b) $CNOT_{0 \rightarrow 1}$ on control 0 to target 1.

Figure 2.2: Conjugation of Pauli operators by Clifford operators.

Because this thesis primarily uses the binary symplectic representation, it is useful to examine the effect a Clifford operation has on a symplectic vector. The action of a Clifford operation on a Pauli operator can be computed by multiplying the symplectic vector of the Pauli operator with the corresponding transformation matrix of the Clifford operation. Let $\phi(P) = (a \mid b) \in \mathbb{F}_2^{2n}$ represent the n -qubit Pauli operator P . For every Clifford operation C , there exists a symplectic matrix $Q_C \in \text{Sp}(2n, \mathbb{F}_2)$ such that

$$\phi(CPC^\dagger) = (a \mid b)Q_C$$

This correspondence follows from the homomorphism from the Clifford group to the binary symplectic group [24], which is the group of all *invertible* linear transformations on symplectic vectors that *preserve* the symplectic inner product:

$$\forall Q \in \text{Sp}(2n, \mathbb{F}_2), \quad v, w \in \mathbb{F}_2^{2n} : \quad \langle v \mid w \rangle_{\text{sp}} = \langle vQ \mid wQ \rangle_{\text{sp}}$$

For the Clifford generators, this yields the following transformations of the relevant symplectic components:

$$\begin{aligned} H_0 &\cong \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} : & (a_0 \mid b_0) &\mapsto (b_0 \mid a_0) \\ S_0 &\cong \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} : & (a_0 \mid b_0) &\mapsto (a_0 \mid a_0 + b_0) \\ CNOT_{0 \rightarrow 1} &\cong \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} : & (a_0 \mid a_1 \mid b_0 \mid b_1) &\mapsto (a_0 \mid a_0 + a_1 \mid b_0 + b_1 \mid b_1) \end{aligned} \quad (2.3)$$

2.5.2 Local Clifford Operations

This thesis is primarily concerned with local Clifford (LC) operations [48], which are tensor products of single-qubit Clifford operations $C = C_1 \otimes C_2 \otimes \cdots \otimes C_n$ with $C_i \in \mathcal{C}_1 =$

$\{I, H, S, HS, SH, HSH\}$. Thus, in the binary symplectic representation, the transformation matrix has the form of a block matrix:

$$Q_C = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \in \mathbb{F}_2^{2n \times 2n} \text{ with } \begin{aligned} A &= \text{diag}(a_{00}, a_{11}, \dots, a_{n-1, n-1}), \\ B &= \text{diag}(b_{00}, b_{11}, \dots, b_{n-1, n-1}), \\ C &= \text{diag}(c_{00}, c_{11}, \dots, c_{n-1, n-1}), \\ D &= \text{diag}(d_{00}, d_{11}, \dots, d_{n-1, n-1}), \end{aligned} \quad Q_{C_i} = \begin{bmatrix} a_{ii} & b_{ii} \\ c_{ii} & d_{ii} \end{bmatrix} \in \text{Sp}(2, \mathbb{F}_2) \quad (2.4)$$

Consequently, each submatrix Q_{C_i} acts independently on the symplectic vector components of the corresponding qubit i . As the group $\text{Sp}(2, \mathbb{F}_2)$ is generated only by H and S operations, there are effectively only six different possible transformations for each qubit i :

$$\begin{aligned} I_i &\cong \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} : (a_i|b_i) \mapsto (a_i|b_i) & (HS)_i &\cong \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} : (a_i|b_i) \mapsto (a_i + b_i|a_i) \\ H_i &\cong \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} : (a_i|b_i) \mapsto (b_i|a_i) & (SH)_i &\cong \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} : (a_i|b_i) \mapsto (b_i|a_i + b_i) \\ S_i &\cong \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} : (a_i|b_i) \mapsto (a_i|a_i + b_i) & (HSH)_i &\cong \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} : (a_i|b_i) \mapsto (a_i + b_i|b_i) \end{aligned} \quad (2.5)$$

The noteworthy isomorphism $\text{Sp}(2, \mathbb{F}_2) \cong \mathcal{S}_3$, with $\mathcal{S}_n$ being the symmetric group on n elements, provides another interpretation of these six transformations as a permutation of the Pauli operators. This observation will be useful in subsequent chapters.

2.6 Equivalence Notions

The core of this thesis is the study of equivalence notions for stabilizer codes. For this purpose, we want to extend our understanding of equivalence to allow different transformations that preserve code equivalence. Effectively, we want to explore equivalence definitions that allow a broader set of codes to be considered equivalent to a given code.

The following notions are independent of representation-specific choices. We therefore define them in terms of code spaces with their associated stabilizer generators. For all equivalence notions in the thesis, the choice of logical operators, and hence the specific encoding map, is not relevant. Equivalence is defined solely based on the code space.

In this section, the different equivalence notions are demonstrated using the code C from above, with the stabilizer group $\mathcal{S}$ and code space $\mathcal{C}$:

$$\begin{aligned} \mathcal{S} &= \langle XXXX, ZZII, IIZZ \rangle \\ &= \{IIII, XXXX, ZZII, IIZZ, YYXX, XXYY, ZZZZ, YYYY\} \\ \mathcal{C} &= \text{span}\{|0_L\rangle, |1_L\rangle\} = \text{span}\left\{\frac{1}{\sqrt{2}}(|0000\rangle + |1111\rangle), \frac{1}{\sqrt{2}}(|0011\rangle + |1100\rangle)\right\} \end{aligned}$$

2.6.1 Row Equivalence

Two stabilizer codes C and C' are considered row-equivalent, or simply equivalent, if and only if they share the exact same code space, which is precisely the case when they have the

same stabilizer group. Row equivalence, therefore, abstracts from the particular choice of stabilizer generators. This means the two codes can be represented by different sets of stabilizer generators, but they generate the same stabilizer group and thus have the same code space. Row equivalence is denoted as

$$C \sim C' \iff C = C'$$

We define a new code C_{RS} with the following stabilizer group and code space:

$$\begin{aligned} \mathcal{S}_{RS} &= \langle \textcolor{red}{YYXX}, \textcolor{red}{ZZZZ}, \textcolor{red}{YYYY} \rangle \\ &= \{IIII, XXXX, ZZII, IIZZ, YYXX, XXYY, ZZZZ, YYYY\} \\ \mathcal{C}_{RS} &= \text{span}\left\{\frac{1}{\sqrt{2}}(|0000\rangle + |1111\rangle), \frac{1}{\sqrt{2}}(|0011\rangle + |1100\rangle)\right\} = \mathcal{C} \end{aligned}$$

It is easy to see that C and C_{RS} have the same stabilizer group and code space. The two representations differ only in their choice of stabilizer generators, shown in red. Thus, they are equivalent under this notion: $C \sim C_{RS}$.

2.6.2 Permutation Equivalence

Two codes C and C' are considered permutation-equivalent if and only if their code spaces are related by a permutation of the physical qubits. Equivalently, their stabilizer groups differ by a common permutation of the Pauli operators in their elements. This notion disregards both the ordering of the physical qubits and the particular choice of stabilizer generators. Permutation equivalence, with $\mathcal{S}_n$ denoting the symmetric group on n elements, is defined as

$$C \sim_P C' \iff \exists \pi \in \mathcal{S}_n : \mathcal{C} = \{ \pi(|\psi\rangle) \mid |\psi\rangle \in \mathcal{C}' \} =: \pi(\mathcal{C}')$$

We define another code C_P with stabilizer group and code space:

$$\begin{aligned} \mathcal{S}_P &= \langle \textcolor{red}{XXXX}, \textcolor{red}{ZIZI}, \textcolor{red}{IZIZ} \rangle \\ &= \{IIII, \textcolor{red}{XXXX}, \textcolor{red}{ZIZI}, \textcolor{red}{IZIZ}, \textcolor{red}{YXYX}, \textcolor{red}{XYXY}, \textcolor{red}{ZZZZ}, \textcolor{red}{YYYY}\} \\ \mathcal{C}_P &= \text{span}\left\{\frac{1}{\sqrt{2}}(|\textcolor{red}{0000}\rangle + |\textcolor{red}{1111}\rangle), \frac{1}{\sqrt{2}}(|\textcolor{red}{0101}\rangle + |\textcolor{red}{1010}\rangle)\right\} = \pi(\mathcal{C}) \end{aligned}$$

It is visible, that applying the permutation $\pi = (\textcolor{red}{0213})$ to every element in $\mathcal{S}$ yields $\mathcal{S}_P$. Equivalently, we recognize $\mathcal{C}_P$ as the code space of C obtained by applying the qubit permutation π to each state. Consequently, the two codes are equivalent under this notion: $C \sim_P C_P$.

2.6.3 Local Clifford Equivalence

Two codes C and C' are regarded local-Clifford-equivalent if and only if they have the same code space up to a LC operation on the physical qubits. This is precisely the case when their stabilizer groups differ by a common LC operation on their elements. This equivalence relation disregards the specific stabilizer generators and any output LC operation, and is written as

$$C \sim_{LC} C' \iff \exists U \in \mathcal{C}_1^{\otimes n} : \mathcal{C} = \{ U |\psi\rangle \mid |\psi\rangle \in \mathcal{C}' \} =: U \cdot \mathcal{C}'$$

We define one final code C_{LC} with the following stabilizer group and code space:

$$\begin{aligned}\mathcal{S}_{LC} &= \langle ZXXX, XZII, IIZZ \rangle \\ &= \{IIII, ZXXX, XZII, IIZZ, YYXX, ZXYX, XZZZ, YYYY\} \\ \mathcal{C}_{LC} &= \text{span}\left\{\frac{1}{2}(|0\rangle + |1\rangle)|000\rangle + (|0\rangle - |1\rangle)|111\rangle, \frac{1}{2}(|0\rangle + |1\rangle)|011\rangle + (|0\rangle - |1\rangle)|100\rangle\right\} \\ &= U \cdot \mathcal{C}\end{aligned}$$

It is obvious that $\mathcal{S}_{LC}$ is obtained by conjugating each element in $\mathcal{S}$ by the LC $U = H \otimes I \otimes I \otimes I$.

Consequently, the code space $\mathcal{C}_{LC}$ is the image of $\mathcal{C}$ under the LC operation U , so the two codes are equivalent under this notion: $C \sim_{LC} C_{LC}$.

2.7 Algorithmic Tools

This section introduces the mathematical and computational tools utilized by the algorithms developed in this thesis.

2.7.1 Graphs

A *undirected graph* G is a pair $(\mathcal{V}, \mathcal{E})$ of a set of vertices $\mathcal{V}$ and edges $\mathcal{E} \subseteq \{\{u, v\} \mid u, v \in \mathcal{V}\}$.

Terminology The *neighborhood* of a vertex is the set of vertices adjacent to v :

$$N(v) = \{u \in \mathcal{V} \mid \{u, v\} \in \mathcal{E}\}.$$

The *adjacency matrix* of a graph is described by the matrix $\Gamma = (a_{ij})$ with entries

$$a_{ij} = \begin{cases} 1 & \text{if } \{i, j\} \in \mathcal{E}, \\ 0 & \text{otherwise} \end{cases}.$$

A *coloring* of a graph is a map of its vertices—and sometimes edges $\mathcal{E}_c \subseteq \mathcal{E}$ —to $\mathbb{N}$, representing the set of colors:

$$c : \mathcal{V} \cup \mathcal{E}_c \rightarrow \mathbb{N}.$$

A graph equipped with such a coloring is called a colored graph. A graph G is called *bipartite* if its vertex set can be partitioned into two disjoint sets such that each edge connects only vertices from different sets, equivalently:

$$\exists c : \mathcal{V} \rightarrow \{0, 1\} : \quad \forall u, v \in \mathcal{V} : \{u, v\} \in \mathcal{E} \implies c(u) \neq c(v)$$

(Colored) Graph Isomorphism Two graphs $G = (\mathcal{V}, \mathcal{E})$ and $G' = (\mathcal{V}', \mathcal{E}')$ are called *isomorphic*, denoted $G \cong G'$, if there exists a bijection $\gamma : \mathcal{V} \rightarrow \mathcal{V}'$ such that

$$\forall u, v \in \mathcal{V} : \quad \{u, v\} \in \mathcal{E} \iff \{\gamma(u), \gamma(v)\} \in \mathcal{E}'.$$

If they are colored graphs with colorings c and c' , then γ has to preserve the coloring

$$\forall v \in \mathcal{V} : c(v) = c'(\gamma(v)).$$

The corresponding decision problems relevant to this thesis are defined as follows:

- ▷ GI: Given two graphs G and G' , determine whether they are isomorphic.
- ▷ CGI: Given two colored graphs G and G' , determine whether they are isomorphic.

2.7.2 Graph States

Graph states [48], as a special class of stabilizer states, are more directly related to QECC again. Since they are used as an algorithmic tool in this thesis, their essential properties are briefly reviewed. They are defined as stabilizer states that can be characterized by a simple undirected graph $G = (\mathcal{V}, \mathcal{E})$, whose vertices correspond to qubits, and edges represent their entanglement.

The n stabilizer generators of a graph state $|G\rangle$ can be directly obtained from the graph structure:

$$\mathcal{S} = \langle \{ K_v \mid v \in \mathcal{V} \} \rangle, \quad K_v = X_v \cdot \prod_{u \in N(v)} Z_u$$

Consequently, its symplectic stabilizer tableau has the form $[I \mid \Gamma]$ where Γ is the adjacency matrix of the graph. In the ZX calculus, a graph state can be represented as a ZX diagram with one phase-free Z -spider with a single output for each vertex, and H -wires connecting the spiders according to the graph structure.

Stabilizer States and Graph States A central result used in this thesis states that every stabilizer state is LC-equivalent to a graph state [48]. Thus, for every stabilizer state $|\psi\rangle$, there exist a graph state $|G\rangle$ and a LC operation C such that $|\psi\rangle = C|G\rangle$.

CSS States and Graph States A key connection between graph states and CSS states—which are stabilizer states with n exclusively X -type or Z -type stabilizer generators—establishes that every bipartite graph state is LC-equivalent to a CSS state, and every CSS state admits an LC-equivalent representation as a bipartite graph state [13].

2.7.3 Matroids

Other than graphs, matroids are perhaps less widely known combinatorial structures that generalize the notion of independence, used in areas such as graph theory, linear algebra, and combinatorial optimization [37].

A matroid $M = (E, \mathcal{I})$ consists of a finite *ground set* of elements E and a collection $\mathcal{I} \subseteq \mathcal{P}(E)$ of *independent subsets* in E , satisfying the following properties:

- (i) $\emptyset \in \mathcal{I}$
- (ii) $A \in \mathcal{I} \wedge B \subset A \implies B \in \mathcal{I}$
- (iii) $A, B \in \mathcal{I} \wedge |A| < |B| \implies \exists e \in B \setminus A : A \cup \{e\} \in \mathcal{I}$

Terminology A subset of E that is not in $\mathcal{I}$ is called *dependent*. A *basis* of M is a maximal independent set, i.e., an independent set such that adding any element from E would make it dependent. A *circuit* of M is a minimal dependent set, i.e., a dependent set such that all of its proper subsets are independent.

The set $\mathcal{I}$ of a matroid can be fully determined by its set of circuits Circ , as

$$\mathcal{I} = \{A \subseteq E \mid \nexists C \in \text{Circ} : C \subseteq A\}$$

Consequently, a matroid is uniquely determined by its circuits and the ground set [37].

Matroids and Matrices A certain construction of a matroid associates it with a matrix A over the field $\mathbb{F}_q$. The columns of A form the ground set, and a subset is declared independent precisely if the corresponding columns are linearly independent [37]. The resulting matroid, denoted $M[A]$, is called the *vector matroid* of A . Notably, the same matroid may represent different matrices, because a matroid retains only information about which column subsets are linearly dependent, not the specific coefficients witnessing those dependencies. Given this representation, matroid theory can be used to study the dependency structure of a matrix's columns, which is particularly useful in coding theory and the study of equivalence classes.

Matroid Isomorphism Two matroids $M = (E, \mathcal{I})$ and $M' = (E', \mathcal{I}')$ are called *isomorphic* [37], if there exists a bijection $\gamma : E \rightarrow E'$ such that

$$\forall A \subseteq E : \quad A \in \mathcal{I} \iff \gamma(A) \in \mathcal{I}'$$

The corresponding decision problem [45] is defined as follows:

▷ MI: Given two matroids M and M' , determine whether they are isomorphic.

2.7.4 Boolean Satisfiability

The algorithms developed in this thesis also employ the decision problem of Boolean satisfiability.

Terminology A Boolean *variable* can take values in $\{0, 1\}$, corresponding to the values FALSE and TRUE, respectively. A Boolean *operator* is a logical connective operating on Boolean variables. The small generating set of the elementary operators AND $\wedge$ (conjunction), OR $\vee$ (disjunction), and NOT $\neg$ or $\bar{x}$ (negation) can be used to construct more complex, useful logical connectives. The thesis additionally utilizes the following operators:

$$\begin{aligned} \text{XOR} : \quad x \oplus y &\equiv (x \vee y) \wedge \neg(x \wedge y) \\ \text{IMPL} : \quad x \implies y &\equiv \neg x \vee y \\ \text{IFF} : \quad x \leftrightarrow y &\equiv (x \implies y) \wedge (y \implies x) \end{aligned}$$

A Boolean *condition* or *formula* is an expression composed of Boolean variables, logical connectives, Boolean constants, and parentheses. An *assignment* of a Boolean formula is a mapping of Boolean variables to explicit Boolean values.

Satisfiability A Boolean formula is called *satisfiable* if there exists an assignment of Boolean values to its variables under which the formula evaluates to TRUE.

The central decision problem [14] is defined as follows:

▷ SAT: Given a Boolean formula, determine whether it is satisfiable.

3 Related Work

The study of equivalence problems relevant to stabilizer codes spans several areas. Classical code equivalence provides algorithms based on invariants, canonical representatives, and permutation-group methods [22, 34, 43]. Other approaches reduce equivalence problems to an isomorphism problem [5, 9, 16, 35, 38, 45]. A further class of strategies employs strong invariants [25, 50, 51] to determine equivalence, or aims to find automorphisms of codes [3, 18, 27, 41]. Additional techniques exploit graph-state representations [11, 20, 21, 42, 53] in the context of normal forms [28–32], or LC transformations [2, 15, 47–49]. These lines of work provide the foundations for the algorithmic approaches investigated in this thesis, while their individual methods are discussed where they are introduced in Chapter 5.

The work most closely related to this thesis is that of Khesin and Li [29], who investigate the equivalence classes of QEC codes through their ZX-diagram representations. They classify different notions of equivalence for the graphical representations of their encoder circuits and propose their own equivalence definition. In particular, their consideration of output permutations and LC operations on outputs of the encoder circuits is relevant for this thesis. Additionally, they examine equivalence relations specifically for CSS codes and introduce a new canonical form. In that work, Khesin and Li rely heavily on the graphical language of the ZX calculus, demonstrating how graphical representations and rewrite rules can expose structural relationships between quantum codes.

The present thesis addresses a complementary problem. Rather than primarily characterizing equivalence classes or canonical graphical forms, it considers the automated decision of whether two given stabilizer codes are equivalent under a specified set of transformations, aiming to develop practical algorithms for this purpose. While occasionally utilizing graphical representations, this thesis focuses on the binary symplectic representation of stabilizer codes.

The emphasis of this thesis is consequently algorithmic and empirical. It adapts different methods, compares their theoretical and practical limitations, and investigates how complementary techniques can be combined into practically usable equivalence-checking procedures.

4 Problem Formulation

This chapter provides a formal statement of the equivalence problems addressed in this thesis, thereby establishing a common set of assumptions for the proposed equivalence-checking algorithms. We will first specify the representation used for the input codes, and then define the equivalence problems explicitly in terms of this representation. Finally, we will introduce the quality criteria used to evaluate the proposed algorithms.

4.1 Input Representation

In Chapter 2, a $[[n, k, d]]$ QECC is defined by a code space, given by a subspace of $\mathcal{H}^{\otimes n}$, along with an encoding isometry $\mathcal{H}^{\otimes k} \rightarrow \mathcal{H}^{\otimes n}$. As this abstract definition is not directly suitable for algorithmic processing, we choose a more amenable representation.

Accordingly, in this thesis, a code C is represented by the following data:

- Parameters: n, k, d
- Stabilizer group: symplectic matrix $S(C)$
 - For CSS codes, additionally: parity-check matrices $H_x(C), H_z(C)$
- Logical framework: symplectic matrices $L_x(C), L_z(C)$

We assume that all input codes satisfy the following conditions:

- All inputs are stabilizer codes.
- All inputs represent valid codes. In particular, the stabilizer generators commute, the parity-check matrices satisfy the required orthogonality condition, and the logical operators satisfy the commutation relations in (2.2).
- All stabilizer matrices and parity-check matrices have full row rank.

These conditions are therefore not verified by the algorithms considered in this thesis. Beyond these assumptions, no restrictions are imposed on the size or structure of the input codes.

Since this thesis considers the phase-free structure of stabilizer codes, codes that differ only by a physical Pauli operation are identified, allowing for potential phase adaptations. The equivalence problems are formulated directly in terms of the binary symplectic representation, which omits the phases of stabilizer generators and logical operators. This restriction does not affect LC equivalence, since Paulis are local Cliffords, but it means that permutation equivalence is considered up to physical Pauli operations.

4.2 Equivalence Problems

To answer the research questions posed in Chapter 1, we formulate the following equivalence problems using the input representation defined above. For each problem, we also identify the elementary transformations that preserve equivalence, which are separated into the respective equivalence-defining operations and the freedom arising from the representation choice. Throughout this section, $\mathcal{GL}_r$ denotes the general linear group of degree r over $\mathbb{F}_2$.

P-EQ STB: Given two stabilizer codes C and C' , are they permutation-equivalent?

$$\exists P \in \mathcal{S}_n : \exists R \in \mathcal{GL}_r : S(C') = R \cdot S(C) \cdot \begin{bmatrix} P & 0 \\ 0 & P \end{bmatrix}$$

Permitted code operation: paired permutation of stabilizer X/Z -columns

Representation freedom: invertible row operations on stabilizer matrix

Related research question: RQ1

P-EQ CSS: Given two CSS codes C and C' , are they permutation-equivalent?

$$\exists P \in \mathcal{S}_n : \exists R_x \in \mathcal{GL}_{r_x}, R_z \in \mathcal{GL}_{r_z} : H_x(C') = R_x \cdot H_x(C) \cdot P \text{ and } H_z(C') = R_z \cdot H_z(C) \cdot P$$

Permitted code operation: paired column permutation of parity-check matrices

Representation freedom: invertible row operations on parity-check matrices

Related research question: RQ2

LC-EQ STB: Given two stabilizer codes C and C' , are they LC-equivalent?

$$\exists Q = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \in \text{Sp}(2n, \mathbb{F}_2) : Q_i = \begin{bmatrix} a_{ii} & b_{ii} \\ c_{ii} & d_{ii} \end{bmatrix} \in \text{Sp}(2, \mathbb{F}_2) : \exists R \in \mathcal{GL}_r : S(C') = R \cdot S(C) \cdot Q$$

Permitted code operation: invertible isolated column operations on stabilizer matrix

Representation freedom: invertible row operations on stabilizer matrix

Related research question: RQ3

LC-EQ CSS: Given a stabilizer code C , is it LC-equivalent to some CSS code?

$$\exists C' \text{ with } S(C') = \begin{bmatrix} H_x(C') & 0 \\ 0 & H_z(C') \end{bmatrix} :$$

$$\exists Q = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \in \text{Sp}(2n, \mathbb{F}_2) : Q_i = \begin{bmatrix} a_{ii} & b_{ii} \\ c_{ii} & d_{ii} \end{bmatrix} \in \text{Sp}(2, \mathbb{F}_2) : \exists R \in \mathcal{GL}_r : S(C') = R \cdot S(C) \cdot Q$$

Permitted code operation: invertible isolated column operations on stabilizer matrix

Representation freedom: invertible row operations on stabilizer matrix

Related research question: RQ4

4.3 Quality Criteria

To answer the research questions, we need to evaluate the proposed algorithmic approaches against a set of quality criteria. These criteria are designed to assess their correctness and practical applicability in more concrete terms.

Soundness The algorithm has to be sound. Thus, whenever the algorithm returns EQUIVALENT, the input codes must indeed be equivalent. While *false positives* are not allowed, the algorithm does not necessarily have to provide a *witness* for the equivalence, such as a permutation or LC operation.

Completeness The algorithm has to be complete. Thus, whenever the algorithm returns NOT EQUIVALENT, the input codes must indeed be inequivalent. Equivalently, *false negatives* are not allowed.

Runtime Efficiency The algorithm should be runtime efficient. The algorithm is considered practically feasible if it processes instances of relevant sizes within the *specified computational budget*.

Memory Efficiency The algorithm should be memory efficient. The algorithm is considered practically feasible if its observed peak memory requirements remain within the *specified computational budget* for instances of relevant sizes.

Maintainability The algorithm should be maintainable. We consider the *effort* required to integrate, implement, and extend an approach, the algorithm's conceptual complexity, and its *dependence* on external software.

5 Algorithmic Approaches

This chapter develops and analyzes algorithmic approaches to the equivalence problems formulated in Chapter 4. We first introduce preprocessing techniques based on efficiently computable invariants, followed by common subroutines shared by several approaches. We then present the individual equivalence-checking algorithms and analyze their correctness, computational complexity, and applicability to the respective problem variants.

5.1 Preprocessing

Before applying the equivalence-checking algorithms, we perform an independent preprocessing step based on a collection of invariant tests. Its purpose is twofold: first, to reject easily distinguishable input pairs before initiating a potentially expensive search; and second, to establish properties that may subsequently be assumed by the algorithms.

An invariant is a property that is preserved under a considered equivalence relation. Consequently, differing invariant values certify that two codes cannot be equivalent. Equal values, however, do not generally imply equivalence, since non-equivalent codes may share the same invariant. Invariant tests may therefore produce false positives, but no false negatives.

The less *discriminant* an invariant is, the more false positives it will produce. In this case, we call it a weak invariant. Conversely, a strong invariant is highly discriminant and produces few false positives. The usefulness of an invariant as a preprocessing criterion depends on the tradeoff between its computational cost and its discriminative power. Weak invariants are typically inexpensive to evaluate but distinguish relatively few non-equivalent instances, whereas stronger invariants tend to require greater computational effort. Since the invariants are employed only as rejection criteria, completeness is not required; instead, we favor invariants that eliminate a substantial fraction of non-equivalent inputs at moderate computational cost.

5.1.1 Invariants under Qubit Permutations

We first consider invariants that are preserved under qubit permutations and changes in stabilizer generators. The invariants in Table 5.1 are intrinsic to the code and therefore independent of the chosen tableau representation. The mentioned *weight enumerator* records how many stabilizers have each total Pauli weight

$$W(\mathcal{S})_i = |\{s \in \mathcal{S} \mid \text{wt}(s) = i\}|,$$

whereas its modified variant, the *Pauli-sensitive weight enumerator*, additionally retains the individual multiplicities of the three non-identity Pauli operators:

$$W(\mathcal{S})_{(w_{\sigma_1}, w_{\sigma_2}, w_{\sigma_3})} = \left| \{s \in \mathcal{S} \mid \forall j \in \{1, 2, 3\} : |\{s_i \mid s_i = \sigma_j\}| = w_{\sigma_j}\} \right|.$$

Table 5.1: *Code-intrinsic invariants under qubit permutations.*

Invariant	Computational cost	Discriminative power
Code parameters n, k, d	Constant	Low
Weight enumerator	Exponential in r ; even with [33]	Higher than code parameters
Pauli-sensitive weight enumerator	Exponential in r	Higher than ordinary weight enumerator

The quantities in Table 5.2, such as the ranks

$$\left(\text{rank}(S(C)^{(x)}), \text{rank}(S(C)^{(z)})\right)$$

depend on the chosen tableau representation, but remain invariant under qubit permutations and invertible row operations. Restricting the test for *linear dependencies*

$$\left|\left\{A \subseteq \{0, \dots, 2n-1\} \mid |A| \leq m, \sum_{j \in A} \text{column}_j(S(C)) = 0\right\}\right|,$$

to a maximum subset size yields a polynomial-time test for every fixed m . Thus, m controls the tradeoff between computational cost and discriminative power.

Table 5.2: *Tableau-derived invariants under qubit permutations.*

Invariant	Computational cost	Discriminative power
Ranks of the X and Z parts	Polynomial	Low
Number of zero columns	Polynomial	Low
Multiplicities of duplicate columns	Polynomial	Low
Linear column-dependency counts	Polynomial for fixed m	Configurable via m

5.1.2 Invariants under Local Clifford Operations

Table 5.3 considers invariants preserved under LC transformations and changes of stabilizer generators. Notable examples include a *local weight distribution* [51], which is an invariant family that can be confined to a low fixed degree and to subsets of bounded size m . Another family of *local invariants* proposed in [25], can similarly be restricted to a low fixed degree and to a maximum subset size m to yield polynomial-time tests with adjustable discriminative power and cost.

Table 5.3: *Invariants under the application of LC transformations.*

Invariant	Computational cost	Discriminative power
Code parameters: n, k, d	Constant	Low
Local weight distribution	Polynomial for fixed m	Configurable via degree and m
Local invariant	Polynomial for fixed m	Configurable via degree and m

5.1.3 Shared Assumptions

The preprocessing stage rejects input pairs whose parameters n or k differ. Subsequent algorithms may therefore assume that both stabilizer matrices have identical dimensions. Trivial positive instances of empty tableaus are handled separately; hence, the remaining inputs may additionally be assumed to satisfy $n, r > 0$.

5.2 Common Subroutines

The algorithms presented in this chapter repeatedly use several elementary operations on stabilizer tableaus. This section introduces these operations, fixes the corresponding notation, and summarizes their computational complexity.

Row-space Equality of Tableaus Two tableaus $S(C)$ and $S(C')$ represent the same stabilizer group if and only if they have the same row space. In the problem formulations of Chapter 4, this condition is expressed through an invertible change-of-basis matrix $R \in \mathcal{GL}_r$ satisfying

$$S(C') = R \cdot S(C).$$

For the purpose of deciding row-space equality, however, the matrix R need not be computed explicitly. Instead, stacking the tableaus vertically and comparing ranks is sufficient to determine whether they represent the same stabilizer group:

$$\text{rank}(S(C)) = \text{rank}\left(\begin{bmatrix} S(C) \\ S(C') \end{bmatrix}\right) = \text{rank}(S(C')) \implies \text{rowspace}(S(C)) = \text{rowspace}(S(C'))$$

Since it is assumed that both tableaus have the same rank, it is sufficient to compare the rank of the stacked tableau with either individual rank. The required ranks can be computed by Gaussian elimination over $\mathbb{F}_2$. For an $r \times 2n$ stabilizer tableau, this requires $\mathcal{O}(r^2n)$ field operations. In the CSS case, the corresponding row-space checks are applied separately to the $r_x \times n$ and $r_z \times n$ parity-check matrices.

Qubit Permutation of Tableaus A qubit permutation $\pi \in S_n$ acts on a symplectic tableau by simultaneously permuting the corresponding columns of its X and Z parts. Thus, the columns associated with qubit i are moved to those associated with qubit $\pi(i)$ in both parts of the tableau. In the following, $\pi(C)$, $\pi(S(C))$, and $\pi(H_{x/z}(C))$ denote the application of π to a code, a symplectic tableau, and a parity-check matrix, respectively. Applying a permutation to an $r \times 2n$ tableau requires $\mathcal{O}(rn)$ time; for an $r_{x/z} \times n$ parity-check matrix, it requires $\mathcal{O}(r_{x/z}n)$ time.

Local Clifford Operation on Tableaus As described in Equation (2.5), a single-qubit Clifford operation acts on the corresponding pair of X - and Z -columns of a symplectic tableau. Applying this transformation to one qubit requires $\mathcal{O}(r)$ time, since all r rows must be updated. Consequently, applying a tensor product of local Clifford operations to all n qubits requires $\mathcal{O}(rn)$ time.

5.3 Brute-Force Search

As a straightforward approach and baseline for comparison with respect to correctness, we first consider a brute-force search over the entire space of allowed code operations. Since every possible transformation is examined, this approach is sound and complete. However, its factorial or exponential runtime restricts its practical applicability to small instances. Despite this limitation, exhaustive search requires no complex structures and provides a useful reference for assessing the performance of the more sophisticated approaches introduced later.

5.3.1 Permutation Equivalence

To decide problems P-EQ STB and P-EQ CSS, the algorithm enumerates all $n!$ permutations of the physical qubits. Each permutation is applied to the first code, after which the resulting tableau or parity-check matrices are compared with those of the second code using the row-space equality test introduced above. For P-EQ STB, the overall runtime is therefore $\mathcal{O}(n! (rn + r^2n))$. For P-EQ CSS, the corresponding runtime is $\mathcal{O}(n! ((r_x + r_z)n + r_x^2n + r_z^2n))$. In both cases, the factorial number of candidate permutations dominates the asymptotic behavior.

To illustrate the following algorithms with small examples running throughout the chapter, we examine the following positive, i.e., equivalent, problem instances with their respective witnesses:

P-EQ STB : Code $C_1 \rightarrow \mathcal{S}_1 = \langle IIX, XZI \rangle$

Code $C_2 \rightarrow \mathcal{S}_2 = \langle XXZ, IXZ \rangle$

$$S(C_1) = \left[\begin{array}{ccc|ccc} 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \end{array} \right] \xrightarrow{\pi=(1\ 2\ 0)} \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \end{array} \right] \xrightarrow{R=\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}} \left[\begin{array}{ccc|ccc} 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 \end{array} \right] = S(C_2)$$

P-EQ CSS : Code $C_1 \rightarrow \mathcal{S}_1 = \langle IXXX, IIZZ, ZZIZ \rangle$

Code $C_2 \rightarrow \mathcal{S}_2 = \langle XIXX, ZZIZ, IZZZ \rangle$

$$\begin{aligned} H_x(C_1) &= \left[\begin{array}{cccc} 0 & 1 & 1 & 1 \end{array} \right] \xrightarrow{\pi=(1\ 3\ 0\ 2)} \left[\begin{array}{cccc} 1 & 0 & 1 & 1 \end{array} \right] \xrightarrow{R_x=\begin{bmatrix} 1 \end{bmatrix}} \left[\begin{array}{cccc} 1 & 0 & 1 & 1 \end{array} \right] = H_x(C_2) \\ H_z(C_1) &= \left[\begin{array}{cccc} 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \end{array} \right] \xrightarrow{\pi=(1\ 3\ 0\ 2)} \left[\begin{array}{cccc} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{array} \right] \xrightarrow{R_z=\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}} \left[\begin{array}{cccc} 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{array} \right] = H_z(C_2) \end{aligned}$$

5.3.2 Local Clifford Equivalence

For problems LC-EQ STB and LC-EQ CSS, the algorithm iterates over all 6^n possible tensor products of single-qubit Clifford operations from Equation (2.5). Each candidate operation is applied to the input tableau, and the resulting code is tested for the property required by the respective problem. For LC-EQ STB, each transformed tableau is compared with that of the second code using the row-space equality test. Thus, the total runtime is $\mathcal{O}(6^n (rn + r^2n))$. For LC-EQ CSS, the transformed tableau is checked for equivalence to a CSS code by comparing

ranks of the full tableau with its X and Z parts:

$$\text{rank}((S(C)Q)^{(x)}) + \text{rank}((S(C)Q)^{(z)}) = \text{rank}(S(C)Q) = r$$

Consequently, the algorithm has an overall runtime of $\mathcal{O}(6^n (rn + 2r^2n))$.

Thus, for both problems, the overall runtime is dominated by the exponential behavior.

Similar to the previous examples, we examine the proposed algorithms for the following LC equivalence problem instances, with their respective witnesses:

LC-EQ STB : Code $C_1 \rightarrow \mathcal{S}_1 = \langle IIX, XZI \rangle$ with $(\overline{X}_1, \overline{Z}_1) = (IZI, ZXI)$

Code $C_2 \rightarrow \mathcal{S}_2 = \langle IYY, YZI \rangle$ with $(\overline{X}_1, \overline{Z}_1) = (ZXI, XXI)$

$$S(C_1) = \left[\begin{array}{ccc|ccc} 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \end{array} \right] \xrightarrow{U=S \otimes I \otimes S} \left[\begin{array}{ccc|ccc} 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 \end{array} \right] \xrightarrow{R=\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}} \left[\begin{array}{ccc|ccc} 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 \end{array} \right] = S(C_2)$$

LC-EQ STB : Code $C \rightarrow \mathcal{S} = \langle ZZI, YZI \rangle$ with $(\overline{X}_1, \overline{Z}_1) = (YIX, XXI)$

$$S(C) = \left[\begin{array}{ccc|ccc} 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 & 0 \end{array} \right] \xrightarrow{U=S \otimes S \otimes (HS)} \left[\begin{array}{ccc|ccc} 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 \end{array} \right] \xrightarrow{R=\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}} \left[\begin{array}{ccc|ccc} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{array} \right] = S(C_{CSS})$$

Key Takeaway

Brute-force search is a reference method for solving all equivalence problems; P-EQ STB, P-EQ CSS, LC-EQ STB, and LC-EQ CSS.

5.4 SAT-Based Approaches

A further approach encodes equivalence problems as instances of the decision problem SAT. For each pair of input codes, we construct a Boolean formula that is satisfiable if and only if the codes are equivalent under the operations permitted by the corresponding problem. The central task is therefore to express the existence of such operations as a conjunction of Boolean constraints.

We employ a common strategy for encoding tableau equivalence under the allowed operations. The equivalence problems involve two independent degrees of freedom: a change of stabilizer generators, represented by invertible row operations, and either a qubit permutation or a LC transformation. To separate these transformations, we introduce an auxiliary tableau aux as an intermediate representation. The first set of constraints requires aux to result from applying an admissible permutation or local Clifford transformation to $S(C)$. The second set requires aux and $S(C')$ to have the same row space.

$$S(C) \xrightarrow{\pi \text{ or } Q} \text{aux} \xleftarrow{R} S(C')$$

Consequently, a valid assignment to the auxiliary variables exists if and only if the two input tableaus are equivalent under the permitted operations. The following sections describe the encoding of the two directions into Boolean constraints, which are then combined to form a single SAT instance.

Row-space Equality in SAT

To encode row-space equality between a source (src) tableau and a target (tgt) tableau, we introduce the Boolean variables:

$$r_{ij} \quad \text{for } i, j \in \{0, \dots, r-1\}$$

The variable r_{ij} is TRUE if and only if row j of src contributes to the linear combination representing row i of tgt. Collectively, these variables represent the entries of a change-of-basis matrix $R = (r_{ij}) \in \mathbb{F}_2^{r \times r}$ satisfying the relation $S(\text{tgt}) = R S(\text{src})$. For every target row i , we require

$$\text{row}_i(S(\text{tgt})) = \sum_{j=0}^{r-1} r_{ij} \cdot \text{row}_j(S(\text{src})) \quad \text{for } i \in \{0, \dots, r-1\}$$

where the sum is evaluated over $\mathbb{F}_2$. Equivalently, for every $i \in \{0, \dots, r-1\}$ and $k \in \{0, \dots, 2n-1\}$, we introduce the constraint

$$S(\text{tgt})_{ik} = \bigoplus_{j=0}^{r-1} (r_{ij} \wedge S(\text{src})_{jk})$$

Since the entries of $S(\text{src})$ are constants, terms for which $S(\text{src})_{jk} = 0$ can be omitted. These constraints ensure that $\text{rowspan}(S(\text{tgt})) \subseteq \text{rowspan}(S(\text{src}))$. By assumption, both tableaus have rank r ; hence, this inclusion is equivalent to row-space equality.

The encoding introduces r^2 Boolean variables and $r \cdot 2n$ constraints. The number of terms occurring in these constraints depends on the density of the source tableau. In the CSS case, the two parity-check matrices are encoded separately, resulting in $r_x^2 + r_z^2$ variables and $(r_x + r_z)n$ constraints.

Qubit Permutation in SAT

To represent a column permutation between a source (src) tableau and a target (tgt) tableau, we introduce the Boolean variables:

$$p_{ij} \quad \text{for } i, j \in \{0, \dots, n-1\}$$

The variable p_{ij} is TRUE if and only if qubit i of src is mapped to qubit j of tgt. Together, these variables represent the entries of a permutation matrix $P = (p_{ij}) \in \mathbb{F}_2^{n \times n}$, which satisfies $S(\text{tgt}) = S(\text{src}) \begin{bmatrix} P & 0 \\ 0 & P \end{bmatrix}$ under this convention.

For P to represent a valid permutation matrix, each source qubit must be mapped to exactly one target qubit, and each target qubit must be the image of exactly one source qubit. Hence, we require that every row and every column of P contains exactly one entry. Equivalently,

$$\sum_{j=0}^{n-1} p_{ij} = 1 \quad \text{for } i, j \in \{0, \dots, n-1\}$$

where the sum is evaluated over $\mathbb{N}$. An exactly-one constraint over Boolean variables can, for example, be expressed as

$$\text{exactly_one}(x_0, x_1, \dots, x_{n-1}) = \left(\bigvee_{j=0}^{n-1} x_j \right) \wedge \left(\bigwedge_{j=0, k=j+1}^{n-1} (\neg x_j \vee \neg x_k) \right).$$

The following constraints therefore, encode the validity of the permutation matrix:

$$\begin{aligned} \text{exactly_one}(p_{i0}, p_{i1}, \dots, p_{i,n-1}) & \quad \text{for } i \in \{0, \dots, n-1\} \\ \text{exactly_one}(p_{0j}, p_{1j}, \dots, p_{n-1,j}) & \quad \text{for } j \in \{0, \dots, n-1\} \end{aligned}$$

It remains to relate the permutation variables to the tableau entries. Whenever p_{ij} is TRUE, the pair of columns representing source qubit i must equal the pair representing target qubit j , which can be formulated as implications:

$$p_{ij} \implies (\text{qubit}_i(\text{src}) \equiv \text{qubit}_j(\text{tgt})) \quad \text{for } i, j \in \{0, \dots, n-1\}$$

Equivalently, for every $i, j \in \{0, \dots, n-1\}$, we introduce the constraint

$$p_{ij} \implies \bigwedge_{k=0}^{r-1} ((S(\text{src})_{ki} \leftrightarrow S(\text{tgt})_{kj} \wedge S(\text{src})_{k,i+n} \leftrightarrow S(\text{tgt})_{k,j+n}))$$

The encoding introduces n^2 permutation variables, $2n$ exactly-one constraints, and n^2 tableau-mapping constraints, each containing $2r$ entry-wise equivalences. In the CSS case, the same permutation is encoded separately on the two parity-check matrices.

Local Clifford Operation in SAT

To represent a LC operation between a source src tableau and a target tgt tableau, we introduce the Boolean variables:

$$c_{ij} \quad \text{for } i \in \{0, \dots, 5\}, j \in \{0, \dots, n-1\}$$

The variable c_{ij} is TRUE if and only if the i -th LC operation from (I, H, S, HS, SH, HSH) is applied to the j -th qubit of src . Since exactly one of the six transformations must be selected for each qubit, we impose

$$\text{exactly_one}(c_{0j}, c_{1j}, \dots, c_{5j}) \quad \text{for } j \in \{0, \dots, n-1\}$$

The selected transformation must additionally relate the corresponding X - and Z -columns of the two tableaus. Let Q_i denote the i -th transformation listed in Equation (2.5). To equal tgt , the X and Z columns of each qubit from src can be transformed by one of the six operations in $\mathcal{C}_1$, according to (2.5):

$$c_{ij} \implies \text{qubit}_j(\text{tgt}) \equiv \text{qubit}_j(\text{src}) \cdot Q_i \quad \text{for } i \in \{0, \dots, 5\}, j \in \{0, \dots, n-1\}$$

Equivalently, for every $j \in \{0, \dots, n-1\}$, we introduce the six constraints:

$$\begin{aligned} c_{0j} & \implies \left(\left(\bigwedge_{k=0}^{r-1} (S(\text{tgt})_{kj} \leftrightarrow S(\text{src})_{kj}) \right) \wedge \left(\bigwedge_{k=0}^{r-1} (S(\text{tgt})_{k,j+n} \leftrightarrow S(\text{src})_{k,j+n}) \right) \right) \\ & \vdots \\ c_{5j} & \implies \left(\left(\bigwedge_{k=0}^{r-1} (S(\text{tgt})_{kj} \leftrightarrow (S(\text{src})_{kj} \oplus S(\text{src})_{k,j+n})) \right) \wedge \left(\bigwedge_{k=0}^{r-1} (S(\text{tgt})_{k,j+n} \leftrightarrow S(\text{src})_{k,j+n}) \right) \right) \end{aligned}$$

The encoding introduces $6n$ Boolean selection variables, n exactly-one constraints, and $6n$ conditional tableau-mapping constraints. Each mapping constraint contains $2r$ entry-wise equivalences.

Projector Criterion for LC-EQ CSS

Unlike the other equivalence problems, LC-EQ CSS does not provide a second, fixed target tableau. Instead, the task is to determine whether the input code can be transformed into some CSS code with a LC operation. Consequently, the row-space equality test used above cannot be applied directly; a criterion characterizing the CSS property itself is required.

Let

$$V_S = \{\phi(s) \mid s \in \mathcal{S}\}$$

denote the symplectic binary representation of the stabilizer group. As observed in [19], a stabilizer code C is CSS if and only if its V_S is preserved under the algebra of the coordinate-wise projections

$$P_I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad P_X = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \quad P_Z = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \quad P_0 = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

whose action on the symplectic components of a single qubit i is given by

$$(x_i \mid z_i) \xrightarrow{P_0} (0 \mid 0) \quad (x_i \mid z_i) \xrightarrow{P_I} (x_i \mid z_i) \quad (x_i \mid z_i) \xrightarrow{P_X} (x_i \mid 0) \quad (x_i \mid z_i) \xrightarrow{P_Z} (0 \mid z_i)$$

Equivalently, the stabilizer group $\mathcal{S}$ is *invariant* under these projectors on all qubits, which [19] defines as

$$\forall P \in \{P_I, P_X, P_Z, P_0\} : \forall v \in V_S : v Q_P \in V_S$$

where Q_P is constructed as in Equation (2.4) by applying P to each qubit.

We now extend this criterion to LC equivalence. We can use it to check whether there is a LC operation $U \cong Q_U$ that transforms $\mathcal{S}$ into $\mathcal{S}'$, which is invariant under the projector Q_{P_X} :

$$\exists U \in \mathcal{C}_1^n : \mathcal{S}' = \{UsU^\dagger \mid s \in \mathcal{S}\} : \forall v \in V_{\mathcal{S}'} : v Q_{P_X} \in V_{\mathcal{S}'}$$

Because Q_U and Q_{P_X} act independently on each qubit, we can construct the combined effect of the local Clifford operation and the projection on an element of the *original* stabilizer group $\mathcal{S}$

$$Q_{U^\dagger} \cdot Q_{P_X} \cdot Q_U =: Q_{U_{P_X}},$$

which has the following effect on a symplectic representation of a Pauli on qubit i :

$$\begin{aligned} I_{P_X} : (x_i \mid z_i) &\mapsto (x_i \mid 0) & HS_{P_X} : (x_i \mid z_i) &\mapsto (z_i \mid z_i) \\ H_{P_X} : (x_i \mid z_i) &\mapsto (0 \mid z_i) & SH_{P_X} : (x_i \mid z_i) &\mapsto (0 \mid x_i + z_i) \\ S_{P_X} : (x_i \mid z_i) &\mapsto (x_i \mid x_i) & HSH_{P_X} : (x_i \mid z_i) &\mapsto (x_i + z_i \mid 0) \end{aligned}$$

These six conditional transformations can be encoded using the same selection variables as in the preceding local Clifford encoding. The required subset relation can be verified by checking whether each row of the transformed tableau can be expressed as a linear combination of the rows of the original tableau, which can be done using the previously introduced row space equality encoding, without assuming that the rank must be preserved.

Table 5.4: Complexity summary of the SAT instances.

	B	K	L
P-EQ STB	n^2 (perm) $+2nr$ (aux) $+r^2$ (rs)	$2n$ (validity perm) $+n^2$ (perm action) $+2nr$ (rs action)	$\mathcal{O}(n^2)$ (validity perm) $+\mathcal{O}(n^2r)$ (perm action) $+\mathcal{O}(nr^2)$ (rs action)
P-EQ CSS	n^2 (perm) $+n(r_x + r_z)$ (aux) $+(r_x^2 + r_z^2)$ (rs)	$2n$ (validity perm) $+n^2$ (perm action) $+n(r_x + r_z)$ (rs action)	$\mathcal{O}(n^2)$ (validity perm) $+\mathcal{O}(n^2(rx + rz))$ (perm action) $+\mathcal{O}(n(r_x^2 + r_z^2))$ (rs action)
LC-EQ STB	$6n$ (lc choice) $+2nr$ (aux) $+r^2$ (rs)	n (validity lc) $+6n$ (perm action) $+2nr$ (rs action)	$\mathcal{O}(n)$ (validity lc) $+\mathcal{O}(n^r)$ (lc action) $+\mathcal{O}(nr^2)$ (rs action)
LC-EQ CSS	$6n$ (lc choice) $+2nr$ (aux) $+r^2$ (rs)	n (validity lc) $+6n$ (lc action) $+2nr$ (rs action)	$\mathcal{O}(n)$ (validity lc) $+\mathcal{O}(n^r)$ (lc action) $+\mathcal{O}(nr^2)$ (rs action)

Complexity

In the worst case, solving a SAT instance requires time exponential in the number of Boolean variables B , yielding the upper bound $\mathcal{O}(2^B)$ up to polynomial factors associated with evaluating the formula. Modern SAT solvers employ preprocessing and heuristics that often substantially reduce the effective search space. Their practical performance, therefore, depends not only on B but also on the structure of the generated formula, including the number of top-level constraints K and their maximum expanded size L . Table 5.4 summarizes these parameters for the proposed encodings.

These parameters characterize the size of an encoding but do not directly determine the solver's runtime. In particular, formulas of comparable size may differ considerably in difficulty, as will become apparent later. The practical performance of the SAT-based approaches must therefore be evaluated empirically.

Key Takeaway

Encoding them as SAT instances and using a solver can decide all equivalence problems; P-EQ STB, P-EQ CSS, LC-EQ STB, and LC-EQ CSS.

Here we showcase some of the constraints of the examples for P-EQ CSS, where each cell (implicitly) contains exactly one constraint. These constraints already include certain optimizations using

$$x \leftrightarrow 0 \equiv \neg x \quad x \leftrightarrow 1 \equiv x$$

that can be applied because of the fact that the $S(C_1)$ and $S(C_2)$ are Boolean constants from the perspective of the SAT solver:

Constraints to ensure the validity of the permutation:

$(p_{00} \vee p_{01} \vee p_{02} \vee p_{03}) \wedge ((\overline{p_{00}} \vee \overline{p_{01}}) \wedge (\overline{p_{00}} \vee \overline{p_{02}}) \wedge (\overline{p_{00}} \vee \overline{p_{03}}) \wedge \dots \wedge (\overline{p_{02}} \vee \overline{p_{03}}))$	...	...	...
$(p_{00} \vee p_{10} \vee p_{20} \vee p_{30}) \wedge ((\overline{p_{00}} \vee \overline{p_{10}}) \wedge (\overline{p_{00}} \vee \overline{p_{20}}) \wedge (\overline{p_{00}} \vee \overline{p_{30}}) \wedge \dots \wedge (\overline{p_{20}} \vee \overline{p_{30}}))$	...	...	...

Constraints encoding the permutation action:

$p_{00} \implies (\overline{aux_{x00}}) \wedge (\overline{aux_{z00}} \wedge aux_{z10})$	$p_{01} \implies (\overline{aux_{x01}}) \wedge (\overline{aux_{z01}} \wedge aux_{z11})$	...	...
$p_{10} \implies (aux_{x00}) \wedge (\overline{aux_{z00}} \wedge aux_{z10})$	...	...	...
$p_{20} \implies (aux_{x00}) \wedge (aux_{z00} \wedge \overline{aux_{z10}})$	...	...	...
$p_{30} \implies (aux_{x00}) \wedge (aux_{z00} \wedge aux_{z10})$	...	...	...

Constraints encoding the row operations:

$aux_{x00} \iff r_{x00}$	$\overline{aux_{x01}}$	$aux_{x02} \iff r_{x00}$	$aux_{x01} \iff r_{x00}$
$aux_{z00} \iff r_{z00}$	$aux_{z01} \iff r_{z00} \oplus r_{z01}$	$aux_{z02} \iff r_{z01}$	$aux_{z03} \iff r_{z00} \oplus r_{z01}$
$aux_{z10} \iff r_{z10}$	$aux_{z11} \iff r_{z10} \oplus r_{z11}$	$aux_{z12} \iff r_{z11}$	$aux_{z13} \iff r_{z10} \oplus r_{z11}$

5.5 Classical-Code Approaches

Although the equivalence problems considered in this thesis concern quantum codes, they are structurally related to the extensively studied equivalence problem for classical linear codes. This connection allows techniques developed in classical coding theory to be leveraged in the quantum setting. In particular, existing approaches employ invariants, partition refinement, and canonical representatives to reduce or organize the permutation search space [22, 34, 43]. The theoretical complexity of code equivalence and its relationship to other isomorphism problems has likewise received considerable attention [5, 9, 38].

The relevant decision problem, referred to here as *Classical Code Equivalence* (CCE), is defined as follows [5, 34, 38]: Given two linear codes C and C' over $\mathbb{F}_q$ with full-rank generator matrices $G, G' \in \mathbb{F}_q^{k \times n}$, are they equivalent? Two linear codes are considered equivalent if they define the same code space up to a permutation of the coordinates. Thus, there exists a permutation of the columns of G such that the resulting matrix has the same row space as G' .

This formulation closely resembles P-EQ STB and P-EQ CSS—in each case, equivalence permits a common permutation of physical coordinates together with changes of basis among the matrix rows. The quantum problems nevertheless impose additional structure, so we must also consider the applicability of classical algorithms to quantum codes.

From a complexity-theoretic perspective, GI admits a polynomial-time reduction to CCE [5, 38]. Furthermore, CCE is not expected to be NP-complete, assuming that the polynomial hierarchy does not collapse [38]. These results place the problem among isomorphism-type problems whose precise complexity remains unresolved.

5.5.1 Sendrier's Support Splitting Algorithm

Sendrier's *Support Splitting Algorithm* [43] was developed as a practical approach to decide CCE. Its central idea is to assign an invariant signature to each coordinate of a code. Coordinates with identical signatures are grouped into equivalence classes, thereby partitioning—or *splitting*—the support of the code. Any permutation witnessing code equivalence must preserve these signatures and is therefore restricted to mappings between corresponding classes.

Algorithm

Signature Computation For each coordinate i , Sendrier proposes a signature derived from the code punctured at that coordinate. Puncturing a code at i removes the i -th coordinate from every codeword, corresponding to deleting the column from the generator matrix. With $\mathcal{C}^{[i]}$ denoting the resulting punctured code space, its hull is defined as

$$\mathcal{H}(\mathcal{C}^{[i]}) = \mathcal{C}^{[i]} \cap \mathcal{C}^{[i]\perp}.$$

The signature assigned to coordinate i is the weight enumerator of this hull

$$\text{sig}(i) := W(\mathcal{H}(\mathcal{C}^{[i]})).$$

Because code equivalence preserves puncturing, duality, intersection, and Hamming weight, equivalent coordinates have identical signatures. Thus, a signature will never falsely rule out actual equivalence. Algorithm 1 summarizes their computation.

Algorithm 1 Punctured Hull Weight Enumerator

Require: Generator matrix $G \in \mathbb{F}_2^{k \times n}$, column index $i \in \{0, \dots, n-1\}$

Ensure: Punctured Hull Weight Enumerator for column i : $W(\mathcal{H}(G^{[i]})) \in \mathbb{N}^{n-1}$

- 1: **puncture code at column i:** # remove column i from G
 $G \rightarrow G^{[i]}$
 - 2: **compute hull:** # intersection of punctured code and its dual
 $\mathcal{H}(G^{[i]}) := \text{rowspace}(G^{[i]}) \cap \text{kernel}(G^{[i]})$
 - 3: **compute weight enumerator:** # iteration exponential in $\dim(\mathcal{H}(G^{[i]})) \leq k$
 $(1, |\{h \in \mathcal{H}(G^{[i]}) \mid \text{wt}(h) = 1\}|, \dots, |\{h \in \mathcal{H}(G^{[i]}) \mid \text{wt}(h) = n-1\}|)$
-

Partitioning As can be seen in Algorithm 2, the signatures induce a partition of the coordinate set. If two codes contain different numbers of coordinates with a particular signature, they cannot be permutation-equivalent. If the signatures happen to be *fully discriminant*, the partitions will consist of singleton sets, and the algorithm can immediately extract a candidate permutation that needs to be verified by applying it to one code and testing row-space equality. Matching partitions, however, do not by themselves establish equivalence, since the signature need not be fully discriminant. If larger classes remain, only permutations mapping corresponding classes to one another need to be considered.

Complexity

A direct computation of the corresponding weight enumerator enumerates all $2^{\dim(\mathcal{H}(\mathcal{C}^{[i]}))}$ hull codewords and is therefore exponential in its runtime. Since a signature is computed for each of the n coordinates, this enumeration constitutes the principal preprocessing cost. Its practical feasibility depends on the hull dimensions, which may be substantially smaller than the dimension of the original code.

With a resulting partition consisting of classes

$$\Pi = \{J_0, \dots, J_{m-1}\},$$

Algorithm 2 Sendrier Support Splitting Algorithm

Require: Check matrices $H_1, H_2 \in \mathbb{F}_2^{r \times n}$

Ensure: Whether the corresponding codes are not permutation-equivalent

```

1: for  $H \in \{H_1, H_2\}$  do
2:   compute generator matrix:  $H \rightarrow G$ 
3:   for  $i = 0, \dots, n-1$  do
4:     compute signature:  $W(\mathcal{H}(G^{[i]}))$  # Algorithm 1
5:   end for
6:   partition columns: # same signature values  $\rightarrow$  same equivalence class
      $\Pi_H := \{w : \{i \mid W(\mathcal{H}(G^{[i]})) = w\} \mid w \in \mathbb{N}^{n-1}\}$ 
7: end for

8: if  $\exists w \in \mathbb{N}^{n-1} : |\Pi_{H_1}(w)| \neq |\Pi_{H_2}(w)|$  then
9:   return NOT EQUIVALENT
10: else
11:   if  $\forall w \in \mathbb{N}^{n-1} : |\Pi_{H_1}(w)| = |\Pi_{H_2}(w)| = 1$  then # signature fully discriminant
12:     extract permutation:  $\{\pi(i) = j \mid \Pi_{H_1}(w) = \{i\} \wedge \Pi_{H_2}(w) = \{j\}\}$  # unique
13:     if  $\text{rowspan}(\pi(G_1)) = \text{rowspan}(G_2)$  then
14:       return EQUIVALENT
15:     else
16:       return NOT EQUIVALENT
17:     end if
18:   else
19:     return NO ANSWER
20:   end if
21: end if
    
```

rather than considering all $n!$ coordinate permutations, it is then sufficient to examine at most

$$\prod_{i=0}^{m-1} |J_i|!$$

permutations compatible with the partition. Highly discriminant signatures yield small classes and can therefore substantially reduce the search space. In the worst case, however, all coordinates receive the same signature, yielding a single class of size n . The permutation search then remains of factorial size, in addition to the cost of computing the signatures.

5.5.2 Feulner's Canonicalization Algorithm

In [22], Feulner proposed an algorithm that computes a canonical representative of a linear code under coordinate permutations [22]. A canonicalization map assigns the same matrix to two generator matrices G and G' if and only if their corresponding classical codes are equivalent. CCE can therefore be decided by canonicalizing both input codes and comparing the resulting representatives.

The algorithm explores a search tree whose branches correspond to admissible coordinate permutations. A column partition can restrict the search to permutations within corresponding classes. During the traversal, partial candidates are compared lexicographically with the best

candidate found so far. Branches that cannot yield a better representative are pruned. This approach can be combined with Sendrier’s algorithm by first computing the signature partition and then constructing canonical forms, building on the received partitions, allowing early termination when equivalence is established or ruled out.

Algorithm

Algorithm 3 presents a simplified version of the canonicalization procedure, adapted to permutation equivalence over $\mathbb{F}_2$. Initially, the classes of the supplied coordinate partition are placed in a fixed order, for example, according to their signature values.

Canonicalization is performed by recursively traversing a search tree of permutations. At recursion depth i , the first i coordinate positions have already been fixed. The prefix of the current matrix is compared with the corresponding prefix of the best complete representative found so far. If it is lexicographically larger, no completion of the current branch can improve the best representative, and the branch is pruned. Otherwise, the search continues with the next position $i + 1$. The algorithm considers every coordinate from the currently admissible partition class as a candidate for position $i + 1$. After moving a candidate to that position, row operations transform the matrix into a semi-canonical form with respect to the fixed prefix. Over $\mathbb{F}_2$, this step can be implemented using Gaussian elimination without further column permutations. The search continues recursively.

Once all coordinate positions have been fixed, the resulting matrix constitutes a candidate representative. The lexicographically smallest candidate encountered during the traversal is returned as the canonical form. If the ordering relation guarantees a unique minimum, the algorithm is complete and sound, as the entire search space is explored and only the worse candidates are pruned.

Complexity

While the worst-case complexity of Feulner’s algorithm is factorial in the number of coordinates, its practical performance can be much better if the search tree can be pruned effectively. However, each visited search-tree node additionally requires polynomial-time matrix operations, primarily Gaussian elimination and lexicographic comparison. The practical runtime, therefore, depends on both the sizes of the partition classes and the effectiveness of prefix-based pruning. Highly discriminant partitions and early lexicographic differences may substantially reduce the explored search tree. For codes with weak invariants or large automorphism groups, however, many branches may remain indistinguishable until considerable depth, and the runtime can approach the factorial worst case.

5.5.3 Adaptation to Quantum Codes

Following the discussion of Classical Code Equivalence, we now consider the applicability of these algorithms to quantum codes. The additional structure of quantum codes requires certain modifications to the classical algorithms, while their underlying principles remain applicable. We consider two different approaches for translating quantum-code equivalence to classical-code equivalence, depending on whether the input codes are CSS or general stabilizer codes.

Algorithm 3 Feulner Canonicalization Algorithm

Require: Generator matrix $G \in \mathbb{F}_2^{k \times n}$, column partition Π

Ensure: Canonical form of G considering Π

```

1: linearize partition:  $\Pi \rightarrow \Pi^S$  # sort signature values, column indices arbitrarily
2: sort columns:  $G \rightarrow G^S$  # swap according to linearized partition

3: function SEARCH( $i, G_i, \Pi_i$ ) # columns  $0 \dots i - 1$  already fixed
4:   if  $G_i \succ_i G_{\text{best}}$  then
5:     return # prune search tree
6:   end if

7:   if  $G_i \prec_i G_{\text{best}}$  then
8:      $G_{\text{best}} := \top$  # better matrix on current branch found
9:   end if

10:  if  $i = n$  then
11:     $G_{\text{best}} := G_i$ 
12:    return
13:  end if

14:  for  $j \in \Pi(w)$  with  $i \in \Pi(w)$  do
15:    remove  $j$ :  $\Pi_{i+1} := \{v : \Pi(v) \mid \forall v \in \mathbb{N}^{n-1} : v \neq w\} \cup \{w : \Pi(w) \setminus \{j\}\}$ 
16:    swap columns  $i$  and  $j$ :  $G_{i+1} := \text{swap}_{i \leftrightarrow j}(G_i)$ 
17:    semi-canonicalize  $G_{i+1}$  # fix columns  $0 \dots i$ 
18:    SEARCH( $i + 1, G_{i+1}, \Pi_{i+1}$ )
19:  end for
20: end function

21:  $G_{\text{best}} := \top$  # single global variable
22: SEARCH( $0, G^S, \Pi^S$ )
23: return  $G_{\text{best}}$ 

```

CSS Codes: H_x, H_z

Since CSS codes are closely related to classical codes, their structure can be exploited by considering the binary parity-check matrices H_x and H_z . This reduces the equivalence problem to two coupled binary instances of CCE, subject to the constraint that the same permutation is applied to both matrices.

The principal challenge is to determine at which stage of the algorithms the information from the two matrices should be combined to identify a permutation compatible with both. One possibility is to compute the signatures and canonical forms for both matrices separately, retain the corresponding permutation information during both searches, and compare the admissible permutations at the end of the process. Although straightforward, this approach may be inefficient since two search trees are explored. Moreover, the independent searches may examine many permutations that are incompatible with the respective other matrix.

An alternative is to compute the signatures and canonical forms for only one of the matrices, and subsequently check the resulting candidate permutations against the other matrix using a

row-space equality check. This approach is more efficient, since only one search tree is explored. Still, it may be less effective at reducing the search space, because information from the other matrix is incorporated only during the final verification step.

A more promising compromise is to construct a column partition from a combined signature of both matrices

$$\text{sig}(i) = (\text{sig}_x(i), \text{sig}_z(i))$$

which will always be at least as discriminant as a single signature, although computing both components incurs a higher preprocessing cost. The resulting partition can be used to restrict the search space, while the canonicalization algorithm can then be applied to one of the matrices. Finally, the candidate permutations can be checked for compatibility with the other matrix. This approach incorporates information from both matrices without requiring the exploration of two independent search trees.

In the following we will compute the combined signatures $\text{sig}(i)$ for the two codes in P-EQ CSS using Sendrier's weight enumerator invariant:

$$H_x(C_1) = \begin{bmatrix} 0 & 1 & 1 & 1 \end{bmatrix} \rightarrow G_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}, \quad H_z(C_1) = \begin{bmatrix} 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \end{bmatrix} \rightarrow G_z = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

i		$G_{x/z}^{[i]}$	$\mathcal{C}$	$\mathcal{C}^\perp$	$\mathcal{H}$	$W(\mathcal{H})$	$\text{sig}(i)$
0	G_x	$\begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$	$\{000, 101, 011, 110\}$	$\{000\}$	$\{000\}$	1	$(1, 1 + X^2)$
	G_z	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}$	$\{000, 100, 011, 111\}$	$\{000, 011\}$	$\{000, 011\}$	$1 + X^2$	
1	G_x	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$	$\{0, 1\}^3$	$\{000\}$	$\{000\}$	1	$(1, 1 + X^2)$
	G_z	$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$	$\{000, 100, 111, 011\}$	$\{000, 011\}$	$\{000, 011\}$	$1 + X^2$	
2	G_x	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$	$\{0, 1\}^3$	$\{000\}$	$\{000\}$	1	$(1, 1)$
	G_z	$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$	$\{000, 110, 101, 011\}$	$\{000\}$	$\{000\}$	1	
3	G_x	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\{0, 1\}^3$	$\{000\}$	$\{000\}$	1	$(1, 1)$
	G_z	$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$	$\{000, 110, 101, 011\}$	$\{000\}$	$\{000\}$	1	

$$H_x(C_2) = \begin{bmatrix} 1 & 0 & 1 & 1 \end{bmatrix} \rightarrow G_x = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}, \quad H_z(C_2) = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} \rightarrow G_z = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

i		$G_{x/z}^{[i]}$	$\mathcal{C}$	$\mathcal{C}^\perp$	$\mathcal{H}$	$W(\mathcal{H})$	$\text{sig}(i)$
0	G_x	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\{0, 1\}^3$	$\{000\}$	$\{000\}$	1	$(1, 1)$
	G_z	$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$	$\{000, 110, 101, 011\}$	$\{000\}$	$\{000\}$	1	
1	G_x	$\begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$	$\{000, 110, 101, 011\}$	$\{000\}$	$\{000\}$	1	$(1, 1 + X^2)$
	G_z	$\begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\{000, 110, 001, 111\}$	$\{000, 110\}$	$\{000, 110\}$	$1 + X^2$	
2	G_x	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 1 \end{bmatrix}$	$\{0, 1\}^3$	$\{000\}$	$\{000\}$	1	$(1, 1)$
	G_z	$\begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$	$\{000, 110, 011, 101\}$	$\{000\}$	$\{000\}$	1	
3	G_x	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$	$\{0, 1\}^3$	$\{000\}$	$\{000\}$	1	$(1, 1 + X^2)$
	G_z	$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	$\{000, 111, 010, 101\}$	$\{000, 101\}$	$\{000, 101\}$	$1 + X^2$	

It is apparent from the signatures that the qubits of the two codes can be partitioned into $\{\{0, 1\}, \{2, 3\}\}$ for code C_1 and $\{\{1, 3\}, \{0, 2\}\}$ for code C_2 . In view of these partitions and signature values, it is only possible to map qubits 0 or 1 of code C_1 to qubits 1 or 3 of code C_2 , and qubits 2 and 3 to qubits 0 and 2. This reduces the search space from $4! = 24$ to $2! \cdot 2! = 4$ possible permutations that need to be checked for equivalence.

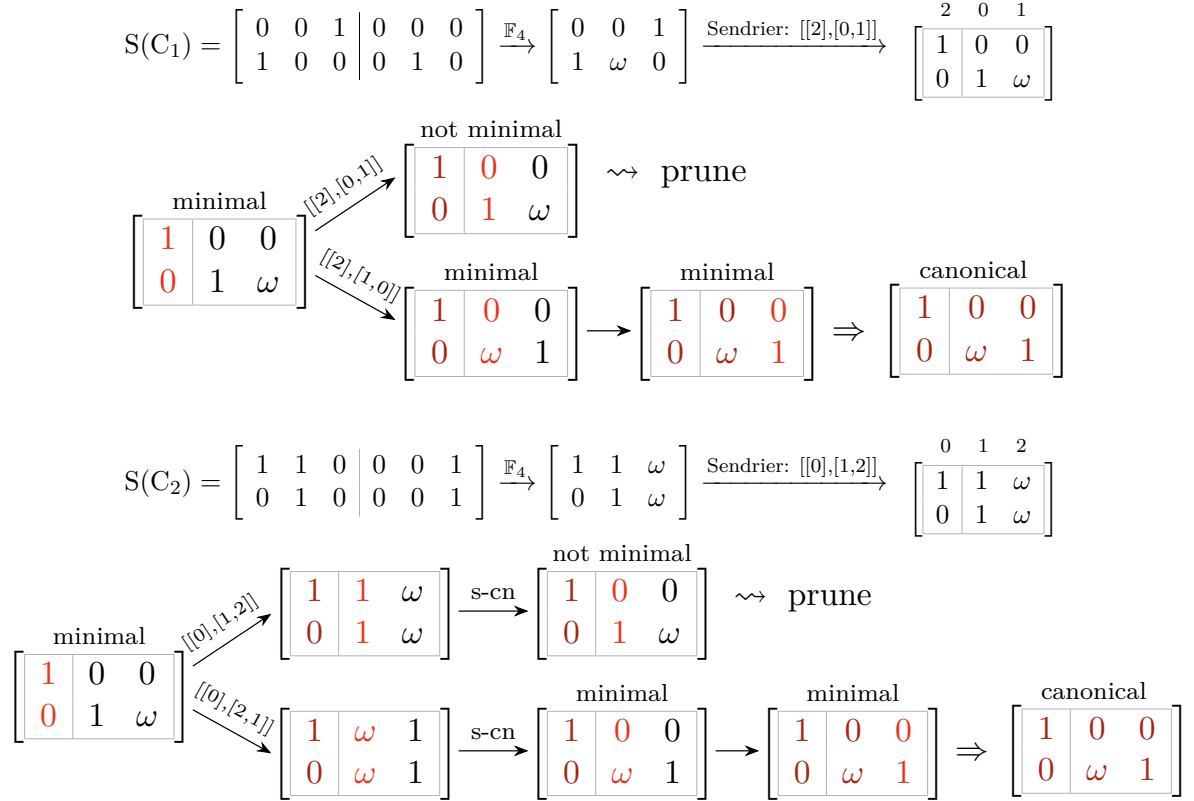
Stabilizer Codes: $\mathbb{F}_4$

Applying classical-code techniques to general stabilizer codes is less direct than in the CSS case. In the binary symplectic representation, each qubit is represented by a pair of X - and Z -coordinates, which must be permuted jointly. Consequently, the tableau cannot be treated as an ordinary binary generator matrix of the stabilizer group whose columns may be permuted independently.

To provide a more suitable framework, we utilize the $\mathbb{F}_4$ representation of stabilizer codes. Under this correspondence, the rows of the stabilizer tableau generate an additive code over $\mathbb{F}_4$, representing the stabilizer group. A qubit permutation then corresponds to an ordinary coordinate permutation of this code. A fixed ordering of the four field elements, e.g. $0 < \omega < 1 < \omega^2$, additionally permits lexicographic comparison and therefore supports the partitioning and canonicalization procedures introduced above. Although the code is contained in $\mathbb{F}_4^n$, it is generally linear only over $\mathbb{F}_2$, rather than over $\mathbb{F}_4$. Thus, the simplified classical algorithms can be applied with only minor modifications, as the allowed operations fit into their binary framework.

Here, after being converted to their $\mathbb{F}_4$ representation and ordered according to a given Sendrier partition, we execute the canonicalization algorithm of Feulner on the two codes in P-EQ STB to determine whether they are equivalent.

In the process, the current position i is marked in light red, whereas the already semi-canonicalized part is marked in dark red. Every time the (light and dark) red part is brought to semi-canonical form (s-cn) and not minimal, the branch is pruned. The process continues until the whole matrix is in canonical form.



We can see that the canonical forms of the two codes are equal, and thus they are permutation-equivalent.

Key Takeaway

Classical-code approaches can be combined and transformed into equivalence-checking methods that solve P-EQ STB and P-EQ CSS for quantum error-correcting codes.

5.5.4 Automorphism Group

Closely related to CCE is the computation of the *automorphism group* of a code space, defined as the set of all coordinate permutations that preserve the code space:

$$\text{Aut}(\mathcal{C}) = \{\pi \in \mathcal{S}_n \mid \pi(\mathcal{C}) = \mathcal{C}\}$$

Since automorphism groups are already studied in several quantum-code contexts [18, 27, 41], it is natural to consider their use for quantum-code permutation equivalence. This inspired the idea to use the automorphism group of a code space to reduce its permutation search space, as

$$\pi \in \mathcal{S}_n : \pi(\mathcal{C}_2) \neq \mathcal{C}_1 \implies \forall a \in \text{Aut}(\mathcal{C}_2) : \pi(a(\mathcal{C}_2)) \neq \mathcal{C}_1$$

thus the permutation $(\pi \circ a)$ cannot lead to equivalence and does not have to be checked. In fact, $a(\mathcal{C}_2) = \mathcal{C}_2$, so all permutations in the same right coset $\pi \circ \text{Aut}(\mathcal{C}_2)$ produce the same image of $\mathcal{C}_2$. It is therefore sufficient to examine one representative of each coset, reducing the number of candidate permutations from $n!$ to

$$\frac{n!}{|\text{Aut}(\mathcal{C}_2)|}.$$

This reduction is particularly effective for codes with large automorphism groups. If the automorphism group is small or trivial, however, the search space is reduced only marginally, and the worst-case runtime remains factorial. Moreover, computing the automorphism group may itself be expensive, particularly for large codes.

Key Takeaway

Reducing the permutation search space using the automorphism group of a code can be a strategy for solving P-EQ STB.

5.6 Isomorphism-Based Approaches

While the previous section focused on classical-code techniques, this section explores approaches that leverage the structural properties of quantum codes. These methods reduce the equivalence problem to a colored graph isomorphism problem, which can be solved using modern algorithms and tools [4, 35].

Consequently, although the complexity of the CGI problem cannot be controlled directly, the size and structure of the constructed graphs can be chosen in a way that may lead to more efficient solutions.

5.6.1 Matroid Isomorphism

Constructing vector matroids from the generator or parity-check matrices of classical codes is a natural application of the matroid construction introduced in Section 2.7.3. Previous work has established close connections between CCE and matroid isomorphism [5, 37, 44]. Moreover, [40] extends the correspondence between linear codes and matroids to quantum codes in the context of quantum secret sharing. In this section, we apply these ideas to our equivalence problem P-EQ CSS, discussing the use of matroid isomorphism to establish permutation equivalence of CSS codes.

Algorithm As shown in Algorithm 4, we first construct a vector matroid from each binary parity-check matrix of both input codes, using the fact that linear dependencies between columns can be computed from the kernel of the binary matrices.

The two matroids associated with each CSS code are then combined into a single colored incidence graph, using the fact that a matroid is fully defined by its ground set and its circuits. This graph retains the dependency structures of both parity-check matrices while allowing simultaneous permutation of the qubits for each. The resulting graphs are then compared using a color-preserving graph-isomorphism algorithm.

Over $\mathbb{F}_2$, a vector matroid representation is unique for a matrix up to invertible row operations, since the only nonzero column-scaling factor is 1. Consequently, two full-row-rank binary matrices represent isomorphic matroids if and only if one can be obtained from the other by an invertible row transformation. The circuit structure, therefore, retains precisely the information required for this aspect of equivalence. Hence, a color-preserving graph isomorphism is equivalent to P-EQ CSS for this construction.

We apply the matroid construction algorithm and the corresponding graph to the codes for P-EQ CSS to provide a visual illustration of its steps and results.

$$\begin{aligned}
 H_x(C_1) &= \begin{bmatrix} 0 & 1 & 1 & 1 \end{bmatrix} \xrightarrow{M[H_x(C_1)]} E = \{0, 1, 2, 3\}, \mathcal{I} = \{\emptyset, \{1\}, \{2\}, \{3\}\} \\
 &\xrightarrow{\text{circuit extraction}} \text{Circ} = \{\{0\}, \{1,2\}, \{2,3\}, \{1,3\}\} \\
 H_z(C_1) &= \begin{bmatrix} 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \end{bmatrix} \xrightarrow{M[H_z(C_1)]} E = \{0, 1, 2, 3\}, \\
 &\mathcal{I} = \{\emptyset, \{0\}, \{1\}, \{2\}, \{3\}, \{0, 2\}, \{0, 3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}\} \\
 &\xrightarrow{\text{circuit extraction}} \text{Circ} = \{\{0, 1\}, \{0, 2, 3\}, \{1, 2, 3\}\} \\
 H_x(C_2) &= \begin{bmatrix} 1 & 0 & 1 & 1 \end{bmatrix} \xrightarrow{M[H_x(C_2)]} E = \{0, 1, 2, 3\}, \mathcal{I} = \{\emptyset, \{0\}, \{2\}, \{3\}\} \\
 &\xrightarrow{\text{circuit extraction}} \text{Circ} = \{\{1\}, \{0, 2\}, \{2, 3\}, \{0, 3\}\} \\
 H_z(C_2) &= \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} \xrightarrow{M[H_z(C_2)]} E = \{0, 1, 2, 3\}, \\
 &\mathcal{I} = \{\emptyset, \{0\}, \{1\}, \{2\}, \{3\}, \{0, 1\}, \{0, 2\}, \{0, 3\}, \{1, 2\}, \{2, 3\}\} \\
 &\xrightarrow{\text{circuit extraction}} \text{Circ} = \{\{1, 3\}, \{0, 1, 2\}, \{0, 3, 2\}\}
 \end{aligned}$$

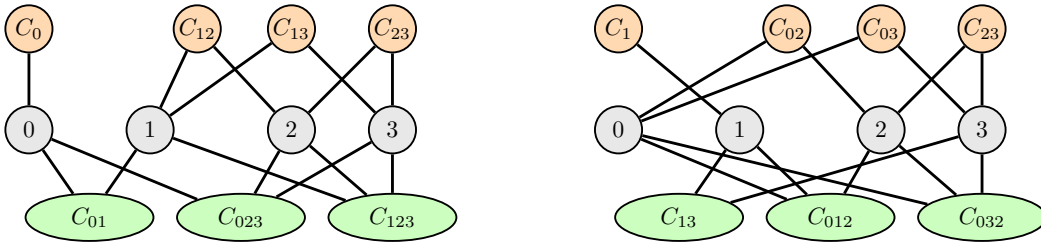


Figure 5.1: Matroid graphs for C_1 (left) and C_2 (right) from the P-EQ CSS example.

From the resulting incidence graphs in Figure 5.1, it is evident that there are multiple possible isomorphisms between the two graphs, corresponding to two possible qubit permutations that establish equivalence of the codes. The permutations from C_1 to C_2 map 0 to 1 and 1 to 3, and either mapping of 2 and 3 to 0 and 2 is valid.

Algorithm 4 Matroid Construction Algorithm

Require: Check matrices $H_x(C_1), H_x(C_2) \in \mathbb{F}_2^{r_x \times n}$, $H_z(C_1), H_z(C_2) \in \mathbb{F}_2^{r_z \times n}$

Ensure: Whether the codes C_1 and C_2 are permutation-equivalent or not

```

1: for  $C \in \{C_1, C_2\}$  do
2:   for  $H \in \{H_x(C), H_z(C)\}$  do
3:     matroid construction:  $H \rightarrow M[H]$  with  $E = \{0, \dots, n-1\}$ 
4:      $\# \forall x \in \text{kernel}(H) : \text{supp}(x) \notin \mathcal{I}$ 

5:     circuit extraction:  $\text{Circ}(M[H])$   $\#$  keep minimal linear dependencies
6:   end for

7:   graph construction:  $\#$  colored combined incident graph  $G_C = (\mathcal{V}, \mathcal{E})$ 
8:    $\mathcal{V} := \{0, \dots, n-1\} \cup \text{Circ}(M[H_x(C)]) \cup \text{Circ}(M[H_z(C)])$ 
9:    $\mathcal{E} := \{\{i, c\}, i \in \{0, \dots, n-1\}, c \in \text{Circ}(M[H_x(C)]) \cup \text{Circ}(M[H_z(C)]) \mid i \in c\}$ 
10:   $c := \{\{0, \dots, n-1\} \mapsto 0, \text{Circ}(M[H_x(C)]) \mapsto 1, \text{Circ}(M[H_z(C)]) \mapsto 2\}$ 
11: end for

12: if  $G_{C_1} \cong G_{C_2}$  then
13:   return EQUIVALENT
14: else
15:   return NOT EQUIVALENT
16: end if
    
```

Complexity Apart from the CGI problem, the main runtime bottleneck of this approach is the circuit extraction of the matroids, as it enumerates all dependent subsets of the ground set to find the minimal ones. Thus, for a check matrix H , this step requires a runtime of $\mathcal{O}(2^{\dim(\text{kernel}(H))})$. In the worst case, the number of circuits can be exponential in the size of the ground set n . Thus, since the proposed incidence graph contains one vertex for every circuit, both its construction time and its memory consumption may be exponential in n .

Key Takeaway

Matroid Isomorphism combined with Colored Graph Isomorphism can be employed to decide the equivalence problem P-EQ CSS.

5.6.2 Signature-Based Graph Construction

A further graph construction builds on Sendrier's approach of partitioning code coordinates by their signatures, but permits several signatures to be combined. We present the construction for CSS codes, but it can be extended to general stabilizer codes by adapting the signature computation.

Algorithm The approach is outlined in Algorithm 5. First, a configurable collection of signatures is evaluated for every qubit. The resulting values are represented by a colored incidence graph that encodes the combined, potentially highly discriminating coordinate partition.

Rather than using CGI directly as a complete equivalence test, we use graph isomorphisms to extract a reduced set of candidate qubit permutations that preserve all selected signature values.

Since the signatures need not be complete invariants, this set may contain invalid candidates. Each candidate must therefore be verified by applying the corresponding permutation and testing row-space equality.

In the following, we provide an example of the graph construction for the codes in P-EQ CSS using two signatures, f_1 and f_2 . The signature f_1 encodes the zero columns in the X and Z parts of the tableau, while the signature f_2 determines the punctured-hull weight enumeration, as discussed in Example 5.5.3. The resulting signature values for each column are as follows:

		0	1	2	3
Code 1	f_1	2	3	3	3
	f_2	$(1, 1 + X^2)$	$(1, 1 + X^2)$	$(1, 1)$	$(1, 1)$
Code 2	f_1	3	2	3	3
	f_2	$(1, 1)$	$(1, 1 + X^2)$	$(1, 1)$	$(1, 1 + X^2)$

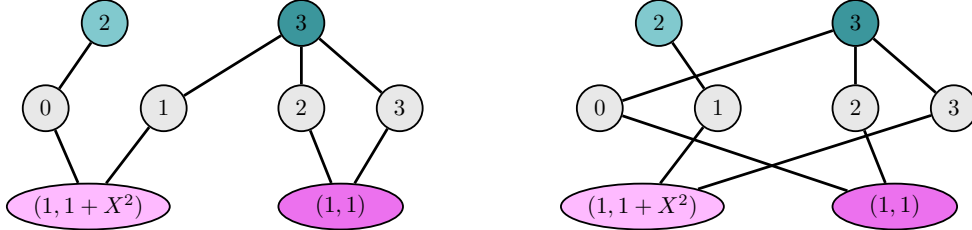


Figure 5.2: Invariant graphs for C_1 (left) and C_2 (right) from the P-EQ CSS example.

The resulting graphs in Figure 5.2 are isomorphic. In this particular example, both candidate permutations from C_1 to C_2 , namely $(1\ 3\ 0\ 2)$ and $(1\ 3\ 2\ 0)$, are valid witnesses. This does not hold in general, since the selected signatures need not form a complete set of invariants.

Complexity The cost of constructing the graph depends primarily on the selected signatures, whose computational requirements may vary considerably. For m signatures, the number of vertices in the constructed graph is bounded by $m \cdot (n + 1)$, and the number of edges by $m \cdot n$. Apart from the cost of solving the resulting CGI instance, the principal potential bottleneck is the verification of the extracted candidate permutations. If the selected signatures are weak, the number of candidates may remain factorial in the number of qubits.

Key Takeaway

The signature-based graph construction can be used as a tool for deciding the equivalence problem P-EQ CSS.

5.6.3 Full Stabilizer Group Mapping

This approach directly reduces P-EQ STB and LC-EQ STB to instances of CGI by representing the complete stabilizer group as a graph. Enumerating all stabilizer elements removes the dependence on the chosen generator basis and therefore makes the construction invariant under

Algorithm 5 Signature-based Graph Construction Algorithm

Require: Check matrices $H_x(C_1), H_x(C_2) \in \mathbb{F}_2^{r_x \times n}$, $H_z(C_1), H_z(C_2) \in \mathbb{F}_2^{r_z \times n}$,
signature computation function $f_0, \dots, f_{m-1}$

Ensure: Whether the codes C_1 and C_2 are permutation-equivalent or not

```

1: for  $C \in \{C_1, C_2\}$  do
2:   for  $f \in \{f_0, \dots, f_{m-1}\}$  do
3:     for  $i \in \{0, \dots, n-1\}$  do
4:       compute signature value:  $f(i, H_x(C), H_z(C))$ 
5:     end for
6:   end for

7:   graph construction: # colored incident graph  $G_C = (\mathcal{V}, \mathcal{E})$ 
8:    $\mathcal{V} := \{0, \dots, n-1\} \cup \{v \mid \exists f_j : \exists i \in \{0, \dots, n-1\} : f_j(i) = v\}$ 
9:    $\mathcal{E} := \{\{i, v\} \mid \exists f_j : f_j(i) = v\}$ 
10:   $c := \{\{0, \dots, n-1\} \mapsto 0\} \cup \{\{v\} \mapsto v \mid \exists f_j : \exists i \in \{0, \dots, n-1\} : f_j(i) = v\}$ 
11: end for

12: extract isomorphisms:  $\Phi = \{\varphi \mid G_{C_1} = \varphi(G_{C_2})\}$ 
13: extract permutations:  $\Pi = \{(\varphi(0), \dots, \varphi(n-1)) \mid \varphi \in \Phi\}$  # potential witnesses

14: for  $\pi \in \Pi$  do
15:   if  $C_1 \sim \pi(C_2)$  then
16:     return EQUIVALENT
17:   end if
18: end for
19: return NOT EQUIVALENT
    
```

row operations. The graph structure can then be adapted to retain precisely the additional freedom permitted by the respective equivalence relation.

We first present the construction for P-EQ STB and subsequently adapt it to LC-EQ STB. As permutation construction also applies to CSS codes, it can be used for P-EQ CSS.

Algorithm

Permutation Equivalence For an instance of P-EQ STB, we enumerate all elements of the stabilizer group and construct a colored incidence graph with one vertex class representing the physical qubits, while the other represents the stabilizer elements. A stabilizer vertex is connected to a qubit vertex whenever it acts non-trivially on that qubit, with the edge color indicating the corresponding Pauli operator. The exact construction is given in Algorithm 6.

The vertex colors distinguish qubits from stabilizer elements but allow arbitrary permutations within each class. Consequently, a color-preserving graph isomorphism may permute the qubit vertices while mapping stabilizer elements according to the induced qubit permutation. The two graphs are isomorphic if and only if their stabilizer groups agree up to a permutation of the qubits. The construction, therefore, provides a sound and complete criterion for P-EQ STB.

Here we apply the graph construction algorithm to the codes C_1 and C_2 for P-EQ STB. The full stabilizer groups are depicted with their binary symplectic representations, which already align with the adjacency structure of the resulting graphs. The stabilizer group elements are ordered left to right in the graphs.

$$\begin{aligned}\mathcal{S}_1 &= \langle (001 \mid 000), (100 \mid 010) \rangle = \{(\text{000} \mid \text{000}), (\text{001} \mid \text{000}), (\text{100} \mid \text{010}), (\text{101} \mid \text{010})\} \\ \mathcal{S}_2 &= \langle (110 \mid 001), (010 \mid 001) \rangle = \{(\text{000} \mid \text{000}), (\text{110} \mid \text{001}), (\text{010} \mid \text{001}), (\text{100} \mid \text{000})\}\end{aligned}$$

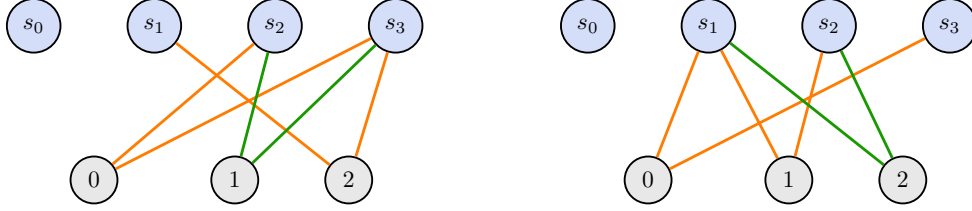


Figure 5.3: Full graphs for C_1 (left) and C_2 (right) from the P-EQ STB example.

As can be seen in Figure 5.3, the graphs are isomorphic with qubit 0 of C_1 mapping to qubit 1 of C_2 , qubit 1 to qubit 2, and qubit 2 to qubit 0.

Algorithm 6 Graph Construction Algorithm

Require: Stabilizer tableaux $S(C_1), S(C_2) \in \mathbb{F}_2^{r \times 2n}$

Ensure: Whether the codes C_1 and C_2 are permutation-equivalent or not

```

1: for  $C \in \{C_1, C_2\}$  do
2:   enumerate stabilizer group:  $\mathcal{S}$  # exponential in  $r$ 
3:   graph construction: # colored incident graph  $G_C = (\mathcal{V}, \mathcal{E})$ 
4:      $\mathcal{V} := \{0, \dots, n-1\} \cup \mathcal{S}$ 
5:      $\mathcal{E} := \mathcal{E}_x \cup \mathcal{E}_z$ 
6:      $\rightarrow \mathcal{E}_x = \{\{i, s\} \mid s \in \mathcal{S}, i \in \{0, \dots, n-1\}, s_i = X \vee s_i = Y\}$ 
7:      $\rightarrow \mathcal{E}_z = \{\{i, s\} \mid s \in \mathcal{S}, i \in \{0, \dots, n-1\}, s_i = Z \vee s_i = Y\}$ 
8:      $c := \{\{0, \dots, n-1\} \mapsto 0, \mathcal{S} \mapsto 1\} \cup \{\mathcal{E}_x \mapsto 2, \mathcal{E}_z \mapsto 3\}$ 
9:   end for

10: if  $G_{C_1} \cong G_{C_2}$  then
11:   return EQUIVALENT
12: else
13:   return NOT EQUIVALENT
14: end if
    
```

Local Clifford Equivalence Inspired by [15], the construction can be adapted to LC-EQ STB by replacing each qubit vertex with three Pauli vertices X_i, Y_i , and Z_i forming a triangle, whose automorphism group is the symmetric group $\mathcal{S}_3$. Since $\text{Sp}(2, \mathbb{F}_2) \cong \mathcal{S}_3$, permuting the three vertices represents one of the six possible LC actions on the nonidentity Pauli operators. Different colors of the qubit nodes prevent permutations of the qubits. Thus, this variant of the graph construction is also sound and complete. The stabilizer group elements are connected to

the corresponding Pauli operators on the qubits, as before:

$$\begin{aligned}\mathcal{V} &= \{X_0, \dots, X_{n-1}\} \cup \{Y_0, \dots, Y_{n-1}\} \cup \{Z_0, \dots, Z_{n-1}\} \cup \mathcal{S} \\ \mathcal{E} &= \bigcup_{i=0}^{n-1} \{\{X_i, Y_i\}, \{Y_i, Z_i\}, \{Z_i, X_i\}\} \cup \{\{A_i, g\} \mid g_i = A, g \in \mathcal{S}, A \in \{X, Y, Z\}\} \\ c &: \{\{X_i, Y_i, Z_i\} \mapsto i \mid i \in \{0, \dots, n-1\}\} \cup \{\mathcal{S} \mapsto n\}\end{aligned}$$

With a similar concept as in the previous example, we apply the graph construction algorithm to the codes C_1 and C_2 for LC-EQ STB, removing the permutation of qubit nodes, but adding the ability to permute Pauli operators.

$$\begin{aligned}\mathcal{S}_1 &= \langle IIX, XZI \rangle = \{\mathbf{III}, \mathbf{IIX}, \mathbf{XZI}, \mathbf{XZX}\} \\ \mathcal{S}_2 &= \langle ILY, YZI \rangle = \{\mathbf{III}, \mathbf{IIY}, \mathbf{YZI}, \mathbf{YZY}\}\end{aligned}$$

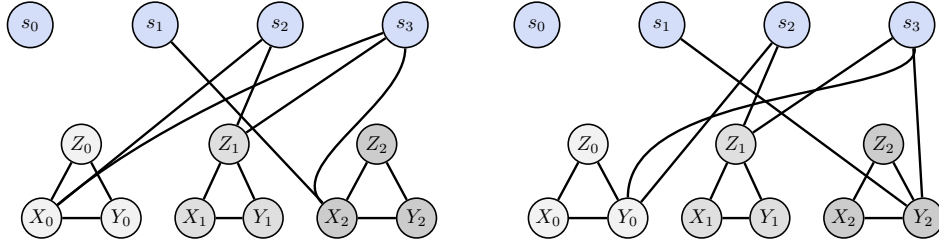


Figure 5.4: Full graphs for C_1 (left) and C_2 (right) from the LC-EQ STB example.

The graphs in Figure 5.4 are isomorphic, using a rotation on qubits 1 and 3.

Complexity

The principal bottleneck is the enumeration of the stabilizer group, which contains 2^r elements for a code with r independent generators. The permutation graph contains $n + 2^r$ vertices and at most $n2^r$ incidence edges. The LC graph contains $3n + 2^r$ vertices, together with $3n$ triangle edges and at most $n2^r$ incidence edges.

Consequently, both the construction time and the memory consumption are exponential in r , even before accounting for the cost of solving the resulting CGI instance.

Key Takeaway

Encoding the entire stabilizer group as a graph while incorporating the degrees of freedom induced by the equivalence relation reduces the problems P-EQ STB and LC-EQ STB to Colored Graph Isomorphism.

5.7 Graph-State Machinery

In contrast to permutation equivalence, local Clifford equivalence has no evident connection to classical coding theory or classical decision problems. Since LC operations are inherently quantum, the relevant literature is largely confined to quantum information theory. A promising approach within this domain uses graph-state methods for deciding local Clifford equivalence. However, because these methods are formulated specifically for graph states, they cannot be

applied directly to stabilizer codes. This section, therefore, first describes a transformation from the given stabilizer codes to graph states and subsequently uses graph-state machinery to solve the LC equivalence problems LC-EQ STB and LC-EQ CSS.

5.7.1 From Stabilizer Code to Stabilizer State

Many of the graph-state techniques considered below operate on stabilizer states rather than stabilizer codes. We therefore require a transformation from an $\llbracket n, k, d \rrbracket$ stabilizer code to a stabilizer state that retains information about the entire code space. Selecting a single logical codeword would generally discard part of this information.

Given a logical framework, the encoding isometry can instead be represented by its *Choi-Jamiołkowski state*, Choi state for short. Equivalently, this state can be viewed as a purification of the maximally mixed state on the code space [1]. The construction introduces k reference qubits and produces a stabilizer state on $n + k$ qubits. Its dependence on the chosen logical framework and the resulting implications for equivalence preservation are discussed subsequently.

Several equivalent routes can be used to perform this transformation. In the following, they are illustrated using code C_1 from the running LC-EQ STB example.

Route A: Stabilizer Tableau Construction

A code space, which is a set of quantum states, can be represented by a uniform mixed state. This mixed state has the density matrix

$$\rho_C = \frac{1}{2^k} \Pi_C$$

using the projector onto the code space in Equation (2.1). Using a given logical framework to fix the previously unspecified logical degrees of freedom, this mixed state can be purified into a larger system by introducing k reference qubits. The exact construction of the stabilizer state from the stabilizer code is described in [1]. The stabilizer group of the purified state is

$$\mathcal{S}_{\text{Choi}} = \langle g_1 \otimes I^k, \dots, g_r \otimes I^k, \bar{X}_1 \otimes X_n, \dots, \bar{X}_k \otimes X_{n+k-1}, \bar{Z}_1 \otimes Z_n, \dots, \bar{Z}_k \otimes Z_{n+k-1} \rangle$$

The resulting stabilizer state $|C\rangle$ contains all the information about the stabilizer group and the encoding of code C :

$$|C\rangle := |\psi\rangle \in \mathcal{H}^{\otimes n+k} \quad \text{such that} \quad \forall s \in \mathcal{S}_{\text{Choi}} : s|\psi\rangle = |\psi\rangle$$

The corresponding stabilizer tableau can be constructed directly from the code's stabilizer tableau and logical framework:

$$\begin{array}{l} S(C) \\ L_x(C) \\ L_z(C) \end{array} \xrightarrow[\text{Tableau}]{\text{Choi}} \left[\begin{array}{cc|cc} S(C)^{(x)} & 0 & S(C)^{(z)} & 0 \\ L_x(C)^{(x)} & I & L_x(C)^{(z)} & 0 \\ L_z(C)^{(x)} & 0 & L_z(C)^{(z)} & I \end{array} \right] \xrightarrow[\text{LC-EQ STB } C_1]{\text{Example}} \left[\begin{array}{ccc|ccc} 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{array} \right]$$

This route provides an efficient and direct construction of the state tableau from the stabilizer tableau and a supplied logical framework. It also makes the dependence on the chosen logical operators explicit, allowing compatibility conditions to be imposed on the logical frameworks when required.

Route B: Circuit Construction

An alternative approach to constructing the Choi state uses an encoder circuit of the code. Such a circuit can be derived solely from the stabilizer tableau using Gaussian elimination, as described in [30, 31]. The choices made during this construction implicitly determine a valid, but generally non-unique, logical framework.

Building upon this encoder circuit, the Choi state is constructed by introducing k reference qubits and preparing Bell pairs between them and the logical input qubits. The resulting circuit produces a stabilizer state, whose tableau can be derived by applying the complete circuit to the initial tableau $[0 \mid I]$.

Although this route is more involved than the direct tableau construction, it provides a more intuitive understanding of the Choi transformation and its dependence on the logical framework. Moreover, because its output is again a stabilizer tableau, it can be incorporated as an independent step into a larger transformation pipeline. However, as it generates implicit logical operators, we cannot impose explicit conditions on the logical frameworks.

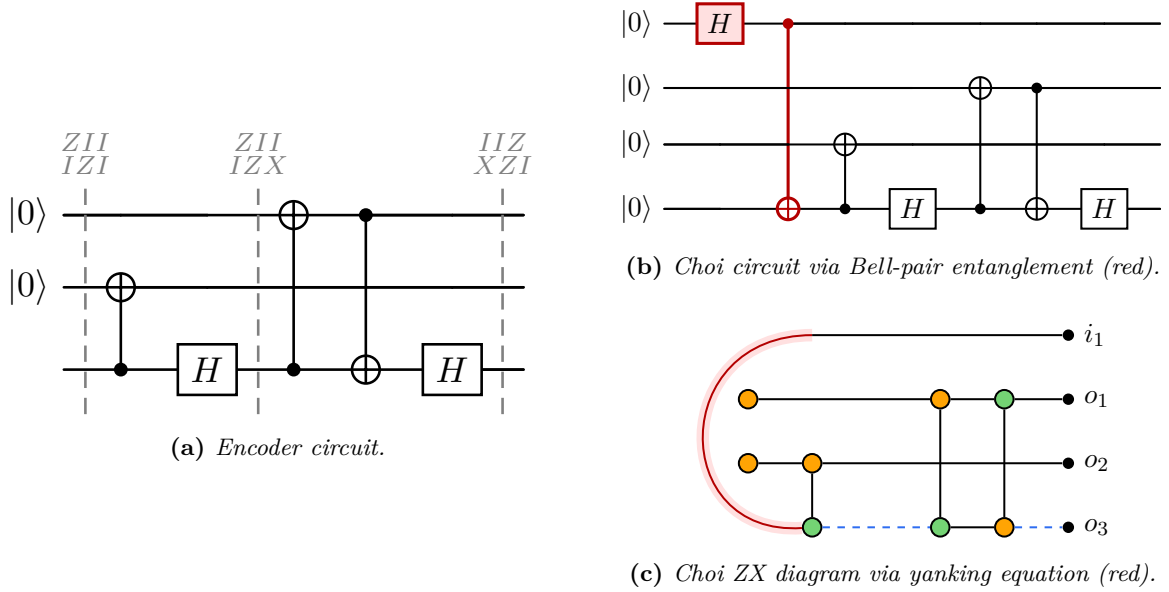


Figure 5.5: Constructing the Choi state of C_1 in LC-EQ STB from an encoder circuit.

The process of transforming the stabilizer tableau of the LC-EQ STB code C_1 into a corresponding Choi state is illustrated in Figure 5.5. The encoder circuit in Figure 5.5a, constructed from the tableau, is the starting point for the Choi transformation.

For the route that uses solely quantum circuits, we need to add one wire representing the reference qubit and entangle it with the input using the circuit to create a Bell state. We then apply all gates of the encoder circuit to the tableau, stabilizing the circuit input state $|0000\rangle$ to obtain the stabilizer tableau for the Choi state:

$$|0000\rangle \rightarrow \left[\begin{array}{cccc|cccc} 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{array} \right] \xrightarrow[\text{CNOT}_{0 \rightarrow 3}]{H_0} \left[\begin{array}{cccc|cccc} 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \end{array} \right] \xrightarrow[\dots]{\text{CNOT}_{3 \rightarrow 2}, H_3} \left[\begin{array}{cccc|cccc} 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \end{array} \right]$$

For the route using the ZX calculus, Figure 5.5c shows the diagram representing the circuit. The ZX diagram is then transformed into a Choi state by applying the yanking equation to the input. The resulting diagram encodes the Choi state and can be simplified further.

Route C: ZX Calculus

Like the circuit route, the ZX-calculus route constructs the Choi state from an encoder circuit. First, the circuit is translated into a ZX diagram by mapping each gate to its corresponding ZX-calculus component. The input wires are then bent using the *yanking equation* [53], thereby producing a diagram representing a state.

Similar to the circuit route, this construction is more involved than the direct tableau route and likewise determines an implicit logical framework. Its output is a ZX diagram rather than a stabilizer tableau, making it less convenient for transformation pipelines that primarily operate on tableau representations. However, the graphical representation enables simplification using ZX-calculus rules.

Limitations

Equivalence Preservation For the resulting state-equivalence problem to provide a sound and complete mapping from the code-equivalence problem, the code-to-state transformation *must* preserve and reflect the LC equivalence relation between two arbitrary codes:

$$C \sim_{LC} C' \iff |C\rangle \sim_{LC} |C'\rangle$$

Revisiting our definition of local Clifford equivalence, we see that it is defined solely in terms of the code space, not the chosen encoder map. However, as previously discussed, the Choi transformation depends on the logical framework, whether explicitly chosen or implicitly determined. The logical framework of a fixed code space is not unique; different choices can yield distinct Choi states. This can result in inequivalent states even when the original codes are equivalent. Essentially, code equivalence is weaker than state equivalence:

$$C \sim_{LC} C' \not\iff |C\rangle \sim_{LC} |C'\rangle$$

Thus, naive LC-equivalence testing of the Choi states is sound but not necessarily complete, potentially leading to false negatives for code equivalence.

A simple, direct LC equivalence check on two Choi states $|C\rangle$ and $|C'\rangle$ is sound and complete for the codes C and C' only if the chosen logical frameworks are *compatible* with the physical LC equivalence. More precisely, let $U \in \mathbb{C}_1^{\otimes n}$ be the LC operation mapping the code space of C to that of C' . Then, the logical frameworks of C and C' are compatible if their logical operators are related by U , up to stabilizer factors:

$$\forall i \in \{1, \dots, k\} : \exists g_{x,i}, g_{z,i} \in \mathcal{S}' : \overline{X}'_i = g_{x,i} U \overline{X}_i U^\dagger \quad \wedge \quad \overline{Z}'_i = g_{z,i} U \overline{Z}_i U^\dagger$$

Under this assumption, the LC operation $\hat{U} = U \otimes I^{\otimes k}$ also maps the corresponding Choi states to one another.

Using the tableau route, we could explicitly impose this compatibility condition on the given logical frameworks of the two codes. However, the circuit and ZX-calculus routes do

not provide a direct way to enforce this condition. Moreover, this condition should not be imposed on the inputs, as LC equivalence is a property of the code space, not the logical framework. Consequently, LC-equivalent codes are not necessarily equipped with compatible logical frameworks. We need to quotient out the concrete choice of logical framework, *before* or *while* utilizing simple LC equivalence methods based on stabilizer states. To account for this freedom, we first characterize the relationships among different logical frameworks.

Logical Freedom In constructing a logical framework for a stabilizer code, there are many arbitrary choices to make. Some of the degrees of freedom are, for instance, the choice of which Pauli operators to assign to the logical operators, the order of the Pauli operators inside one tuple, as well as the order of the tuples themselves.

As already mentioned in Section 2.3.4, all these possible logical frameworks are related to each other by Clifford operations on the logical input qubits. Thus, a change of logical framework corresponds to an arbitrary Clifford operation on the logical input qubits. Below, we show the concrete effect that Clifford operations on the logical qubits have on an initial logical framework $(\bar{X}_1, \bar{Z}_1), \dots, (\bar{X}_i, \bar{Z}_i), \dots, (\bar{X}_j, \bar{Z}_j), \dots, (\bar{X}_k, \bar{Z}_k)$:

$$\begin{aligned} H_i &: (\bar{X}_1, \bar{Z}_1), \dots, (\bar{Z}_i, \bar{X}_i), \dots, (\bar{X}_k, \bar{Z}_k) \\ S_i &: (\bar{X}_1, \bar{Z}_1), \dots, (\bar{X}_i \bar{Z}_i, \bar{Z}_i), \dots, (\bar{X}_k, \bar{Z}_k) \\ CNOT_{i \rightarrow j} &: (\bar{X}_1, \bar{Z}_1), \dots, (\bar{X}_i \bar{X}_j, \bar{Z}_i), \dots, (\bar{X}_j, \bar{Z}_i \bar{Z}_j), \dots, (\bar{X}_k, \bar{Z}_k) \end{aligned}$$

All resulting logical frameworks are valid, satisfying Equation (2.2). Thus, changing the logical framework leaves the represented code space unchanged but induces a Clifford operation on the reference subsystem of the corresponding Choi state, changing the state. This is illustrated for the tableau and ZX diagram representations of the Choi state in Appendix A.

Equivalence Checking As two Choi states should not be considered inequivalent merely because they were constructed using different logical frameworks, this must be taken into account during the equivalence-checking process. States differing by Clifford operations on the reference qubits have to be treated as the same, as their represented codes are the same.

The problem now lies in the fact that methods for checking LC equivalence of stabilizer states only check for *local* Clifford operations on all qubits, including reference qubits. However, as previously established, the equivalence-preserving transformations on the reference qubits are not restricted to local Clifford operations, but can be *arbitrary*—possibly entangling—Clifford operations. Thus, a standard LC-equivalence test on all $n + k$ qubits is guaranteed to be complete only for $k < 2$ or when compatible logical frameworks are used. For $k \geq 2$, the required reference operation need not be local:

$$\begin{aligned} C \sim_{LC} C' &\iff |C'\rangle = (U_{\text{phys}} \otimes U_{\text{ref}})|C\rangle && \text{with } U_{\text{phys}} \in \mathcal{C}_1^{\otimes n}, U_{\text{ref}} \in \mathcal{C}_1^{\otimes k} \\ C \sim_{LC} C' &\iff |C'\rangle = (U_{\text{phys}} \otimes C_{\text{ref}})|C\rangle && \text{with } U_{\text{phys}} \in \mathcal{C}_1^{\otimes n}, C_{\text{ref}} \in \mathcal{C}_k \end{aligned}$$

Consequently, in our equivalence checking process, we have to account for this logical freedom by either extending the allowed equivalence-preserving transformations to arbitrary Clifford operations on the reference qubits, using the reference qubits as a gauge [26], or finding other ways to quotient out the logical freedom.

5.7.2 From Stabilizer State to Graph State

Once a stabilizer code has been mapped to a stabilizer state, the resulting state can be converted into a graph state. As established in Section 2.7.2, every stabilizer state is LC-equivalent to a graph state. The conversion, therefore, remains within the LC orbit of the stabilizer state and preserves LC equivalence between stabilizer states. Any limitations inherited from the preceding code-to-state transformation, however, remain unaffected.

The conversion can be performed using either a stabilizer tableau or a ZX diagram. Both routes are illustrated using the Choi state associated with code C_1 from the running LC-EQ STB example.

Route A: Stabilizer Tableau Construction

A graph state with adjacency matrix Γ has a stabilizer tableau of the form

$$\left[I \mid \Gamma \right],$$

where Γ is symmetric and has a zero diagonal. The objective is therefore to transform the input state tableau into this form using row operations and LC operations.

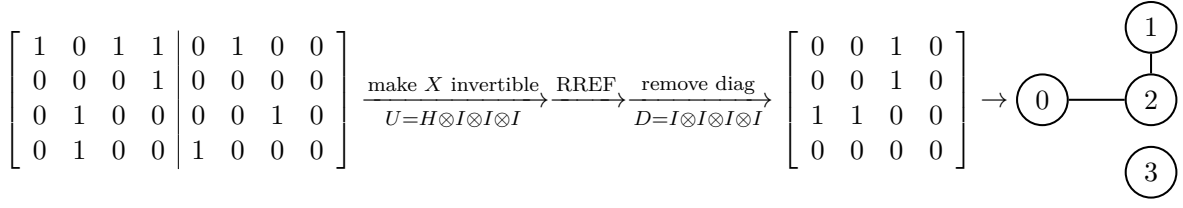
Algorithm 7 summarizes the construction [47, 48]. First, a local Clifford transformation is chosen such that the X -part of the tableau becomes invertible. Gaussian elimination on the tableau then transforms this part into the identity matrix. The commutation relations of the stabilizer generators ensure that Γ' is symmetric. Its diagonal entries correspond to self-loops and are removed by effectively applying suitable phase gates to the respective qubits. The resulting matrix Γ is the adjacency matrix of a simple graph, and the associated graph state is LC-equivalent to the original stabilizer state.

Route B: ZX Calculus

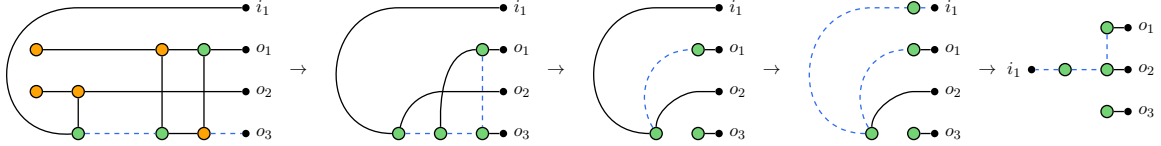
A graph state can also be extracted from a ZX diagram representing the stabilizer state. Using the rewrite rules of the ZX calculus, the diagram is transformed into a graph-like encoder-respecting form satisfying the conditions described in [20, 30].

In this form, the Z -spiders and the edges between them determine an underlying graph state, while local Clifford decorations on the output qubits describe the transformation between this graph state and the original stabilizer state. The underlying graph can therefore be extracted directly from the connectivity of the Z -spiders.

We apply both constructions to the Choi state of code C_1 obtained in Example 5.7.1.



(a) Tableau construction.



(b) ZX-diagram construction.

 Figure 5.6: Code-to-graph constructions, continuing C_1 of LC-EQ STB from Figure 5.5.

In the tableau route shown in Figure 5.6a, a local Clifford operation first makes $S(C_1)^{(x)}$ invertible. Row reduction then yields the adjacency matrix, after which suitable local Clifford operations remove any diagonal entries.

Following the ZX-calculus route in Figure 5.6b, we transform the ZX diagram of the Choi state into a graph-like encoder-respecting form using the simplification rules of the ZX calculus.

In this case, both routes lead to the same graph state, when the LC operations U and D , or the corresponding LC decorations in the diagram, are taken into account. This agreement is specific to the chosen transformations and is not required in general.

Algorithm 7 Transformation State to Graph on Tableau

Require: Stabilizer tableau $S(C) \in \mathbb{F}_2^{n \times 2n}$

Ensure: Graph of an LC-equivalent graph state to $|C\rangle$

- 1: **make X invertible:** $S(C) = [X \mid Z] \xrightarrow{U \in \mathbb{C}_1^{\otimes n}} [X' \mid Z']$ with $\exists (X')^{-1}$
 - 2: **RREF:** $[X' \mid Z'] \rightarrow [I \mid \Gamma']$ # row operations
 - 3: **remove diagonal:** $\Gamma := \Gamma' + \text{diag}(\Gamma')$ # equivalent to $D = S^{\Gamma'_{0,0}} \otimes \dots \otimes S^{\Gamma'_{n-1,n-1}}$
 - 4: **graph construction:** # graph $G = (\mathcal{V}, \mathcal{E})$
 - 5: $\mathcal{V} := \{0, \dots, n-1\}$
 - 6: $\mathcal{E} := \{\{i, j\} \mid \Gamma_{i,j} = 1\}$
 - 7: **return G** # graph state $|G\rangle = DU|C\rangle$
-

Limitations

If only the underlying graph is retained, the local Clifford transformation relating the graph state to the original stabilizer state is lost. This is sufficient for algorithms that operate exclusively on the underlying graphs. However, the original stabilizer state cannot generally be reconstructed from the graph alone.

Algorithms that require an explicit witness, track concrete Clifford operations, or distinguish operations on physical and reference qubits must therefore retain the accompanying local Clifford information. The alternative graph representations introduced below preserve this additional information.

5.7.3 Alternative Graph Constructions

Reduced Stabilizer Graphs

Elliott, Eastin, and Caves introduce *reduced stabilizer graphs* as a graphical representation of stabilizer states in [21]. The construction extends an ordinary graph state by decorating its vertices with node types, self-loops, and signs. These decorations encode the local Clifford operations relating the represented stabilizer state to the underlying graph state.

Unlike a construction that retains only the underlying graph, a reduced stabilizer graph preserves sufficient information to reconstruct the represented stabilizer state. Algorithm 8 summarizes the construction from a stabilizer tableau. Moreover, Elliott et al. provide graphical transformation rules describing the action of arbitrary Clifford operations directly on this representation.

Algorithm 8 Elliott Construction Algorithm

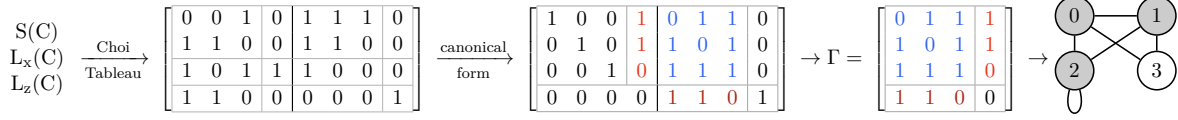
Require: Stabilizer tableau $S(C) \in \mathbb{F}_2^{r \times 2n}$

Ensure: Reduced stabilizer graph of C

- 1: **code to state:** $C \xrightarrow{\text{Choi}} |C\rangle$
 - 2: **canonical form:** $S(|C\rangle) \rightarrow \left[\begin{array}{cc|cc} I & A & B & 0 \\ 0 & 0 & A^\top & I \end{array} \right] \# \text{ row operations + qubit swaps}$
 - 3: **adjacency matrix:** $\Gamma = \left[\begin{array}{cc} B & A \\ A^\top & 0 \end{array} \right]$
 - 4: **swap back qubits:** $\Gamma \rightarrow \Gamma'$
 - 5: **graph construction:** $\# \text{ colored graph } G = (\mathcal{V}, \mathcal{E}), c : \mathcal{V} \rightarrow \{0, 1\}$
 - 6: $\mathcal{V} := \{(i, +) \mid i \in \{0, \dots, n + k - 1\}\}$
 - 7: $\mathcal{E} := \{\{v_i, v_j\} \mid v_i = (i, +), v_j = (j, +) \in \mathcal{V}, \Gamma'_{i,j} = 1\} \# \text{ keep self-loops}$
 - 8: $c := \{\{v \mid v \in \mathcal{V} \text{ in 0-part}\} \mapsto 0, \{v \mid v \in \mathcal{V} \text{ in B-part}\} \mapsto 1\} \# \text{ hollow/solid}$
 - 9: **return** G, c
-

We illustrate the construction of a *reduced stabilizer graph* using the Choi state associated with the running LC-EQ CSS instance. First, the tableau is transformed into the canonical block

form. The submatrices A and B , highlighted in red and blue, respectively, are extracted to construct the adjacency matrix. The resulting graph has one self-loop, one hollow node, and positive signs on each node, which are therefore omitted from the illustration.



KLS Normal Form

Khesin, Lu, and Shor introduce a canonical graphical representation of Clifford encoders in the ZX calculus, here referred to as the *Khesin-Lu-Shor (KLS) normal form* [30, 31]. The representation is an *encoder-respecting*, *semi-bipartite* graph with k input vertices and n ordered output vertices. Input vertices represent the logical qubits, whereas output vertices represent the physical qubits.

A ZX diagram is in KLS normal form if it satisfies the following conditions:

- *Edge rule*: The diagram is graph-like and encoder-respecting.
- *Hadamard rule*: No output node may have both an H gate on its output edge and an edge connecting it to a lower-numbered node.
- *RREF rule*: The partial adjacency matrix of inputs and outputs must be in Row Reduced Echelon Form (RREF).
- *Clifford rule*: Each output node that is associated with a pivot column of the aforementioned partial adjacency matrix is a pivot vertex. Input vertices and pivot vertices carry no local Clifford decorations, and there are no input-input or pivot-pivot edges.

The construction of such a normal form from a stabilizer code tableau in [30, 31] is formulated primarily in terms of the ZX calculus. In the pipeline considered here, the preceding code-to-state and state-to-graph transformations may instead be performed using stabilizer tableaus, provided that all local Clifford operations and the distinction between input and output vertices are retained. The resulting decorated graph is then transformed first into *Hu-Khesin (HK) normal form* [28] and subsequently into KLS normal form. Algorithm 9 summarizes this procedure; the individual normalization rules are described in the cited works.

Role of the Normal Form The principal benefit of the KLS normal form in this thesis is that it removes the freedom associated with the logical framework. Different encoders of the same stabilizer code that differ only by Clifford operations on their logical inputs, are mapped to the same normal form, up to the encoder equivalence defined in [30, 31]. The normal form can therefore be used to quotient out the arbitrary logical-framework choices introduced during the Choi-state construction.

In addition, the representation retains the distinction between logical inputs and physical outputs and visually exposes the code's connectivity structure. The conversion is efficient: the tableau-to-graph compilation described by Khesin, Lu, and Shor has cubic worst-case runtime [31].

Algorithm 9 KLS Canonicalization Algorithm

Require: Stabilizer tableau $S(C) \in \mathbb{F}_2^{r \times 2n}$

Ensure: KLS canonical form of C

- 1: **code to state:** $C \xrightarrow{\text{Choi}} |C\rangle$ # encoder circuit route
 - 2: **state to graph:** $|C\rangle \rightarrow G = (\mathcal{V}, \mathcal{E})$ # record applied LCs U_i and in/-output nodes
 - 3: $\mathcal{V} := \{(i, \text{in}, U_i) \mid i \in \{0, \dots, k-1\}\} \cup \{(i, \text{out}, U_i) \mid i \in \{k, \dots, n+k-1\}\}$
 - 4: $\mathcal{E} \subseteq \{\{i, j\} \mid i, j \in \mathcal{V}\}$

 - 5: **HK normal form:** $G \rightarrow G_{\text{HK}}$ # ensure Hadamard rule

 - 6: **KLS normal form:** $G_{\text{HK}} \rightarrow G_{\text{KLS}}$ # ensure RREF and Clifford rules

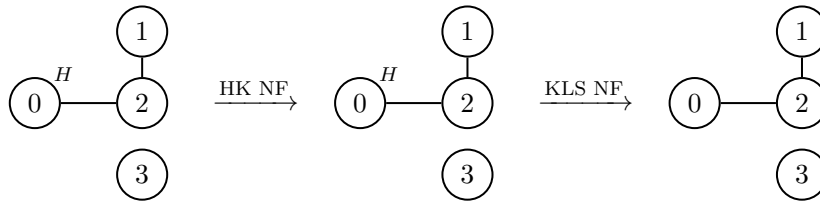
 - 7: **return** G_{KLS}
-

Another useful property of the KLS normal form is that it happens to be bipartite if and only if the represented code is a CSS code [31].

Limitations The KLS normal form is defined relative to a fixed ordering of the physical output vertices. It therefore does not quotient out qubit permutations. However, more importantly for the present application, applying a local Clifford operation to a physical output generally changes the resulting normal form. Consequently, two LC-equivalent codes need not have identical KLS normal forms.

The construction thus provides a canonical representative with respect to logical-input freedom, but not with respect to the complete physical LC orbit. It cannot by itself serve as a prepared canonical form for the code space on which a simple LC-equivalence check can be performed.

Continuing Example 5.7.2 for code C_1 of the running LC-EQ STB instance, we transform the previously obtained decorated graph state into KLS normal form, as it already satisfies the HK normal-form conditions. During the subsequent KLS normalization, the Hadamard decoration on the input vertex is absorbed:



5.7.4 Local Clifford Equivalence of Graph States

Having established transformations from stabilizer codes to graph states, we now consider how graph-state techniques can be used to decide LC-EQ STB and LC-EQ CSS. The following approaches differ in how they explore local Clifford equivalence and in how they account for the restrictions imposed by the preceding transformations.

Orbit Traversal

The local Clifford orbit of a graph state is the set of all graph states that can be reached from the given graph state by applying a sequence of local Clifford operations [2, 11, 16]. Thus, traversing this orbit allows us to explore all graph states that are LC-equivalent to the given graph state.

Using results described in Section 2.7.2, we can define success criteria depending on the equivalence problem under consideration. For LC-EQ STB, the objective is to determine whether the graph-state representation of the second code occurs in the LC orbit of the first. For LC-EQ CSS, the objective is to determine whether the orbit contains a bipartite graph, since this certifies the existence of an LC-equivalent CSS state.

However, as discussed in Section 5.7.1, a simple LC orbit traversal on the Choi states is not guaranteed to be complete for both LC-EQ STB and LC-EQ CSS. Thus, in this section, we aim to address the limitations of the previous transformation.

A further practical limitation is the size of the orbit, which may grow exponentially with the number of qubits [2, 17]. Complete orbit traversal may consequently become infeasible for larger instances, even when the logical-framework issue has been resolved.

Assumptions The simplest case occurs when the code-to-state transformation preserves and reflects the relevant equivalence relation for the considered instance. As established above, this is guaranteed when $k < 2$ or when the codes' logical frameworks are compatible. Under these conditions, direct traversal of the LC orbit of the Choi states is sound and complete for deciding LC-EQ STB and LC-EQ CSS.

For graph states, local Clifford transformations can be represented by sequences of *local complementations* on the underlying graph [2, 8, 47]. A local complementation at a vertex v replaces the subgraph induced by the neighborhood of v with its graph-theoretic complement. Each such operation can be performed efficiently and produces a graph state that is LC-equivalent to the original one.

Consequently, the local Clifford decorations introduced during the state-to-graph transformation need not be retained in this case. It is sufficient to traverse the local-complementation orbit of the underlying graph until either the target graph is reached for LC-EQ STB or a bipartite graph is found for LC-EQ CSS. If the complete orbit is exhausted without satisfying the respective criterion, the instance is negative.

Extended Clifford Orbit If the assumptions above are not satisfied, the logical freedom must be incorporated explicitly into the equivalence check. In this case, the LC information introduced during the state-to-graph transformation must be retained.

As a simple LC orbit traversal on the Choi states is not guaranteed to be complete, we have to traverse the full Clifford orbit on the logical qubits as well. Consequently, in addition to all possible LC operations on the n physical qubits, we have to consider all possible arbitrary Clifford operations on the k reference qubits. Unlike local Clifford operations on graph states, arbitrary reference local complementations cannot generally represent Clifford operations on a bare graph. A representation retaining the Clifford decorations, such as a reduced stabilizer graph or a ZX diagram, is therefore required.

Unfortunately, the size of the reference Clifford orbit grows super-exponentially with k and rapidly becomes infeasible. As mentioned in Section 2.5.1, the number of Clifford operations on

k qubits is given by $|\mathcal{C}_k| = 2^{k^2+2k} \prod_{j=1}^k (4^j - 1)$, which is significantly larger than the size of the local Clifford group on n physical qubits.

Figure 5.7 illustrates two representations of orbit traversal for the running LC-EQ CSS instance.

On the left in that figure, only the underlying graph from the following construction is retained:

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \xrightarrow[U=H \otimes I \otimes I \otimes I]{\text{make } X \text{ invertible}} \xrightarrow{\text{RREF}} \xrightarrow[D=I \otimes I \otimes S \otimes I]{\text{remove diag}} \Gamma = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \rightarrow \begin{array}{cc} (0) & (1) \\ (2) & (3) \end{array}$$

Local complementation at qubit 0 produces a bipartite graph and thus yields a positive witness.

On the right, the same search is represented using the reduced stabilizer graph obtained in Example 5.7.3, which retains the local Clifford decorations and permits Clifford operations to be propagated explicitly. In the illustrated case, applying a Hadamard to qubit 2 yields a representation with a bipartite underlying graph.



Figure 5.7: LC (left) and Clifford (right) orbit traversal for the LC-EQ CSS example.

Since this example has only one reference qubit, it does not exhibit the additional complexity caused by entangling Clifford operations on a multi-qubit reference subsystem. Such operations arise only for $k \geq 2$.

Canonicalized Orbit Traversal An alternative way to account for logical-framework freedom is to combine physical LC-orbit traversal with repeated normalization into KLS normal form. First, the input codes are transformed into their respective KLS normal forms. The orbit is then explored by applying local Clifford operations to the physical output vertices and renormalizing the resulting encoder graph after each operation.

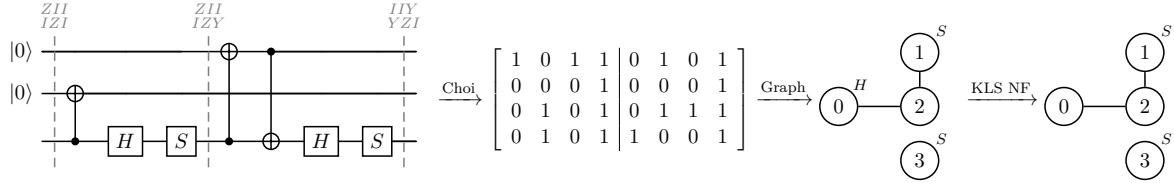
For LC-EQ STB, each normalized representative is compared with the KLS normal form of the second code. For LC-EQ CSS, each representative is tested for bipartiteness, since a bipartite KLS graph corresponds to a CSS code.

The principal advantage of this approach is that KLS normalization quotients out the logical-input Clifford freedom at every step. It therefore avoids explicit traversal of the full Clifford orbit on the reference subsystem. A physical local Clifford operation can be applied as a decoration of the corresponding output vertex before the graph is renormalized.

The approach, however, does not eliminate traversal of the physical LC orbit, whose number of distinct representatives may still grow exponentially. Moreover, each visited representative requires an additional KLS normalization, introducing polynomial overhead at each step. Thus,

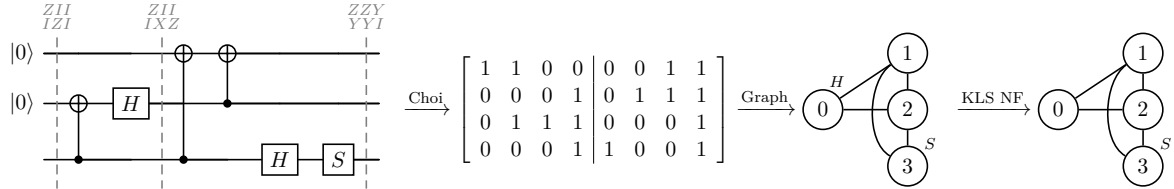
normalization may merge branches that differ only in their logical framework, but it does not eliminate the principal cost of exploring physical local Clifford transformations.

We first apply canonicalized orbit traversal to the running LC-EQ STB instance. The KLS normal form of code C_1 was obtained in Example 5.7.3; the corresponding construction for C_2 is summarized below:



In this instance, the two initial normal forms already reveal a suitable LC transformation on the physical outputs, so no extended orbit traversal is required.

For LC-EQ CSS, we first construct the initial KLS normal form from the code's encoder circuit:



Next, we traverse the LC orbit using recanonicalization. In this case, applying S on qubit 1 is sufficient to find a KLS normal form that happens to be a bipartite graph, as shown in Figure 5.8.

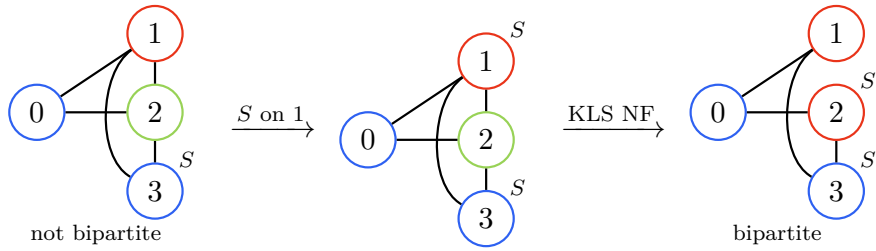


Figure 5.8: Orbit traversal using the KLS normal form for the LC-EQ CSS example.

Key Takeaway

For LC-EQ CSS, we can explore the equivalence class using several strategies: traversing only the LC orbit for restricted instances; additionally traversing the full Clifford orbit on the reference qubits; and traversing the LC orbit with KLS normalization. For LC-EQ STB, we consider only traversal of the LC orbit combined with normalization.

Efficient Algorithm via Local Complementation

A central result for the graph-state approaches considered in this thesis is the polynomial-time algorithm of Van den Nest, Dehaene, and De Moor for deciding LC equivalence of graph states [47]. Their construction adapts Bouchet's polynomial-time algorithm for recognizing graphs related by local complementations [7,8] to the binary symplectic representation of graph states.

The algorithm relies on the correspondence of LC operations on graph states and local complementations on the underlying graphs. In contrast to explicit orbit traversal, the algorithm determines whether such a sequence exists without enumerating the orbit.

Algorithm A local Clifford transformation is represented by diagonal matrices as introduced in Equation (2.4). Requiring this transformation to map the first graph state with adjacency matrix Γ to the second with adjacency matrix Γ' yields a homogeneous Linear System of Equations (LSE) over $\mathbb{F}_2$ consisting of n^2 equations in the $4n$ unknowns

$$a_0, \dots, a_{n-1}, \quad b_0, \dots, b_{n-1}, \quad c_0, \dots, c_{n-1}, \quad d_0, \dots, d_{n-1}.$$

A solution of the LSE represents a valid LC transformation only if every local block is invertible. The additional constraints are therefore

$$\forall i \in \{0, \dots, n-1\} : \quad a_i d_i + b_i c_i = 1.$$

The graph states are LC-equivalent if and only if the solution space $\mathcal{M}$ of the LSE contains a vector satisfying all of these constraints.

Naively searching $\mathcal{M}$ would require exponential time. However, Bouchet's result substantially reduces the number of candidates that must be examined, under the assumption that the two graphs are connected graphs. In particular, if $\dim(\mathcal{M}) > 4$, then the LSE has a solution that satisfies the additional conditions, if and only if the set

$$\mathcal{M}' = \{b + b' \mid b, b' \in B\} \subseteq \mathcal{M} \quad \text{with } B \text{ a basis of } \mathcal{M}$$

contains a solution that satisfies the constraints.

Algorithm 10 summarizes this procedure. Since local complementation neither creates edges between connected components nor merges distinct components, disconnected graphs can be handled component-wise once their component partitions are verified to agree. The graphs are LC-equivalent if and only if all corresponding components are locally equivalent.

Complexity Constructing the linear system requires polynomial time. The coefficient matrix has n^2 rows and $4n$ columns, so computing a basis of its kernel by Gaussian elimination requires at most $\mathcal{O}(n^4)$ field operations using a direct implementation. If the solution space dimension is at most four, at most 16 candidate vectors are tested. Otherwise, the basis contains at most $4n$ vectors, yielding $\mathcal{O}(n^2)$ pairwise sums. Testing the constraints for each candidate remains polynomial.

The complete algorithm therefore decides LC equivalence of graph states in polynomial time in the number of qubits n .

Algorithm 10 Efficient Algorithm for LC-Equivalence of Graphs

Require: Connected graphs G_1, G_2 with $\mathcal{V}_1 = \mathcal{V}_2 = \{0, \dots, n-1\}$, adjacencies $\Gamma^{(1)}, \Gamma^{(2)}$

Ensure: Whether there is a sequence of local complementations transforming G_1 into G_2

```

1: solve lse: #  $n^2$  equations,  $4n$  unknowns
2:    $M \rightarrow j, k \in \{0, \dots, n-1\} : (\sum_{i=0}^{n-1} \Gamma_{i,j}^{(1)} \Gamma_{i,k}^{(2)} c_i) + \Gamma_{j,k}^{(1)} a_k + \Gamma_{j,k}^{(2)} d_j + \delta_{j,k} b_j = 0$ 
3:   # solution  $v = (a_0, \dots, a_{n-1}, b_0, \dots, c_0, \dots, d_0, \dots)^T \in \text{kernel}(M)$ 

4: if  $\dim(\text{kernel}(M)) > 4$  then # possible simplification
5:    $\mathcal{M}' := \{b + b' \mid b, b' \text{ basis vectors of } \text{kernel}(M)\} \subseteq \text{kernel}(M)$ 
6:   if  $\exists v \in \mathcal{M}' : \forall i \in \{0, \dots, n-1\} : a_i d_i + b_i c_i = 1$  then
7:     return EQUIVALENT
8:   end if
9: else
10:  for  $v \in \text{kernel}(M)$  do # full search of solution space
11:    if  $\forall i \in \{0, \dots, n-1\} : a_i d_i + b_i c_i = 1$  then
12:      return EQUIVALENT
13:    end if
14:  end for
15: end if
16: return NOT EQUIVALENT
    
```

Limitations The algorithm operates on two explicitly given graph states. Applying it to stabilizer codes, therefore, requires the preceding code-to-state and state-to-graph transformations. Since the algorithm is tailored to checking pure LC equivalence on graph states, it cannot account for the logical freedom introduced by the code-to-state transformation, which is discussed in Section 5.7.1. The resulting procedure remains sound for LC-EQ STB, but is guaranteed to be complete only when $k < 2$ or when compatible logical frameworks are used.

Furthermore, the algorithm requires both target adjacency matrices in order to construct the linear system. It therefore does not directly solve LC-EQ CSS, where no target CSS code or bipartite graph is given, and the task is instead existential.

Key Takeaway

The proposed efficient algorithm for deciding LC equivalence can only be applied to restricted instances of LC-EQ STB.

5.8 Comparison of Approaches

The following tables summarize the prototype approaches considered for the four equivalence problems, including their principal correctness guarantees and theoretical worst-case complexities in terms of the input-code dimensions. Although several of these bounds appear untenable, their practical performance is investigated in the following chapters.

While the quality criteria do not require a witness, each algorithm can be extended to return an explicit equivalence transformation.

Table 5.5: Comparison of the proposed approaches.

Problem	Approach	Sec.	Sound	Complete	Runtime
P-EQ STB	Brute Force	5.3	✓	✓	$\mathcal{O}(n! \cdot (rn + r^2n))$
	SAT	5.4	✓	✓	$\mathcal{O}(2^{n^2+2nr+r^2})$
	Classical	5.5	✓	✓	$\mathcal{O}(2^r + n! \cdot r^2n)$
	Automorphism	5.5	✓	✓	$\mathcal{O}(n!)$
	GI	5.6	✓	✓	$\mathcal{O}(2^r + (m \cdot (n + 1))!)$
P-EQ CSS	Brute Force	5.3	✓	✓	$\mathcal{O}(n! \cdot ((r_x + r_z)n + (r_x^2 + r_z^2)n))$
	SAT	5.4	✓	✓	$\mathcal{O}(2^{n^2+n(r_x+r_z)+(r_x^2+r_z^2)})$
	Classical	5.5	✓	✓	$\mathcal{O}(2^k + n! \cdot k^2n)$
	GI	5.6	✓	✓	$\mathcal{O}(2^r + (n + 2^r)!)$
	MI	5.6	✓	✓	$\mathcal{O}(2^n + (2 \cdot 2^n + n)!)$
LC-EQ STB	Brute Force	5.3	✓	✓	$\mathcal{O}(6^n \cdot (rn + r^2n))$
	SAT	5.4	✓	✓	$\mathcal{O}(2^{6n+2nr+r^2})$
	GI	5.6	✓	✓	$\mathcal{O}(2^r + (3n + 2^r)!)$
	KLS Orbit	5.7	✓	✓	$\mathcal{O}(2^n \cdot n^3)$
	Graph-state LSE	5.7	✓	×	$\mathcal{O}(2^n)$
LC-EQ CSS	Brute Force	5.3	✓	✓	$\mathcal{O}(6^n \cdot (rn + 2r^2n))$
	SAT	5.4	✓	✓	$\mathcal{O}(2^{6n+2nr+r^2})$
	LC Orbit	5.7	✓	×	$\mathcal{O}(2^n)$
	KLS Orbit	5.7	✓	✓	$\mathcal{O}(2^n \cdot n^3)$
	Clifford orbit	5.7	✓	✓	$\mathcal{O}(2^n \cdot 2^{n^2})$

6 Implementation

Having developed theoretical approaches to equivalence problems, we now evaluate their suitability for automated equivalence checking using prototype implementations. This chapter presents the software architecture and discusses design decisions concerning runtime efficiency, memory efficiency, and maintainability.

6.1 Software Architecture

The prototypes are available in the public repository eQECC. It comprises independent prototype implementations of the proposed algorithms and invariants, together with the supporting infrastructure for testing and benchmarking. The software is implemented in Python and uses established libraries for linear algebra (`numpy`, `ldpc`, and `gap-guava`), graph isomorphism (`pynauty`), and constraint solving (`z3-solver`).

The representation layer builds on data structures from MQT-QECC, which provide representations of stabilizer and CSS codes consistent with the theoretical input model. We extend this layer with dedicated types for $\mathbb{F}_4$ elements and graphs carrying LC decorations.

The architecture is modular and extensible: algorithms are implemented as self-contained modules organized by equivalence problem. A comprehensive validation framework supports unit, smoke, and regression testing using both randomized and deterministic test cases. This framework verifies individual components, complete algorithms, and previously established results.

6.2 Integration Design

As the algorithms are intended for integration into MQT-QECC, the prototypes were designed to remain compatible with its existing architecture. They use the same programming language and software stack, thereby avoiding additional dependencies and their associated maintenance burden.

Where possible, the implementations operate directly on MQT-QECC code representations. Specialized data structures, such as decorated graphs, are introduced only where required by an algorithm, limiting conversion overhead and reducing the changes required for future integration.

The prototypes also follow MQT-QECC's coding and documentation conventions, including consistent naming, formatted and typed code, explicit interfaces, and comprehensive docstrings. These design decisions facilitate review, integration, and subsequent maintenance within the existing project.

6.3 Implementation Choices

6.3.1 Runtime

Considering the practical applicability of the algorithms, we focus on runtime efficiency in this section. Although implementation choices do not change the asymptotic complexity of the proposed algorithms, they can considerably affect their practical runtime. We therefore avoid redundant computations through techniques such as lazy evaluation and caching where appropriate. Frequently used elementary operations are implemented using lookup tables, while recurring graph structures are cached for efficient access. We further use efficient built-in representations, such as integers for bitmasks and hashes for object comparison. Finally, computationally intensive operations are delegated to optimized library implementations where possible, reducing the performance overhead associated with Python.

6.3.2 Memory

In addition to runtime, memory consumption is a central constraint on the practical applicability of the algorithms. Excessive runtime may remain manageable for small instances by allowing longer execution, whereas exceeding the available memory typically causes the computation to terminate. Careful control of peak memory usage is therefore particularly important and depends on appropriate implementation choices.

Where possible, the prototypes avoid materializing large intermediate results. For example, during matroid circuit extraction, candidate sets are generated and refined using streaming filters, rather than storing all 2^r candidates simultaneously before selecting the inclusion-minimal sets. This reduces peak memory consumption at the cost of some additional processing overhead.

Temporary data structures are also discarded once the required information has been extracted. In graph-isomorphism checks, for instance, each graph can be released after computing its certificate, rather than retaining both graphs throughout the comparison. Representations are further adapted to the structure of the data: symmetric matrices require only one triangular half to be stored, while sparse graphs can be represented by their edges rather than by a full adjacency matrix.

7 Experimental Evaluation

This chapter evaluates the practical performance of the approaches introduced in Chapter 5. The experiments examine their computational efficiency, scalability, and practical applicability. The results subsequently inform the selection and combination of components for the hybrid algorithms developed in Chapter 8.

7.1 Analytical Objectives

Invariants The invariant benchmarks assess whether the selected higher-complexity invariants are computationally viable as preliminary filters for each equivalence problem. In particular, the evaluation determines the parameter ranges within which these invariants can reduce overall computational effort without becoming a performance bottleneck themselves. Their runtime is therefore considered relative to that of the corresponding equivalence algorithms.

Algorithms For each equivalence problem defined in Chapter 4, the evaluation compares the runtime and memory characteristics of the implemented prototypes. It further investigates how their performance scales with the relevant problem parameters and identifies the parameter ranges in which each approach remains practically feasible. The experiments are conducted at the level of complete algorithms, not individual functions, because the objective is to assess end-to-end performance and scalability, rather than isolated implementation details.

7.2 Experimental Setup

All benchmarks were executed in the environment summarized in Table 7.1. Each benchmark process was subject to a runtime limit of 5400s for full prototype algorithms and 60s for invariants, and a memory limit of 13 GiB. This computational budget defines the practical feasibility criteria used throughout the evaluation. While runtime is recorded for all executions that complete within the timeout, memory consumption is monitored only against its specified limit. Consequently, the evaluation distinguishes successful runs from Out-of-Memory (OOM) runs but does not provide fine-grained memory measurements. All randomized experiments used a global seed of 42 to ensure reproducibility.

7.2.1 Input Instance Generation

A benchmark *instance* consists either of a pair of codes (C, C') that are equivalent or inequivalent under the considered equivalence notion, or of a single code C' that is either LC-equivalent to some CSS code or not LC-equivalent to any CSS code. The generated input codes for the prototype algorithms may themselves be either CSS codes or general stabilizer codes.

Table 7.1: *Hardware and software environment used for the experiments.*

Hardware		Software	
Architecture	x86_64	Operating system	Debian GNU/Linux 13.5
Processor	AMD Ryzen 7 3700X (8 Cores)	Kernel	Linux 6.12.90+deb13.1-amd64
Memory	32 GB DDR4-3200 (2 × 16 GB, CL16)	Python	Version 3.13.5
		NumPy	Version 2.4.6
		LDPC	Version 2.4.1
		PyNauty	Version 2.8.8.1
		Z3 Solver	Version 4.16.0
		PyZX	Version 0.10.3

Generation of the Base Code C A base code C is obtained either from a collection of structured codes or through randomized generation. *Random codes* are constructed by applying a randomly sampled sequence of arbitrary Clifford operations to a standard-form tableau, which is parameterized by the code length n and dimension k . *Structured codes* are drawn from the MQT-QECC project and the library QECirc. Throughout the evaluation, they are referenced using the identifiers listed in Table 7.2.

Table 7.2: *Structured codes used for evaluation.*

Code	ID	(n, k)	Code	ID	(n, k)
Bell Pair	<i>bell</i>	(2, 0)	15-Qubit Optimal	<i>optimal</i>	(15, 3)
3-Qubit Repetition	<i>rep</i>	(3, 1)	Hamming	<i>hamm-n</i>	(15, 7); (31, 21)
5-Qubit Perfect	<i>perf</i>	(5, 1)	Golay	<i>golay</i>	(23, 1)
Steane	<i>steane</i>	(7, 1)	Rotated Surface D5	<i>rotated</i>	(25, 1)
Shor	<i>shor</i>	(9, 1)	Color 4.8.8	<i>sqr-oct</i>	(31, 1)
Carbon	<i>carbon</i>	(12, 2)	Color 6.6.6	<i>hexagon</i>	(37, 1)
Tetrahedral	<i>tetra</i>	(15, 1)	Bivariate Bicycle (BB) [10]	<i>bb-n</i>	(72, 12); (90, 8); (108, 8); (144, 12)

Generation of the Comparison Code C' For pairwise equivalence problems, the base code C is supplemented by a code C' of the same type and dimension. Positive instances are constructed by applying randomly sampled transformations that preserve the equivalence relation under consideration. For negative instances, candidate codes of the same type and dimension are sampled randomly. A candidate is accepted if it differs from C with respect to an invariant that is not evaluated by the benchmarked algorithm. If no such candidate is found after a fixed number of attempts, the instance is discarded.

7.2.2 Resource Restrictions

To enable timeouts and memory monitoring, each benchmark case—i.e., the execution of a specific algorithm on a specific input—is run as a separate subprocess. While supervising memory usage introduces some runtime overhead, it is necessary to prevent the system from running out of memory. In this evaluation, the overhead is considered acceptable because it does not significantly affect the overall assessment of the algorithm's performance.

7.3 Methodology

7.3.1 Statistical Variation across Randomized Instances

For each *measurement*, we fix one parameter setting—either a specific pair (n, k) or a particular structured code—and generate multiple instances, i.e., concrete inputs for the decision algorithms, using different random seeds. We typically generate 5 instances for invariant benchmarks and 10 instances for algorithm benchmarks. These samples allow us to assess the variability in algorithm performance across instances of the same problem size and compute statistical measures such as their mean and standard deviation. This provides a more robust characterization of algorithm behavior on randomized inputs. In addition to supporting reproducibility, this approach helps identify instances whose runtimes differ significantly due to properties other than size, such as symmetry or the density of their tableaux.

To conserve computational resources, we dynamically adjust the number of instances evaluated for a given parameter setting. If several seeds already result in timeouts or memory errors, we reduce the number of remaining instances for that setting, since additional seeds are likely to encounter similar limitations.

Because this evaluation does not focus on platform-specific runtime variation for identical inputs, each instance is executed only once. Preliminary experiments indicated that the runtime variation between repeated executions of the same instance is negligible compared with that between different instances generated under the same parameter setting.

7.3.2 Selection of Parameter Settings

To obtain a representative overview of the algorithms’ scaling behavior, we aim to measure across a broad range of parameter settings, although exhaustive evaluation of all possible settings is unrealistic. Instead, we select a subset that still covers a sufficiently broad range of parameters. To conserve computational resources, parameter thresholds for n and k are dynamically adjusted based on the algorithms’ observed feasibility.

7.4 Results

7.4.1 Figure Conventions

The figures in this chapter use a consistent set of visualization conventions for runtime heatmaps. They visualize the mean runtime of the benchmarked algorithms as a function of the two relevant problem parameters, n and r . Each cell corresponds to a specific measurement, with its color indicating the mean runtime of the associated randomized instances on a logarithmic scale. The color scale is consistent across all relevant heatmaps, allowing for direct comparison of the measured runtimes. Unless stated otherwise, each cell aggregates both positive and negative randomized benchmark instances. Separate heatmaps for positive and negative instances are provided in Appendix A. Pink lines serve as common visual guides, indicating the predominant parameter dependence. A vertical line highlights a dependence on n , whereas a horizontal line highlights a dependence on r . The positions of these lines are chosen as visual guidance and do not necessarily represent a clear feasibility threshold. Blue crosses indicate measurements for which at least one input instance exceeded the memory limit.

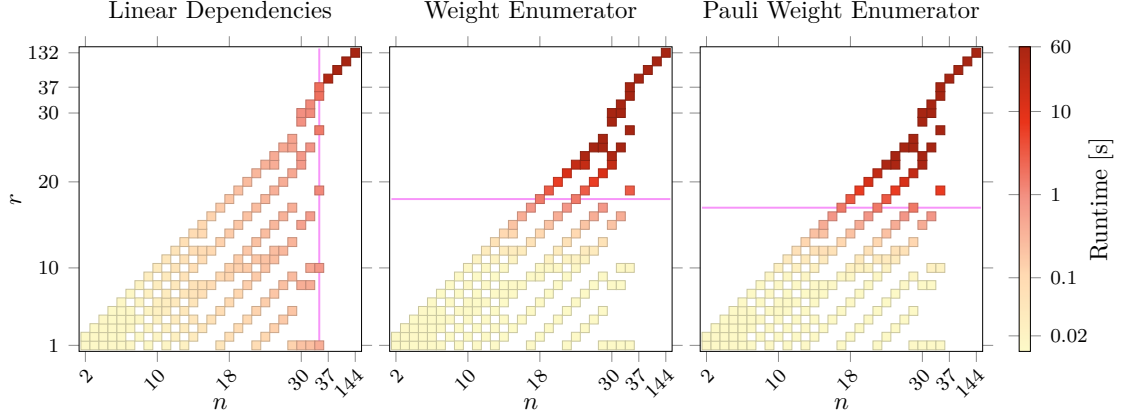


Figure 7.1: *Runtimes of the three permutation invariants on randomized stabilizer-code instances.*

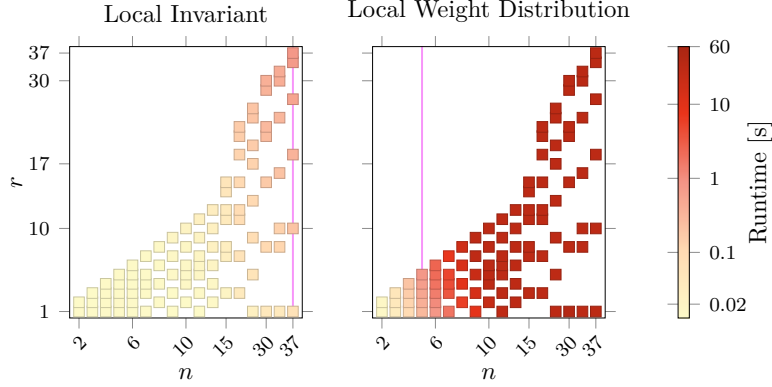


Figure 7.2: *Runtimes of the two LC invariants, both with maximum subset size $m = 2$, on randomized stabilizer-code instances.*

7.4.2 Invariant Benchmarks

This section evaluates the choice of invariants and their parameter-dependent runtime behavior. We consider the computationally more expensive permutation invariants introduced in Section 5.1.1, namely linear dependencies among subsets of at most three columns of the stabilizer tableau and the (Pauli-sensitive) weight enumerators. For LC equivalence, we consider the local weight distribution and local invariant introduced in Section 5.1.2, both evaluated at degree 2.

Figure 7.1 compares the runtime behavior of the three permutation invariants. The runtimes of the linear-dependency invariant and the weight enumerators exhibit clear dependencies on n and r , respectively. Although the difference is not pronounced for smaller instances, the linear-dependency invariant generally has a lower runtime than both weight enumerators and scales more favorably for larger codes.

Regarding the two LC invariants, Figure 7.2 shows the considerably more favorable scaling of the local invariant compared to the weight distribution. Both primarily depend on n and use a maximum subset size of 2, motivated by the results in Appendix A.

These results identify *which* invariant is preferable for each equivalence notion. However, determining *when* to apply it is beneficial and requires comparing it with the preferred algorithm for each equivalence problem. Thus, we first evaluate the prototypes in the following section.

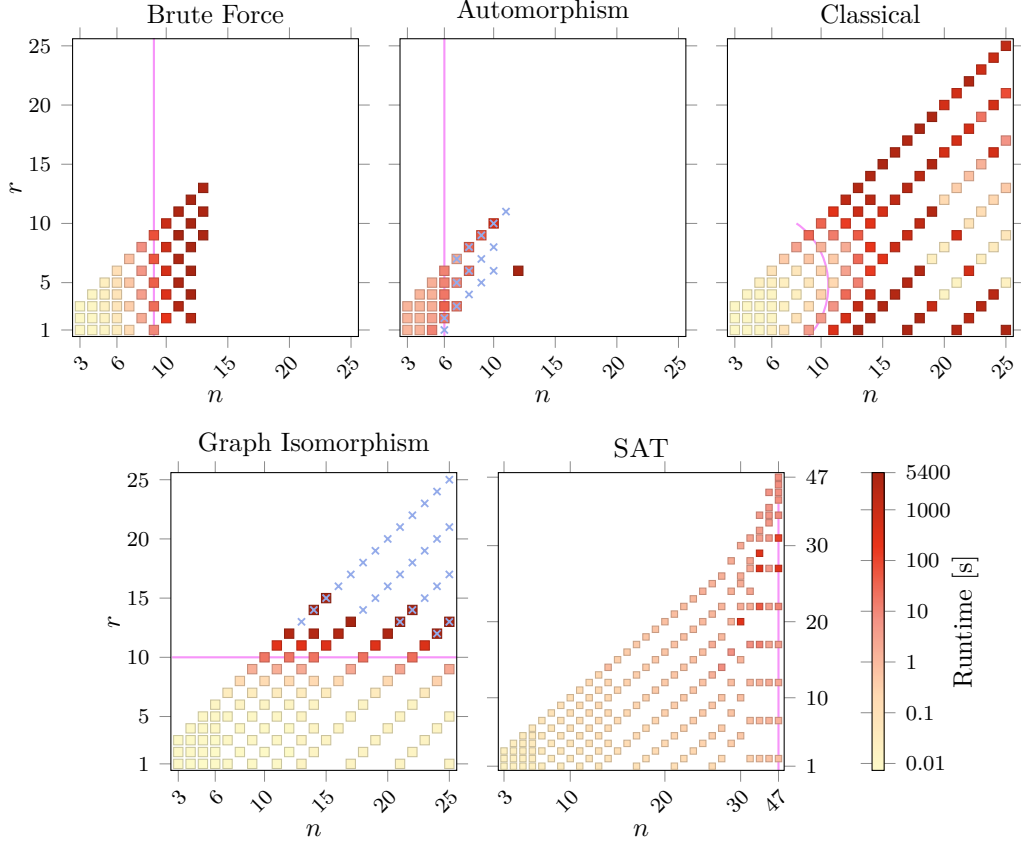


Figure 7.3: *Runtimes of the prototypes for P-EQ STB on randomized stabilizer-code instances.*

7.4.3 Performance Comparison of Algorithmic Approaches

The comparison focuses on the runtime and memory behavior of the algorithmic approaches for each equivalence problem. In particular, we examine their dependence on the problem parameters and identify the approaches that remain practically feasible across the largest parameter ranges.

P-EQ STB Figure 7.3 presents the runtime heatmaps of the permutation-equivalence algorithms for general stabilizer codes listed in Table 5.5. The approaches exhibit substantially different scaling behavior. The runtimes of the brute-force, automorphism-based, and SAT-based approaches depend primarily on n , whereas the GI-based approach exhibits a stronger dependence on r . The classical approach displays a less uniform pattern, with comparatively higher runtimes for boundary values of r —which becomes more apparent in the separated negative instances in Figure A.5a. OOM runs occur predominantly for the GI- and automorphism-based approaches. Thus, most prototypes are limited to a very narrow range of parameters due to practical feasibility constraints. The SAT-based approach, by contrast, exhibits the lowest runtime for the majority of parameter settings. Although several high-runtime outliers occur—particularly among negative instances, as apparent in Figure A.5a—it successfully terminated within the timeout throughout most of the evaluated range.

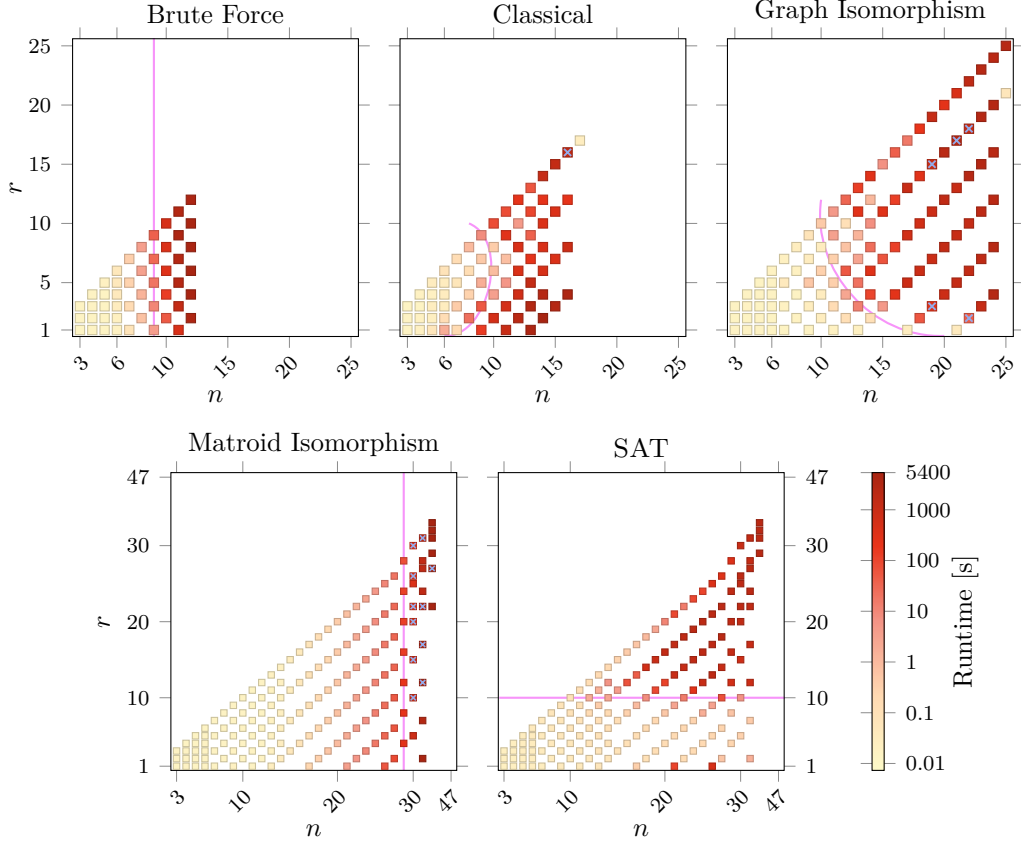


Figure 7.4: *Runtimes of the prototypes for P-EQ CSS on randomized CSS-code instances.*

P-EQ CSS For the specialization of this equivalence problem to CSS codes, Figure 7.4 shows the runtime heatmaps of the corresponding algorithms listed in Table 5.5. The runtimes of the brute-force and matroid-based approaches exhibit a strong dependence on n , whereas the remaining approaches show no clear one-dimensional dependence. As shown in Figure A.5b in the Appendix, both the invariant-based graph-isomorphism approach and the classical approach perform considerably better on negative instances. Although only the matroid- and graph-based approaches encounter memory errors, the MI prototype remains applicable across the broadest range of parameter values. In contrast to P-EQ STB, the SAT-based approach performs poorly for codes with large r , and additionally exhibits a high runtime variance for certain parameter settings.

LC-EQ STB Figure 7.5 illustrates the runtime heatmaps of the approaches listed in Table 5.5, deciding LC equivalence for two stabilizer codes. Similar to the results for P-EQ STB, the brute-force and SAT-based approaches depend predominantly on n , whereas the GI-based approach displays a stronger dependence on r . Traversing the LC orbit utilizing the KLS normal form exhibits a less distinct scaling pattern. OOM runs occur only for the two graph-based approaches, namely GI and KLS, even though the GI-based approach achieves low runtimes for small values of r . The LSE-based approach performs particularly well within its restricted domain of applicability. However, overall, the SAT-based approach provides the

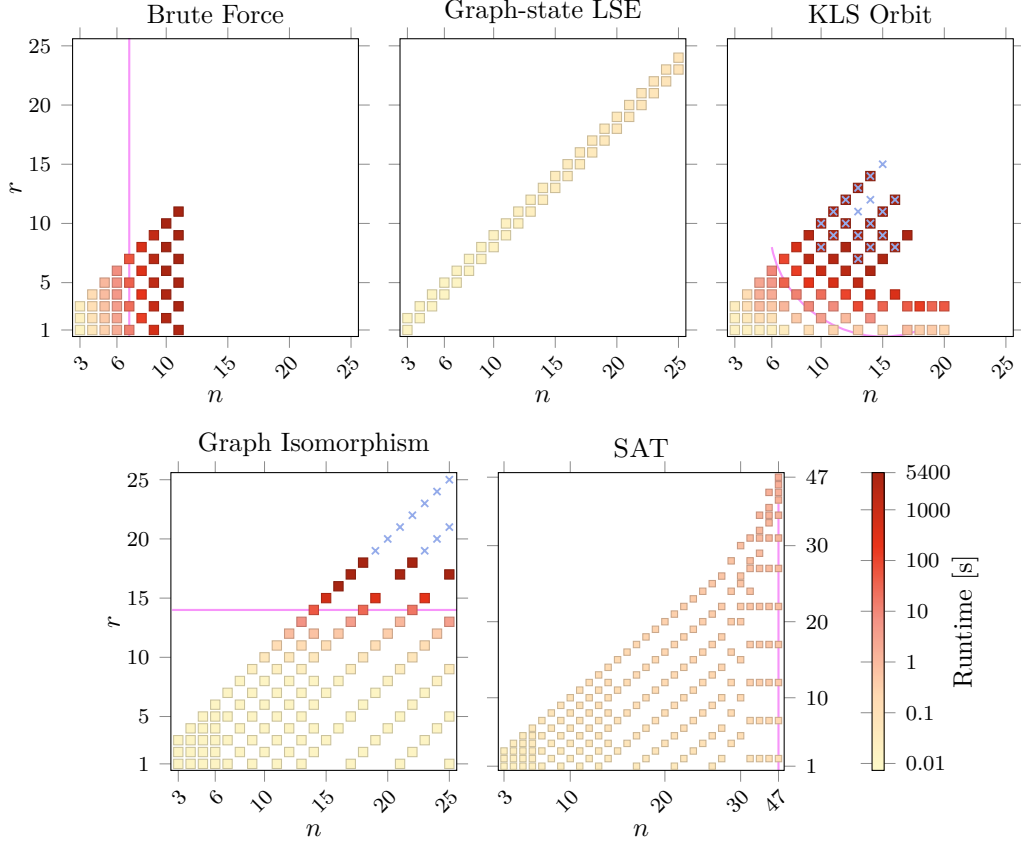


Figure 7.5: *Runtimes of the prototypes for LC-EQ STB on randomized stabilizer-code instances.*

best performance, achieving the lowest runtime across all evaluated parameter settings and exhibiting no significant outliers.

LC-EQ CSS The performance heatmaps of the approaches for LC-EQ CSS are depicted in Figure 7.6. The runtime of brute-force search depends strongly on n , whereas that of the SAT-based approach is primarily determined by r . The graph-state-based approaches exhibit less distinct scaling behavior. In particular, both complete orbit-traversal variants perform poorly, especially on negative instances, as shown in Figure A.5d in the Appendix. Only the graph-state-based approaches utilizing orbit traversal—including the restricted simple LC orbit—exhibit memory errors. By contrast, the SAT-based approach stays feasible across a broad parameter range, although it exhibits occasional high-runtime outliers, particularly on positive instances.

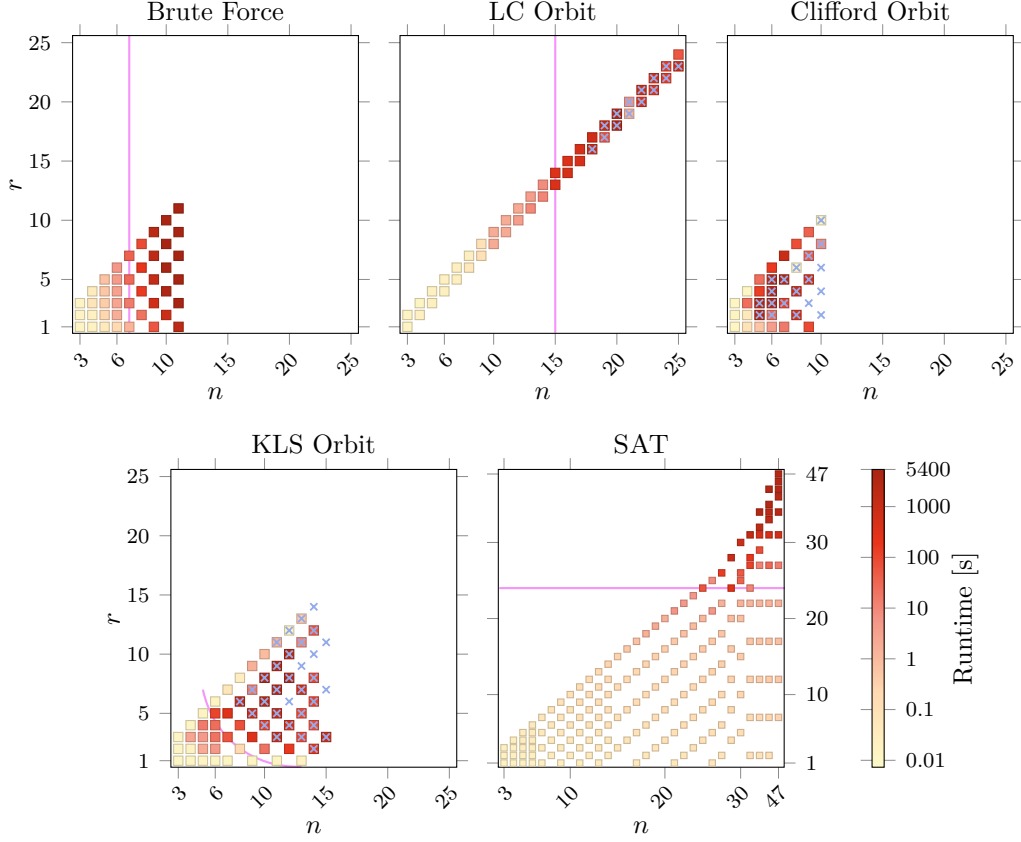


Figure 7.6: Runtimes of the prototypes for LC-EQ CSS on randomized stabilizer-code instances.

7.4.4 Performance on Small Codes

For sufficiently small codes, brute-force methods may outperform more sophisticated approaches because they incur little computational overhead and have modest memory requirements. We therefore determine the parameter ranges in which brute-force methods remain preferable to the most promising prototype algorithms identified in the preceding section.

For P-EQ STB, Figure 7.7a shows that the brute-force approach outperforms both the SAT-based approach and the selected invariant for code dimensions up to $n = 5$. In the CSS version of the problem, Figure 7.7b indicates that the brute-force approach is also preferable for up to $n = 5$. Figure 7.7c reveals that the brute-force approach for LC-EQ STB does not consistently outperform the corresponding SAT-based approach, even for small codes. At the same time, the selected invariant consistently has the lowest runtime among the three evaluated methods. For LC-EQ CSS, Figure 7.7d shows that brute force is preferable for codes with at most 3 physical qubits. However, due to an insufficient number of negative instances, this threshold is based primarily on positive instances. Comparison-based LC invariants are not applicable because no fixed target code is given.

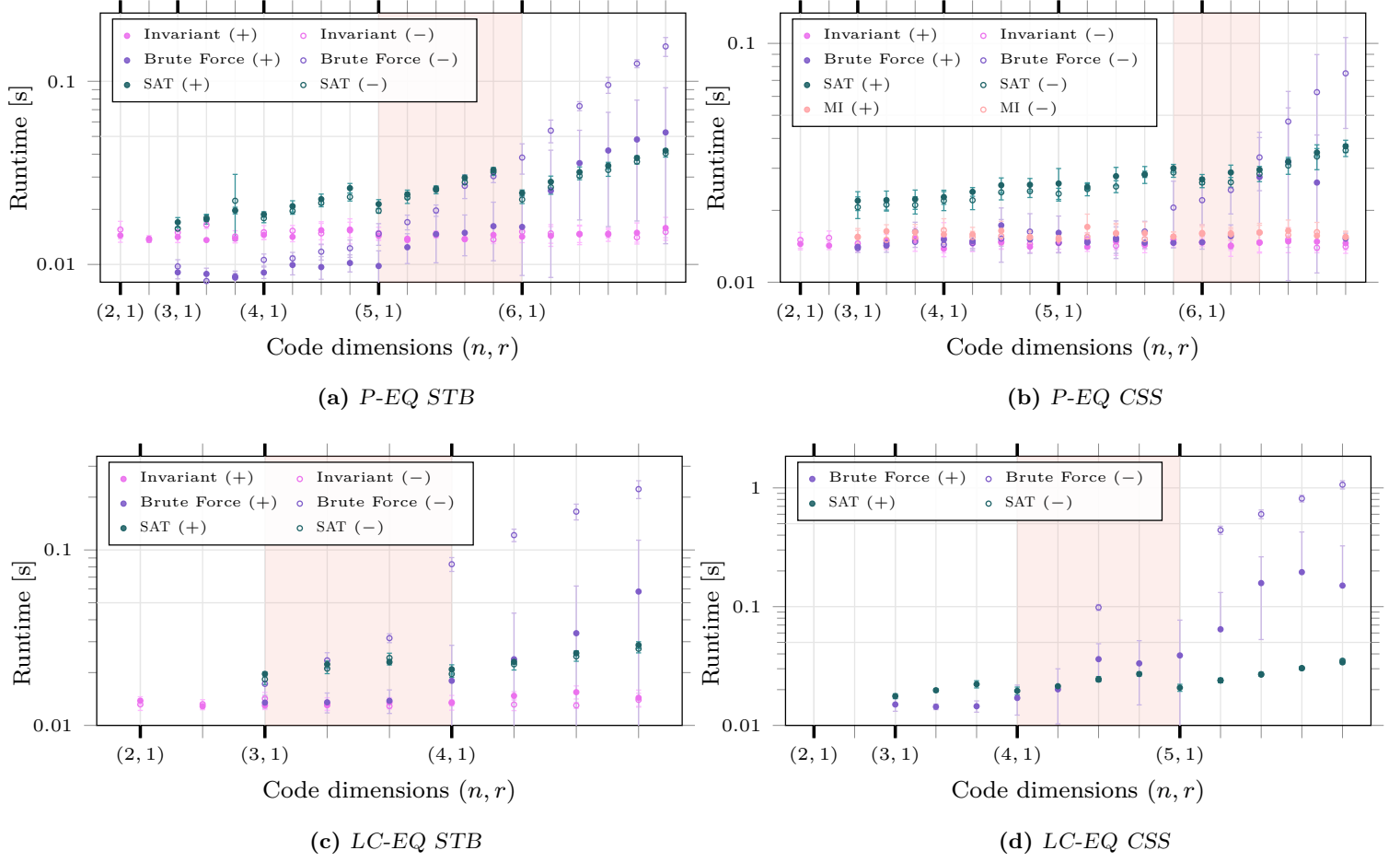


Figure 7.7: Runtime comparison between the selected permutation or LC invariant, the brute-force algorithm, and the respective alternative prototype algorithms. Each positive (+) and negative (-) data point shows the mean runtime over randomly generated instances, and error bars indicate one standard deviation. Instances are ordered along the x-axis lexicographically by their tableau dimensions (n, r) . The shaded red region marks the transition in relative runtime behavior between the brute-force and respective other prototype algorithms.

7.4.5 Performance on Large Codes

This section examines the performance of the preferred algorithmic approaches and certain invariants on larger codes with n up to 47, for a representative fixed number of logical qubits $k = 8$, so that both a scaling with n and with r can be observed. Additionally, selected algorithms are evaluated on the large, structured BB codes listed in Table 7.2.

P-EQ STB Figure 7.8a shows that the SAT-based approach solves most evaluated instances within one minute. Occasional high-runtime outliers nevertheless occur, as is visible in a negative instance. The linear-dependency invariant remains beneficial across the entire parameter range. By contrast, computing Sendrier’s signatures becomes more expensive than the SAT-based approach for codes with $n > 20$. All negative BB instances are solved within the timeout, with runtimes from 23 to 190 seconds, whereas all positive instances exceed it.

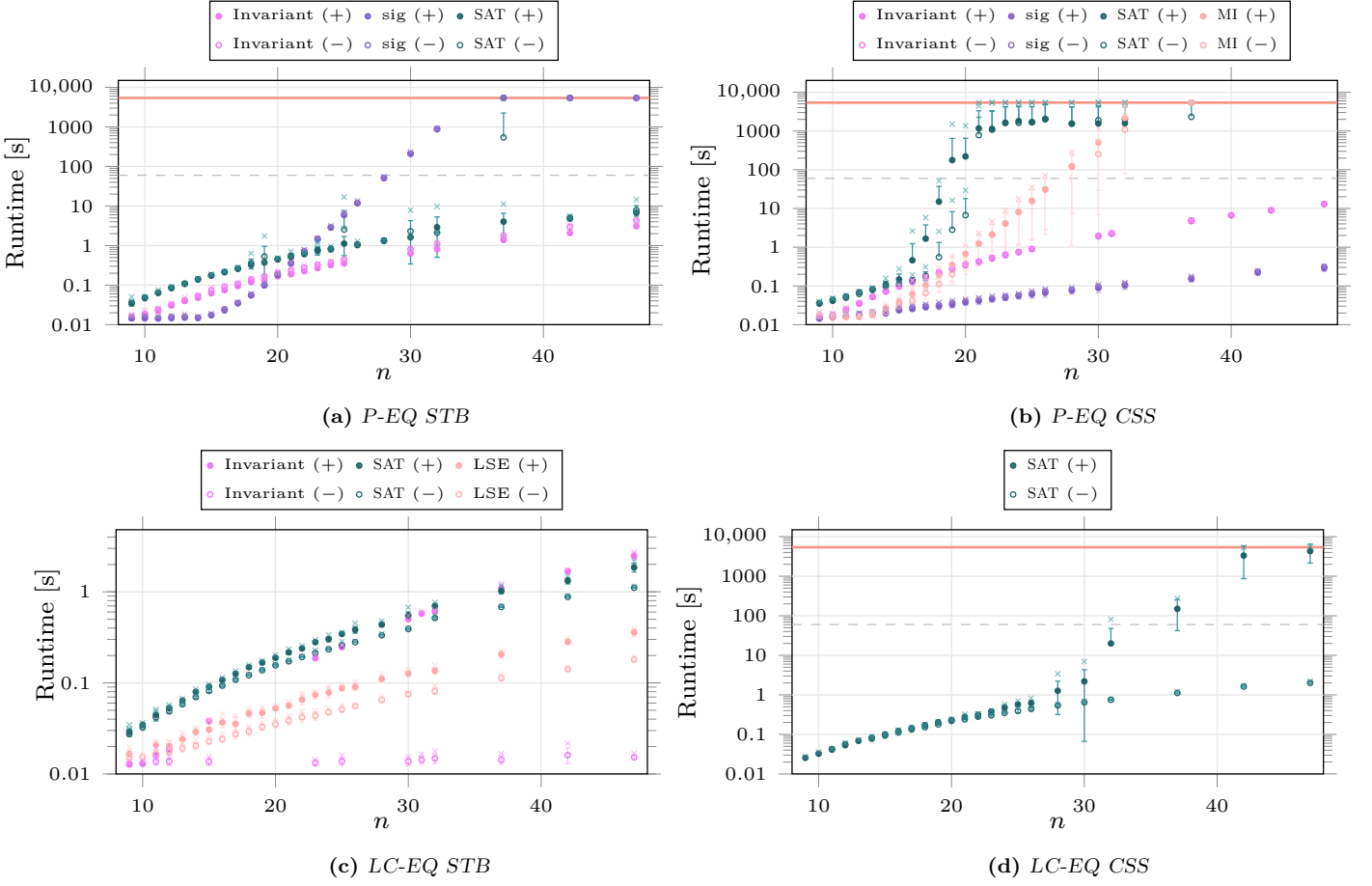


Figure 7.8: Runtime comparison between the selected permutation or LC invariant and the preferred prototype algorithms. The permutation problems also include the runtimes of computing Sendrier’s signatures (sig) for the corresponding instances. Each positive (+) and negative (−) data point shows the mean runtime over randomly generated instances; error bars indicate one standard deviation, and crosses mark the maximum runtime among them. Instances use $k = 8$, except for the graph-state LSE approach, which uses $k = 1$, with $9 \leq n \leq 47$. The red line indicates the benchmark timeout; the dashed line indicates one minute.

P-EQ CSS In the CSS setting, Figure 7.8b indicates that the runtime of the SAT-based approach increases sharply at approximately 19 physical qubits, beyond which some instances consistently exceed the timeout. By contrast, the matroid-based approach exhibits a substantially more gradual increase and handles instances with up to 29 physical qubits. Using the linear-dependency invariant can improve performance across most of the parameter range; the exception is $10 \leq n < 20$, where it provides no significant runtime advantage over the matroid-based approach. Unlike for P-EQ STB, however, computing Sendrier’s punctured-hull weight-enumerator signatures incurs the lowest runtime throughout the evaluated range. For BB codes, the SAT-based approach exceeds the timeout for every instance, irrespective of whether the instance is positive or negative, while the MI-based approach exceeds the memory budget in all cases.

LC-EQ STB Figure 7.8c reveals that, for codes with $n \leq 47$, the SAT-based approach consistently determines equivalence within 2 seconds and exhibits a small runtime variance for all evaluated parameters. Where applicable, however, the graph-state LSE approach remains slightly faster. From approximately $n = 30$ onward, the selected invariant no longer yields a substantial runtime improvement for positive instances. Similar to P-EQ STB, all negative BB-code instances are solved within the timeout, with runtimes between 2 and 11 seconds, whereas almost all positive instances cannot be solved within the time budget.

LC-EQ CSS Figure 7.8d shows a clear separation between positive and negative instances. The SAT-based approach solves all negative instances within approximately 2 seconds, with comparatively no substantial increase in runtime as n grows. In contrast, the runtime for positive instances increases markedly, and instances begin to exceed the timeout for $n > 40$. This behavior persists for the larger BB codes—all positive instances time out, whereas all negative instances are solved within 5 to 29 seconds.

8 Integration

This chapter translates the findings from Chapter 7 into the design of hybrid algorithms for the equivalence problems under consideration. Each hybrid combines invariant-based filtering and decision procedures based on their observed applicability and performance, following the strategy shown in Figure 8.1. The resulting designs are evaluated on the structured benchmark instances.

The hybrid algorithms are implemented within the MQT-QECC framework, providing a practical interface for automated code-equivalence checking. The implementations for P-EQ STB, P-EQ CSS, and LC-EQ STB also support witness extraction, which is a minor extension to the basic algorithm. This is particularly relevant when equivalence is expected, and only a corresponding transformation between the codes is required.

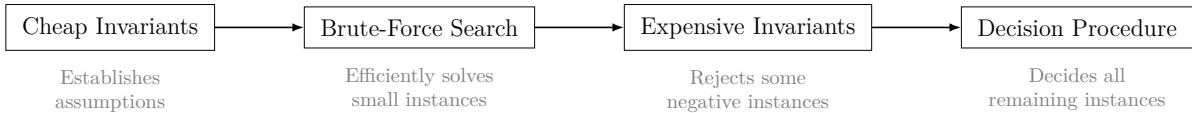


Figure 8.1: *Conceptual Hybrid Design*

8.1 Permutation Equivalence of Stabilizer Codes

Design To decide P-EQ STB, the hybrid algorithm applies constant- or linear-time invariants to reject inequivalent instances and to ensure that certain assumptions are satisfied for the subsequent algorithms. In particular, the codes must have the same number of physical and logical qubits and, consequently, tableaux of matching dimensions. Further inexpensive checks guarantee that their tableaux have the same number of zero columns and the same multiplicities of identical columns.

For input codes with at most 5 physical qubits, the hybrid performs a brute-force search. In this range, brute force is faster than the more sophisticated invariants and decision procedures.

For larger codes, the linear-dependency invariant is evaluated before the decision procedure, allowing inequivalent instances to be rejected early.

As observed in Section 7.4.3, the classical approach does not provide the best overall performance for this equivalence problem. Nevertheless, it exhibits distinctive behavior, particularly for negative instances: some large instances are rejected early by Sendrier’s Support Splitting Algorithm and therefore never enter the factorial permutation-tree search. This effect is especially pronounced for intermediate values of r . A possible explanation is that the hull signature depends on both the code and its dual, leading to less favorable behavior when the number of stabilizer generators is either very small or large. Thus, Sendrier’s signatures can quickly identify some negative instances, even for larger n , which is a behavior we can exploit. Even though the signature computation alone is not a sound equivalence decision procedure in itself,

we can use it as another invariant, followed by a more complex—sound and complete—algorithm for the remaining cases. Moreover, the signature computation induces a partition of the qubits of each code. This partition can be incorporated into the primary algorithm to restrict the search for a valid permutation.

This strategy synergizes with the SAT-based decision procedure, for which the evaluation revealed occasional high-runtime outliers among negative instances. Overall, the SAT-based approach solved the largest share of tested parameter settings within the time constraint and did not exceed the memory limit. It is therefore selected as the primary decision procedure for this problem. Its use of a standard solving framework also supports maintainability.

The hybrid applies the signature-based preprocessing only to codes with at most 20 physical qubits. Beyond this threshold, computing Sendrier’s signatures itself is already more expensive than running the SAT-based decision procedure and would therefore increase, rather than reduce, the overall runtime. Whenever the signatures are computed, their induced partition is incorporated into the SAT encoding, so that it permits only permutations that respect this partition. Accordingly, the permutation variables are restricted to

$$p_{ij} \quad \text{for } i \in \{0, \dots, n-1\}, \quad j \in \Pi_{S(C^*)}(w) \text{ with } i \in \Pi_{S(C)}(w).$$

Validation Figure 8.3 shows that the hybrid procedure for P-EQ STB performs well on the structured benchmark instances up to approximately $n = 31$ physical qubits, whereas runtimes are comparatively high for larger color-code and BB-code instances. This behavior is likely attributable to the structure of these instances, since the evaluated codes and their corresponding partners are CSS codes, so that the rank invariant does not immediately reject them. Since this implementation of Sendrier’s signatures incurs substantial overhead for stabilizer codes of this size, these signatures are omitted, as described above. Consequently, the SAT-based primary decision procedure cannot benefit from the resulting column partitions. As will be discussed in more detail in the next section, this SAT encoding appears to struggle with tableau representations of CSS codes, so that the hybrid procedure likely inherits this limitation. This may explain why, with few exceptions, both positive and negative cases among the larger evaluated CSS instances exceed the time limit. Unfortunately, this interpretation cannot be assessed on large structured non-CSS-code instances, as no such codes are available in our benchmark set.

8.2 Permutation Equivalence of CSS Codes

Design For deciding P-EQ CSS, we employ a preprocessing strategy analogous to that used for P-EQ STB. First, the same inexpensive invariants are applied to reject inequivalent instances and verify the required assumptions. For codes with at most 5 physical qubits, the hybrid performs a brute-force search, which outperforms the more sophisticated invariants and decision procedures in this range. For larger codes, except those with $10 < n < 20$, the linear-dependency invariant enables early rejection without increasing the runtime relative to the primary approach.

As for P-EQ STB, Sendrier’s signatures provide an additional invariant for P-EQ CSS, and their induced column partition can constrain the search for a valid permutation. In the CSS

setting, however, computing these signatures remains less expensive than the primary decision procedures throughout the evaluated parameter range. The improvement over P-EQ STB signature computation likely reflects an implementation artifact: the binary CSS case uses highly optimized libraries, whereas $\mathbb{F}_4$ arithmetic is implemented primarily in Python. Hence, the P-EQ CSS hybrid can apply this additional preprocessing to all instances.

However, selecting the primary decision procedure is less straightforward in the CSS case. Our evaluation shows that the SAT-based approach performs substantially worse on CSS codes than on general stabilizer codes. The additional experiment presented in Appendix A indicates that this disparity arises from the structure of the CSS instances rather than solely from the chosen Boolean encoding.

Although the SAT instances for P-EQ CSS contain fewer variables and constraints than those for P-EQ STB, they require longer solution times in practice. One possible explanation is that the coupled X and Z components of each qubit in a non-CSS stabilizer tableau allow the solver to eliminate candidate permutations more effectively than the separated H_X and H_Z components of a CSS code. Using the general stabilizer encoding for CSS codes does not eliminate this difference, since the underlying coupling remains absent. This observation additionally explains the behavior of the SAT-encoding in the previous section. Consequently, incorporating the partition information alone does not make the SAT-based approach a suitable primary procedure for P-EQ CSS.

Instead, the matroid-based approach performs well on small- to medium-sized CSS codes, although its memory requirements limit its applicability for larger instances, and it entails greater maintenance effort due to an external GI tool. The partition information induced by Sendrier’s signature computation can be incorporated into this approach through graph coloring, as illustrated by the following example:

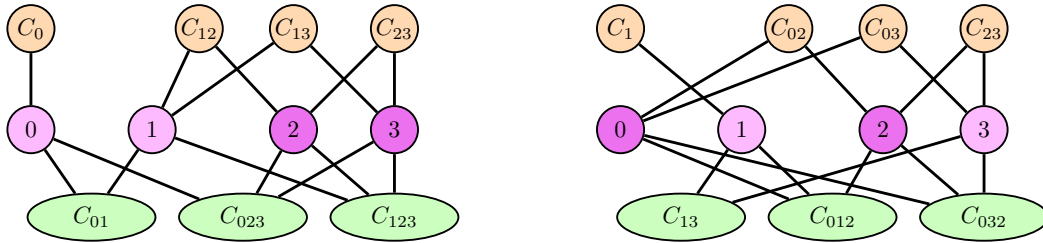


Figure 8.2: Matroid-invariant graphs for C_1 (left) and C_2 (right) from the P-EQ CSS example.

Figure 8.2 augments the matroid representation from Figure 5.1 with the single invariant-based qubit coloring introduced in Figure 5.2.

The resulting hybrid, therefore, combines both primary decision procedures. It applies the matroid-based approach to all codes with $5 < n \leq 17$ and to codes with $17 < n < 30$ and $r \geq 9$. For all remaining codes, it uses the SAT-based approach, which avoids the memory limitations of the matroid-based approach for larger codes and is preferable for medium-sized codes with small r .

Validation Figure 8.3 indicates that, in contrast to the results on randomized instances, the hybrid procedure for P-EQ CSS performs better on the structured benchmark instances than

its P-EQ STB counterpart. This difference is particularly pronounced for codes with more than 30 physical qubits. A likely explanation lies in the fact that the P-EQ CSS hybrid can leverage Sendrier’s signatures for all evaluated instances, since their computational cost is lower in the CSS setting. These signatures depend on the code itself through its hull, rather than solely on its tableau representation. Thus, for CSS codes, they may eliminate more candidate permutations than the SAT-internal checks, whose effectiveness appears to be limited by the uncoupled structure of CSS matrices. Consequently, the P-EQ CSS hybrid can often quickly reject inequivalent instances, even for larger codes, while providing the primary decision procedure with partition information that reduces its search space. By contrast, the P-EQ STB hybrid cannot leverage Sendrier’s signatures for larger codes and still inherits the limitations of the SAT-based approach on CSS inputs. Accordingly, the evaluated negative P-EQ CSS instances can typically be decided within seconds. If the signatures do not rule out equivalence, however, the SAT-based procedure may still exceed the time limit on negative instances, as observed for the BB code with $n = 144$. Moreover, equivalent instances with $n \geq 72$ consistently reach the time limit.

8.3 Local Clifford Equivalence of Stabilizer Codes

Design For LC-EQ STB, the hybrid first verifies that the two codes have the same number of physical and logical qubits. For codes with at most one logical qubit, the hybrid applies the graph-state LSE approach, which is sound and complete within this parameter range and achieved favorable runtime performance in the evaluation.

Since brute-force search did not provide a runtime advantage for any of the evaluated instances, it is omitted from the hybrid. For codes with at most 30 physical qubits, the degree-2 local invariant with a maximum subset size of 2 columns is evaluated before invoking the general decision procedure.

The SAT-based approach is again selected as the primary decision procedure because it was fastest at every measured setting outside the LSE method’s restricted domain, remained consistently very fast across a wide parameter range, and produced no OOM runs in the evaluation.

Validation This hybrid procedure performs well on the structured LC-EQ STB instances, as shown in Figure 8.3. Even for large instances, such as the evaluated BB codes, it establishes inequivalence within 2 to 12 seconds. For large positive instances, the hybrid is even able to yield a concrete answer within the timeout limit. This outcome is not guaranteed, however, as for every benchmarked BB code with $n > 72$, at least one equivalent instance exceeds the timeout limit.

8.4 Local Clifford Equivalence of Stabilizer Codes to CSS Codes

Design To decide whether a given stabilizer code is LC-equivalent to some CSS code, the hybrid cannot apply any proposed prefilters, since all considered invariants require a pair of codes. For codes with at most 3 physical qubits, it uses a brute-force search, which outperforms the more complex procedures within this range. For larger codes, it applies the SAT-based

prototype with the projector criterion. This approach remained computationally feasible across the largest evaluated parameter range and avoided the memory errors encountered by the graph-based approaches. Its runtime nevertheless increases as r grows, particularly for positive instances. A limitation of the current projector encoding is that it only determines whether a suitable transformation exists and cannot extract a corresponding witness.

Validation Figure 8.3 indicates that, on structured instances, the proposed LC-EQ CSS procedure exhibits runtime behavior comparable to, though slightly worse than, that of the LC-EQ STB hybrid. For large instances, inequivalence is established within 5 to 28 seconds, while equivalent instances with up to $n = 90$ physical qubits can still be resolved within the timeout limit. However, this performance is not consistent across all inputs, as for each benchmarked BB code with $n > 72$, at least one equivalent instance reaches the computational time limit.

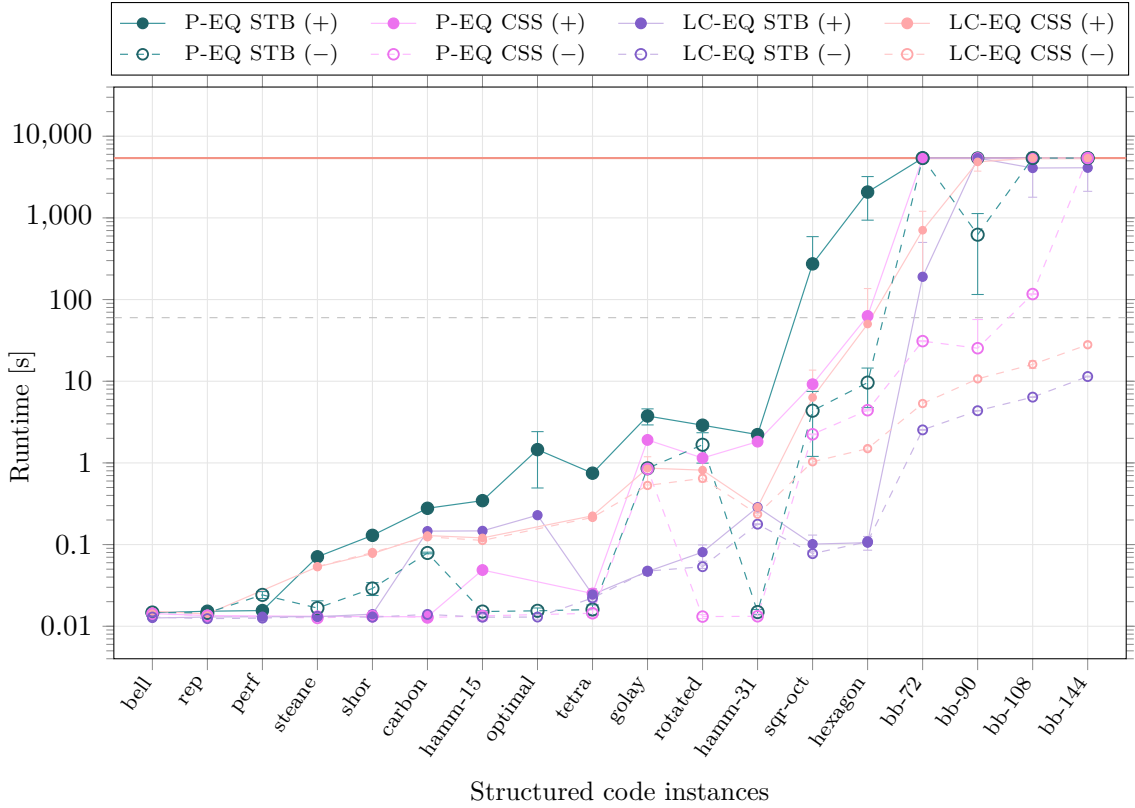


Figure 8.3: Runtime comparison between the hybrid algorithms, one slot per code sorted by their dimensions (n, r) . Each positive (+) and negative (−) data point shows the mean runtime over randomly generated instances based on the structured codes, and error bars indicate one standard deviation. Connecting lines serve only as visual guides and do not imply interpolation between data points. The red line indicates the benchmark timeout; the dashed grey line indicates one minute.

9 Discussion

9.1 Answers to the Research Questions

Permutation Equivalence of Stabilizer Codes Utilizing invariants, signature computation, and a SAT-based decision procedure, permutation equivalence can be decided practically for the evaluated general stabilizer-code instances with up to approximately 50 physical qubits. Most instances are resolved within one minute, although occasional high-runtime outliers suggest that performance depends on additional structural properties of the input representation. However, the distinct behavior of CSS-code inputs requires a separate choice of decision procedure, which is addressed below.

Permutation Equivalence of CSS Codes Employing invariants, classical signature-based preprocessing, and SAT- and MI-based decision procedures, we can reliably decide permutation equivalence in practice for the evaluated CSS codes with up to $n = 30$ physical qubits. For larger codes with more than approximately 9 stabilizer generators, the primary decision procedures become infeasible for both positive and negative instances. Deciding negative instances, therefore, relies primarily on detecting signature mismatches, which can rapidly establish inequivalence but are not guaranteed to occur. In our experiments, this approach resolves some negative instances with up to approximately $n = 144$ physical qubits, whereas positive instances of this size are generally impossible to decide within the computational budget.

Local Clifford Equivalence of Stabilizer Codes By combining invariants, a SAT-based decision procedure, and graph-state methods, LC equivalence can be decided reliably in practice for the evaluated stabilizer codes with up to $n = 72$ physical qubits. For the evaluated codes with $72 < n \leq 144$, inequivalence can generally be established within a few seconds. Although the proposed procedure can also establish equivalence in some instances within this range, it is not guaranteed to do so.

Local Clifford Equivalence of Stabilizer Codes to CSS Codes Using brute-force search and a SAT-based decision procedure, LC-equivalence to an arbitrary CSS code can be determined consistently for the evaluated stabilizer codes with up to $n = 72$ physical qubits. Beyond this range, the approach remains effective at establishing inequivalence, whereas its ability to confirm equivalence depends on the particular instance.

9.2 Broader Implications

Several findings extend beyond the thesis’s individual equivalence problems.

First, the additional structure of CSS codes does not necessarily translate into lower computational complexity in practice. Although the separation into X - and Z -type components simplifies many problems in QEC, the experiments revealed that permutation-equivalence instances for this code class can be particularly challenging for a SAT solver. Additional experiments indicate that this behavior arises from inherent properties of the CSS structure, illustrating that a more structured representation does not necessarily make the computational problem easier.

Second, the results demonstrate the value of adapting techniques from classical coding theory to quantum-code equivalence. Established methods from classical coding theory, such as signature-based preprocessing, can drastically enhance the performance of quantum-specific algorithms when appropriately adapted. Similarly, the correspondence between binary linear codes and vector matroids reformulates matroid isomorphism into a practically useful CSS-specific procedure. These results highlight the benefits of interaction between classical and quantum coding theory, while also emphasizing the need to account carefully for the distinct characteristics of quantum codes when translating classical techniques.

In a different vein, the investigation showed that deciding LC equivalence of stabilizer codes is more challenging than initially anticipated. In general, results established for stabilizer states cannot be transferred straightforwardly to stabilizer codes. The limitations of the underlying transformations due to the freedom in the choice of logical operators must be carefully accounted for when designing algorithms for LC-equivalence checking of codes.

The evaluation shows that no single algorithmic paradigm is uniformly preferable across equivalence notions, code classes, and parameter ranges. The relative performance of the approaches depends on parameters such as the number of physical qubits, the number of stabilizer generators, and whether the instances are equivalent. In particular, several invariants efficiently reject large negative instances while offering little benefit to positive instances, whereas graph-based approaches may achieve favorable runtimes only within parameter ranges where memory requirements remain manageable. These complementary feasibility characteristics motivate the design of procedures integrating different algorithms.

Finally, the results suggest a practical, albeit incomplete, approach to equivalence checking. A lightweight tool could run the corresponding hybrid procedure with a problem-specific time budget. Any result obtained within this period would be conclusive. Otherwise, the instance would remain formally unresolved, although failure to terminate may serve as a tentative indication of equivalence, based on the observed runtime behavior.

9.3 Methodological Limitations

While the experimental evaluation provides valuable insights, its results should be interpreted in light of several assumptions and limitations.

Because the prototype approaches were evaluated exclusively on synthetic randomized instances, the observed performance characteristics may not generalize to all relevant code families. Although parameterized random instances are useful for identifying general trends, they may not reflect the properties of practically relevant codes.

Furthermore, each parameter setting was evaluated using only 5 to 10 random seeds. While this is sufficient to identify broad trends, it does not permit a reliable characterization of

heavy-tailed runtime distributions. Consequently, the results may not generalize to larger sets of instances. This limitation is particularly pronounced for LC-EQ CSS, where difficulties in generating suitable negative instances led to an evaluation predominantly based on positive instances.

On a related note, the randomized tableau-generation process does not control parameters such as density or symmetry, which may affect the algorithms' runtime. Consequently, the observed performance characteristics may be influenced by hidden variables.

Moreover, the evaluation was conducted on a single hardware and software configuration and depends on the specific implementation quality of the prototype algorithms. The results may therefore differ in other environments or partly reflect implementation maturity rather than the underlying approaches.

Finally, instances exceeding the available memory or runtime limits were not examined further, thereby limiting the conclusions that can be drawn about the algorithm's performance within the corresponding parameter ranges.

10 Conclusion

10.1 Summary of Contributions

This thesis investigated automated equivalence checking of stabilizer codes under qubit permutations and local Clifford operations. Four equivalence problems for general stabilizer and CSS codes were formulated uniformly in terms of binary symplectic tableaus and parity-check matrices. Based on these formulations, the thesis compiled, adapted, and developed a range of algorithmic approaches to solve the problems. These include brute-force procedures, SAT-based encodings, techniques from classical code equivalence, reductions to matroid and graph isomorphism, and graph-state-based methods. Invariants under the respective transformations were additionally examined.

The investigation of graph-state-based approaches identified a crucial limitation of reductions from stabilizer codes to stabilizer states. In particular, the logical freedom introduced by the choice of encoding can require entangling Clifford operations on the reference subsystem. Consequently, algorithms inherited from the stabilizer-state setting are generally not complete for codes with multiple logical qubits. The thesis characterized the conditions under which these reductions remain sound and complete and examined alternative ways of incorporating the logical freedom.

The proposed approaches were implemented as modular prototypes together with infrastructure for generating, validating, and benchmarking randomized and structured codes as problem instances. The experimental results support the use of problem-specific combinations of algorithms and procedure components. The SAT-based procedures exhibited favorable performance for permutation and local Clifford equivalence of general stabilizer codes, whereas the matroid-based procedure remained competitive for permutation equivalence of CSS codes.

Based on these findings, hybrid procedures combining inexpensive checks, stronger invariants, and parameter-dependent decision algorithms were designed and integrated into MQT-QECC. The resulting implementations provide a practical interface for automated equivalence checking and for extracting explicit equivalence transformations.

The experimental evaluation characterizes the practical feasibility of the proposed hybrid procedures. Although complete decisions become infeasible beyond problem-dependent code sizes, signature computations and SAT-based checks can still efficiently establish non-equivalence for many larger instances, without providing a general runtime guarantee.

10.2 Future Work

As this thesis combines theoretical research with practical algorithm engineering, the results suggest several directions for future work.

Theoretical directions

- Extend the considered equivalence problems to retain the *phase information* omitted by the binary symplectic representation. This would require an adaptation of the input representation and a corresponding reformulation of the equivalence conditions. Establishing soundness and completeness of the resulting algorithms would clarify the extent to which the reductions developed in this thesis remain applicable when phases are represented explicitly.
- Develop specialized equivalence notions for *structured code families*. For geometrically local codes, such as color codes, arbitrary qubit permutations may not represent physically meaningful transformations. Restricting the admissible permutations to automorphisms of the underlying structure would provide a more application-oriented notion of equivalence and could reduce the search space.
- Investigate the *joint equivalence relation* induced by qubit permutations and local Clifford operations. A dedicated decision procedure could combine permutation constraints and local symplectic constraints within a single formulation. It would further be of interest to determine whether their joint structure permits stronger invariants or more efficient procedures than the sequential application of the existing algorithms.
- Characterize the *influence of code properties* beyond the dimensions of the input tableaus on computational difficulty. Candidate parameters include tableau density or symmetry. A systematic parameterized analysis could determine whether these quantities predict runtime or memory consumption and could consequently guide the automatic selection of algorithms.
- Address the *logical freedom* introduced by the code-to-state transformations. Future work could investigate equivalence-preserving canonical representatives, the use of transversal gates, or equivalence procedures treating the reference subsystem as a gauge, thereby enabling complete graph-based procedures for codes with multiple logical qubits.

Practical directions

- Examine the suitability of individual algorithmic components for *parallel execution*. Independent branches of canonicalization, orbit searches, or invariant computations could be distributed across multiple processor cores. Further evaluation is required to determine which components provide sufficient computational granularity and independence to offset the overhead of synchronization and communication.
- Improve the *algorithm implementations* through more memory-efficient data structures and hardware-aware computations to extend the range of instances that can be solved within the resource limits.
- Evaluate the performance of the proposed algorithms on a *wider range of hardware and software configurations*, to assess their robustness and scalability across different computing environments. Recording additional measures, including peak memory consumption, would provide a more complete characterization of practical feasibility.

Taken together, these directions provide a basis for developing equivalence-checking methods that are more general in their theoretical scope and scalable in practice.

A Appendix

Logical Freedom

As an example for the change of a logical framework, we analyze how an S operation on the i -th logical qubit affects the logical framework of a code. In the Choi tableau, this operation corresponds to an S operation on the $(n + i)$ -th qubit:

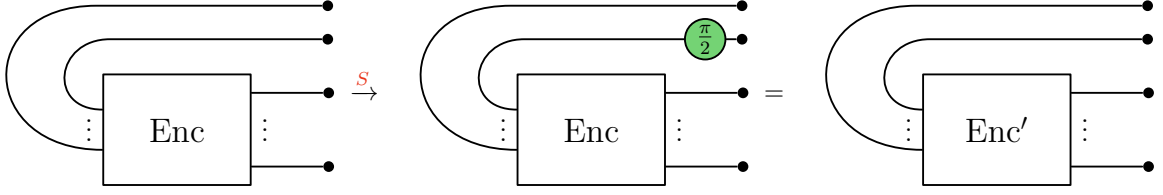
$$\begin{array}{c} S(C) \\ L_x(C) \\ L_z(C) \end{array} \xrightarrow[\text{Tableau}]{\text{Choi}} \left[\begin{array}{c|c} \begin{array}{c} S(C)^{(x)} \\ \phi(\bar{X}_1)^{(x)} \\ \vdots \\ \phi(\bar{X}_i)^{(x)} \\ \vdots \\ \phi(\bar{X}_k)^{(x)} \\ \phi(\bar{Z}_1)^{(x)} \\ \vdots \\ \phi(\bar{Z}_i)^{(x)} \\ \vdots \\ \phi(\bar{Z}_k)^{(x)} \end{array} & \begin{array}{c} 0 \dots 0 \dots 0 \\ 1 \dots 0 \dots 0 \\ \ddots \\ 0 \dots 1 \dots 0 \\ \ddots \\ 0 \dots 0 \dots 1 \\ 0 \dots 0 \dots 0 \\ \ddots \\ 0 \dots 0 \dots 0 \\ \ddots \\ 0 \dots 0 \dots 0 \\ 0 \dots 0 \dots 0 \end{array} \end{array} \right] \xrightarrow{S} \left[\begin{array}{c|c} \begin{array}{c} S(C)^{(x)} \\ \phi(\bar{X}_1)^{(x)} \\ \vdots \\ \phi(\bar{X}_i)^{(x)} \\ \vdots \\ \phi(\bar{X}_k)^{(x)} \\ \phi(\bar{Z}_1)^{(x)} \\ \vdots \\ \phi(\bar{Z}_i)^{(x)} \\ \vdots \\ \phi(\bar{Z}_k)^{(x)} \end{array} & \begin{array}{c} 0 \dots 0 \dots 0 \\ 1 \dots 0 \dots 0 \\ \ddots \\ 0 \dots 1 \dots 0 \\ \ddots \\ 0 \dots 0 \dots 1 \\ 0 \dots 0 \dots 0 \\ \ddots \\ 0 \dots 0 \dots 0 \\ \ddots \\ 0 \dots 0 \dots 0 \\ 0 \dots 0 \dots 0 \end{array} \end{array} \right]$$

This transformed tableau is now restored to the standard Choi form by using a row operation to cancel out the top-right red 1.

$$\left[\begin{array}{c|c} \begin{array}{c} S(C)^{(x)} \\ \phi(\bar{X}_1)^{(x)} \\ \vdots \\ \phi(\bar{X}_i)^{(x)} + \phi(\bar{Z}_i)^{(x)} \\ \vdots \\ \phi(\bar{X}_k)^{(x)} \\ \phi(\bar{Z}_1)^{(x)} \\ \vdots \\ \phi(\bar{Z}_i)^{(x)} \\ \vdots \\ \phi(\bar{Z}_k)^{(x)} \end{array} & \begin{array}{c} 0 \dots 0 \dots 0 \\ 1 \dots 0 \dots 0 \\ \ddots \\ 0 \dots 1 \dots 0 \\ \ddots \\ 0 \dots 0 \dots 1 \\ 0 \dots 0 \dots 0 \\ \ddots \\ 0 \dots 0 \dots 0 \\ \ddots \\ 0 \dots 0 \dots 0 \\ 0 \dots 0 \dots 0 \end{array} \end{array} \right] = \left[\begin{array}{c|c} \begin{array}{c} S(C)^{(x)} \\ \phi(\bar{X}_1)^{(x)} \\ \vdots \\ \phi(\bar{X}_i)^{(x)} + \phi(\bar{Z}_i)^{(x)} \\ \vdots \\ \phi(\bar{X}_k)^{(x)} \\ \phi(\bar{Z}_1)^{(x)} \\ \vdots \\ \phi(\bar{Z}_i)^{(x)} \\ \vdots \\ \phi(\bar{Z}_k)^{(x)} \end{array} & \begin{array}{c} 0 \dots 0 \dots 0 \\ 1 \dots 0 \dots 0 \\ \ddots \\ 0 \dots 0 \dots 0 \\ \ddots \\ 0 \dots 0 \dots 0 \\ 1 \dots 0 \dots 0 \\ \ddots \\ 0 \dots 1 \dots 0 \\ \ddots \\ 0 \dots 0 \dots 1 \end{array} \end{array} \right] = \left[\begin{array}{c|c} S(C)^{(x)} & 0 \\ L'_x(C)^{(x)} & I \\ L'_z(C)^{(x)} & 0 \end{array} \right]$$

The matrices $L'_x(C)$ and $L'_z(C)$ represent the new valid logical framework, which has been transformed by a logical S . The stabilizer generators remain unchanged, indicating that the code space of the original code C is preserved. However, the states themselves differ, exhibiting distinct properties, highlighting the impact of logical operations on the logical framework without altering the underlying stabilizers.

In the ZX-calculus representation, this logical S operation is depicted as follows:



The Z-spider is absorbed into the encoder diagram, resulting in a modified encoder state. This transformation illustrates how logical operations—especially when entangling—can alter the entire object under consideration, obscuring the fact that the underlying stabilizers remain unchanged. This highlights the importance of considering logical freedom in equivalence checking processes.

Counterexamples

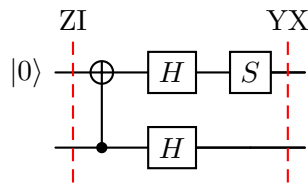
This section presents a collection of counterexamples or corrected interpretations of claims in the literature, which would have significantly simplified some of the discussed approaches. However, their stated formulations require additional assumptions or depend on differing interpretations.

Bipartiteness of KLS Normal Forms

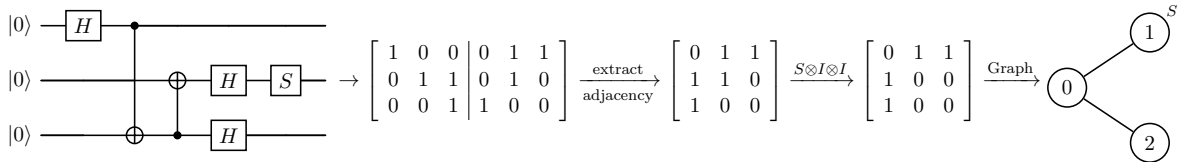
Claim [31] (Fact 1): The KLS normal form of a code C is bipartite if and only if C is LC-equivalent to a CSS code.

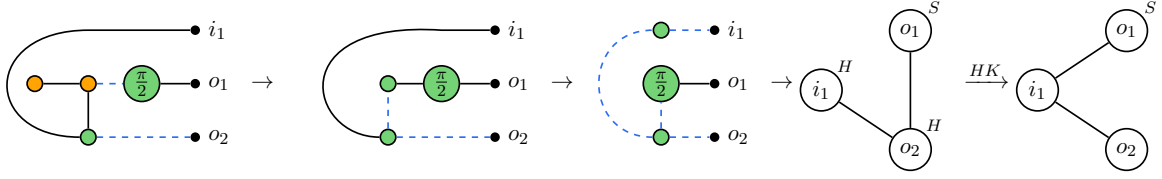
Corrected statement: The KLS normal form of C is bipartite if and only if C itself is a CSS code.

Consider the code C with stabilizer group $\mathcal{S} = \langle YX \rangle$. The local Clifford operation S_1 maps it to the CSS stabilizer group $\mathcal{S}_{\text{CSS}} = \langle XX \rangle$, thus witnessing LC-equivalence.

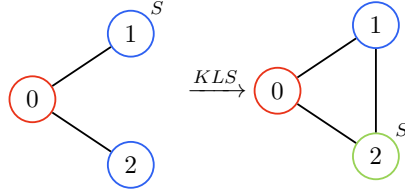


Starting from the encoder circuit for C above, both the ZX-calculus construction and the circuit-tableau construction yield the following graph in HK normal form:





Although this graph is bipartite, its conversion to KLS normal form removes the local Clifford operator on the pivot by introducing an edge between the two blue vertices. The resulting graph is not bipartite:



Disconnected Graphs and Local Complementation

Claim [47] (Lemma 1): Let B be a basis of the solution space $\mathcal{M}$ of the LSE described in Section 5.7.4. If $\dim(\mathcal{M}) > 4$, then $\mathcal{M}$ contains a vector satisfying the invertibility constraints if and only if $\mathcal{M}' = \{b + b' \mid b, b' \in B\}$ contains such a vector.

Correction: This result applies to connected graphs. For disconnected graphs, it must be applied separately to each connected component, since the proof in [7] assumes connectedness.

Consider the stabilizer groups $\mathcal{S}_1 = \langle \text{ZI}, \text{IZ} \rangle$ and $\mathcal{S}_2 = \langle \text{XI}, \text{IX} \rangle$. Conjugation by $H \otimes H$ maps $\mathcal{S}_1$ to $\mathcal{S}_2$, establishing their LC equivalence:

$$S(C_1) = \left[\begin{array}{cc|cc} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array} \right] \xrightarrow[U=H \otimes H]{\text{make } X \text{ invertible}} \left[\begin{array}{cc|cc} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{array} \right] \rightarrow \Gamma_1 = \left[\begin{array}{cc} 0 & 0 \\ 0 & 0 \end{array} \right] \rightarrow \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$S(C_2) = \left[\begin{array}{cc|cc} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{array} \right] \xrightarrow[U=I \otimes I]{\text{make } X \text{ invertible}} \left[\begin{array}{cc|cc} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{array} \right] \rightarrow \Gamma_1 = \left[\begin{array}{cc} 0 & 0 \\ 0 & 0 \end{array} \right] \rightarrow \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

The resulting disconnected graphs are already identical. Their LC equivalence is therefore represented by the identity matrix Q , which yields the following assignment to the unknowns of the LSE:

$$Q = \left[\begin{array}{cc|cc} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array} \right] \xrightarrow{\text{expected solution}} \left. \begin{array}{l} a_0 = a_1 = d_0 = d_1 = 1 \\ b_0 = b_1 = c_0 = c_1 = 0 \end{array} \right\} \rightarrow [1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1]^\top$$

The LSE constructed from the adjacency matrices Γ_1 and Γ_2 has the following solution-space basis:

$$\left[\begin{array}{cc|cc|cc|cc} 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{array} \right] \cdot \begin{bmatrix} a_0 \\ a_1 \\ b_0 \\ b_1 \\ c_0 \\ c_1 \\ d_0 \\ d_1 \end{bmatrix} \stackrel{!}{=} 0 \xrightarrow{\text{solution}} \mathcal{M} = \left\langle \begin{bmatrix} b_1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} b_2 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} b_3 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} b_4 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} b_5 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} b_6 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \right\rangle$$

The following vectors in $\mathcal{M}$ satisfy the invertibility constraints:

$$\left. \begin{array}{l} a_0 \cdot d_0 + b_0 \cdot c_0 = 1 \\ a_1 \cdot d_1 + b_1 \cdot c_1 = 1 \end{array} \right\} \xrightarrow[\text{satisfying constraints}]{\text{solutions in } \mathcal{M}} \left\{ \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \right\}$$

Since $\dim(\mathcal{M}) = 6 > 4$, the claimed reduction would require $\mathcal{M}'$ to contain a vector satisfying the invertibility constraints. However, every such vector in $\mathcal{M}$ has a weight at least 4, whereas every sum of two basis vectors has weight at most 2. Hence, $\mathcal{M}'$ contains no valid candidate, providing a counterexample for disconnected graphs.

Qubit Permutations and KLS Normal Form

Possible interpretation [29]: Relabeling the output qubits of a KLS normal form yields the KLS normal form of the permuted code. Consequently, KLS normal forms could serve as canonical representatives under permutation equivalence.

Correction: Relabeling the output qubits does not generally preserve KLS normal form. The relabeled graph must first be recanonicalized to obtain the normal form of the permuted code. Therefore, isomorphism of independently computed KLS normal forms is not a valid criterion for permutation equivalence.

Consider the codes C_1 and C_2 with stabilizer groups $\mathcal{S}_1 = \langle \text{IXXI}, \text{XIXX} \rangle$ and $\mathcal{S}_2 = \langle \text{IIXX}, \text{XXIX} \rangle$, which are permutation equivalent:

$$S(C_1) = \left[\begin{array}{cccc|cccc} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \end{array} \right] \xrightarrow{\pi=(0231)} \left[\begin{array}{cccc|cccc} 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \end{array} \right] = S(C_2)$$

Nevertheless, the corresponding KLS normal-form graphs are not isomorphic, as shown in Figure A.1.

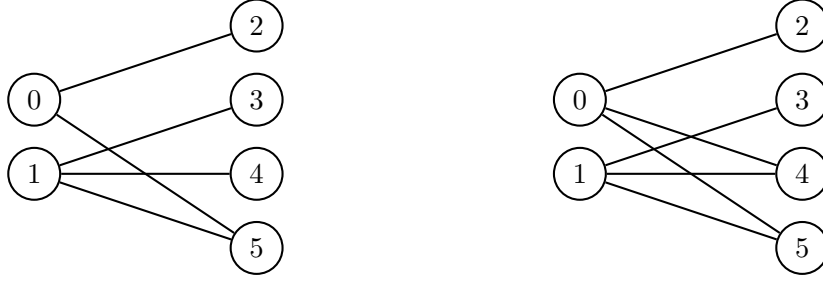


Figure A.1: KLS normal-form graphs of C_1 (left) and C_2 (right).

Logical Freedom and Local Clifford Equivalence

The following example illustrates the limitation discussed in Section 5.7.1. Since the Choi–Jamiołkowski transformation depends on the chosen logical framework, direct comparison of the resulting states need not preserve LC equivalence between codes.

Consider the codes C_1 and C_2 with stabilizer groups $\mathcal{S}_1 = \langle IXI \rangle$ and $\mathcal{S}_2 = \langle IYI \rangle$. The local Clifford operation S_2 maps $\mathcal{S}_1$ to $\mathcal{S}_2$, so the codes are LC-equivalent. For the logical frameworks specified below, the tableau-based Choi construction yields the following graph states:

$$\begin{aligned}
 \left. \begin{aligned}
 S(C_1) &= \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \\
 L_x(C_1) &= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \\
 L_z(C_1) &= \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}
 \end{aligned} \right\} \xrightarrow[\substack{D=I \otimes I \otimes I \otimes I}]{\substack{\text{Choi} \\ U=H \otimes I \otimes I \otimes I}} \Gamma_1 = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix} \rightarrow \begin{array}{c} \text{Graph 1} \end{array}
 \end{aligned}$$

Graph 1: A graph with nodes 0, 1, 2, 3, 4. Edges connect (3,0), (3,1), (4,2), and (4,3).

$$\begin{aligned}
 \left. \begin{aligned}
 S(C_2) &= \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix} \\
 L_x(C_2) &= \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \\
 L_z(C_2) &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}
 \end{aligned} \right\} \xrightarrow[\substack{D=I \otimes S \otimes I \otimes I}]{\substack{\text{Choi} \\ U=I \otimes I \otimes I \otimes I}} \Gamma_2 = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \rightarrow \begin{array}{c} \text{Graph 2} \end{array}
 \end{aligned}$$

Graph 2: A graph with nodes 0, 1, 2, 3, 4. Edges connect (3,0) and (4,2).

The two graphs are not related by any sequence of local complementations. Thus, the corresponding Choi states are not LC-equivalent, despite the LC equivalence of the underlying codes. This discrepancy results from the incompatible logical frameworks. Applying the non-local Clifford operations

$$\text{SWAP}_{3,4} = \text{CNOT}_{3 \rightarrow 4} \text{CNOT}_{4 \rightarrow 3} \text{CNOT}_{3 \rightarrow 4}$$

to the reference qubits aligns the logical frameworks, after which the resulting states are LC-equivalent. However, those operations are not included in a standard LC-equivalence test, e.g. using the LSE approach.

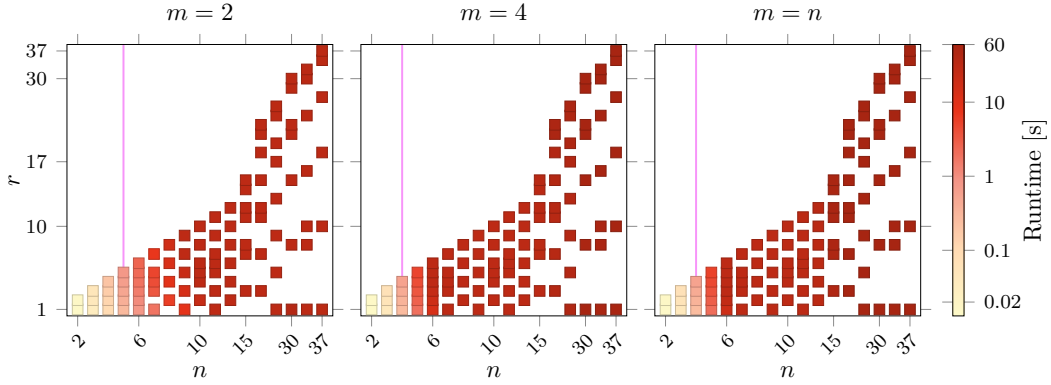


Figure A.2: Runtime comparison of the local degree-2 weight enumerator invariant for different maximum subset sizes m on randomized stabilizer-code instances.

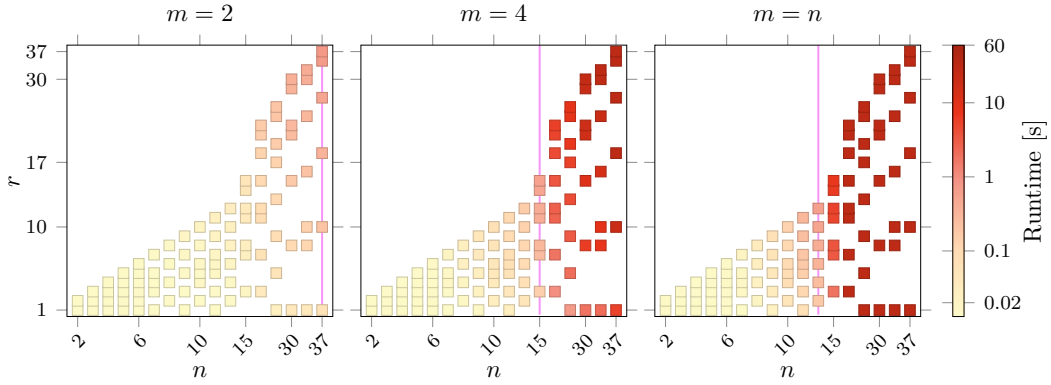


Figure A.3: Runtime comparison of the local degree-2 invariant introduced for different maximum subset sizes m on randomized stabilizer-code instances.

Supplementary Evaluation Results

This section presents supplementary benchmark results—using the same experimental setup and plotting conventions—that support the analysis in Chapter 7.

Detailed Invariant Benchmarks

Figures A.2 and A.3 provide detailed comparisons of the evaluated subset sizes for the LC invariants, which make clear that only the local invariant with a maximal subset size of 2 is computationally feasible for larger codes.

SAT Encoding for CSS Codes

Figure A.4 compares the general stabilizer encoding—encoding the whole stabilizer tableau—and the CSS-specific encoding—only considering the parity-check matrices—of identical CSS input instances, isolating the influence of the encoding from that of the code structure. It is evident that the CSS-specific encoding has no significant effect on the runtime performance of the SAT instance, suggesting that the observed performance differences are inherent to the

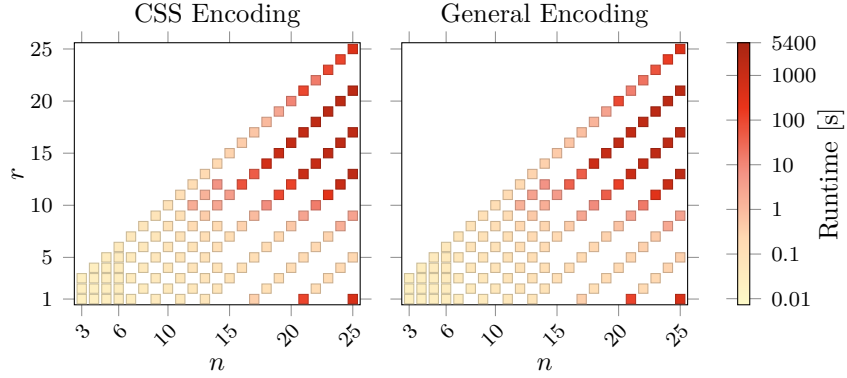
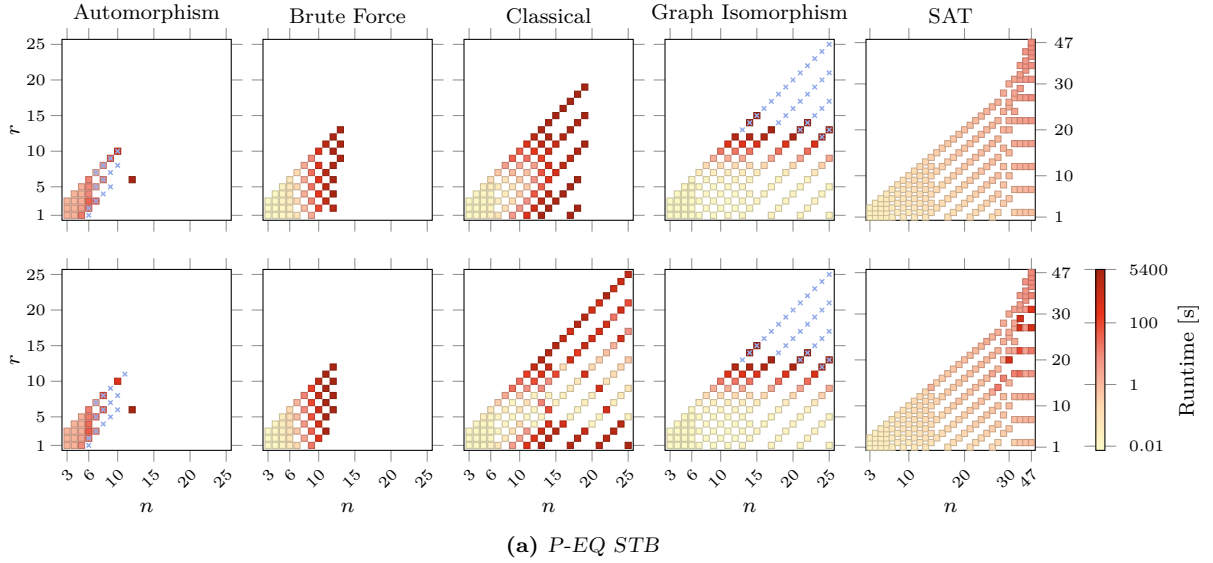


Figure A.4: Runtime behavior of the two SAT encodings on identical CSS input instances.

structure of CSS codes rather than the encoding itself.

Positive and Negative Instances

Figures A.5a, A.5b, A.5c, and A.5d separate the aggregated results from Chapter 7 into equivalent and inequivalent instances to highlight the strengths and weaknesses in runtime behavior of the different prototype solutions.



A Appendix

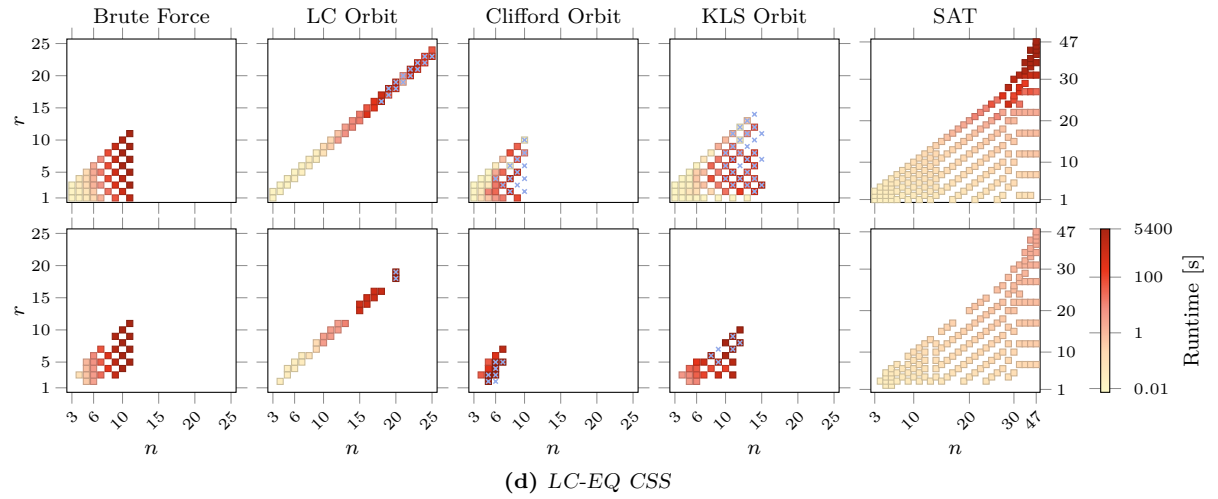
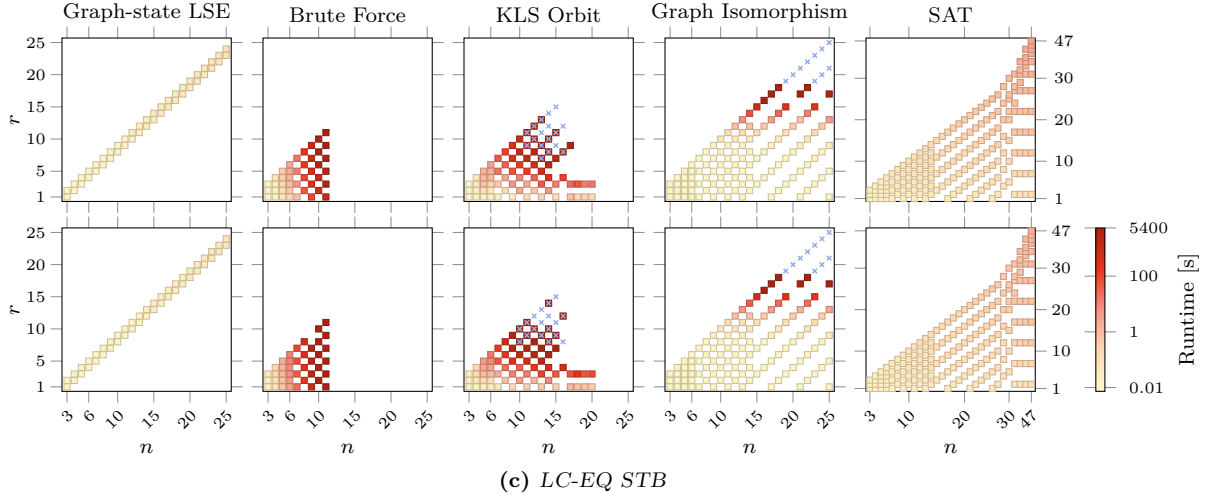
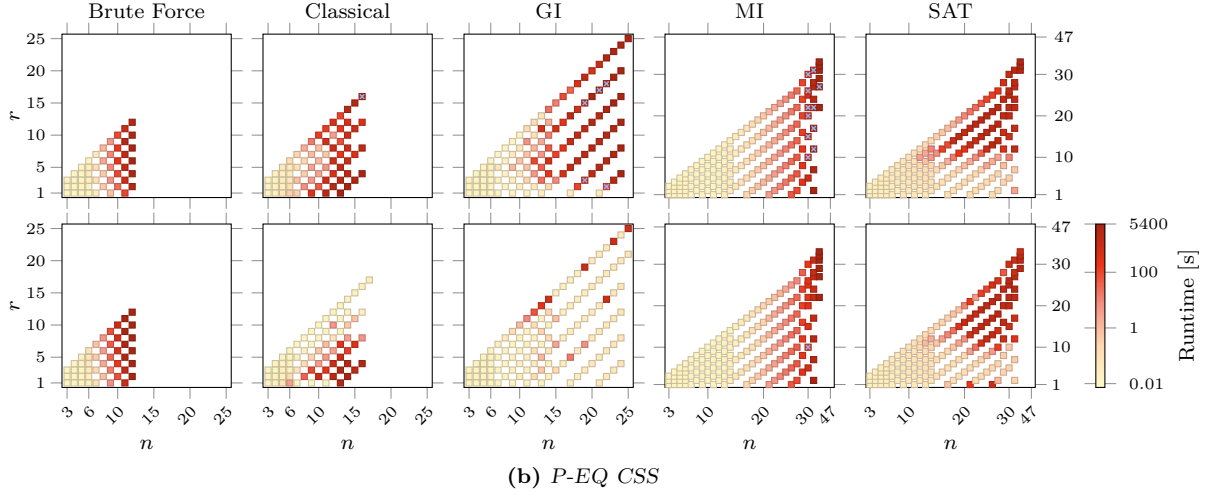


Figure A.5: Runtime heatmaps for the prototype solutions on randomized code instances, separated into positive (top) and negative (bottom) instances.

Notation

$\mathcal{S}_n$ Symmetric group of permutations of n elements.

$\mathbb{F}_2$ Binary field.

$\mathbb{F}_4$ Finite field with four elements.

$\mathcal{GL}_r$ General linear group of invertible $r \times r$ matrices over $\mathbb{F}_2$.

$\mathcal{H}^{\otimes n}$ Hilbert space of n qubits.

$\mathcal{U}_m$ Group of $m \times m$ unitary matrices.

$\mathcal{P}_n$ n -qubit Pauli group.

$\widehat{\mathcal{C}}_n$ n -qubit Clifford group including global phases.

$\mathcal{C}_n$ n -qubit Clifford group modulo global phases.

$\mathrm{Sp}(m, \mathbb{F}_2)$ Symplectic group of degree m over $\mathbb{F}_2$.

$\mathcal{S}$ Stabilizer group of a quantum code.

$\llbracket n, k, d \rrbracket$ Quantum code with n physical qubits, k logical qubits, and distance d .

$r = n - k$ Number of independent generators of an $\llbracket n, k, d \rrbracket$ stabilizer code.

$\mathcal{C}$ Code space of a quantum or classical code.

$\mathcal{C}(\mathcal{S})$ Centralizer of the stabilizer group in the Pauli group.

$\mathrm{S}(\mathbf{C})$ Binary symplectic stabilizer tableau of code $\mathbf{C}$.

$\mathrm{L}_x(\mathbf{C})$ Binary symplectic matrix of logical X operators of code $\mathbf{C}$.

$\mathrm{L}_z(\mathbf{C})$ Binary symplectic matrix of logical Z operators of code $\mathbf{C}$.

$\mathrm{H}_x(\mathbf{C})$ X -check matrix of a CSS code $\mathbf{C}$.

$\mathrm{H}_z(\mathbf{C})$ Z -check matrix of a CSS code $\mathbf{C}$.

$\mathrm{span}\{x_1, \dots, x_m\}$ Linear span of the given elements.

$\mathrm{diag}(v_0, \dots, v_{n-1})$ Diagonal matrix with entries $v_0, \dots, v_{n-1}$.

$\mathrm{wt}(x)$ Hamming weight of x ; for a Pauli operator, the number of non-identity tensor factors.

- $W(A)$ Weight enumerator of A .
- $\mathcal{H}(A)$ Hull of a linear code A , given by $A \cap A^\perp$; for a matrix A , the hull of its row space.
- $\langle u | v \rangle_{\mathbf{sp}}$ Symplectic inner product of binary symplectic vectors u and v .
- $\text{tr}(u)$ Field trace of u from $\mathbb{F}_4$ to $\mathbb{F}_2$.
- $u * v$ $\mathbb{F}_2$ -valued trace inner product of vectors u and v over $\mathbb{F}_4$.
- $\mathcal{V}$ Vertex set of a graph.
- $\mathcal{E}$ Edge set of a graph.
- $N(v)$ Neighborhood of vertex v .
- $\mathcal{P}(A)$ Power set of A .
- $|A|$ Cardinality of the set A .
- c Vertex or edge coloring of a graph.
- $\mathcal{I}$ Collection of independent sets of a matroid.
- E** Ground set of a matroid.
- Circ** Set of circuits of a matroid.
- NP** Class of decision problems solvable in nondeterministic polynomial time.
- SAT** Boolean Satisfiability Problem.
- GI** Graph Isomorphism Problem.
- CGI** Colored Graph Isomorphism Problem.
- CCE** Classical Code Equivalence Problem.
- MI** Matroid Isomorphism Problem.
- P-EQ STB** Permutation Equivalence Problem for stabilizer codes.
- P-EQ CSS** Permutation Equivalence Problem for CSS codes.
- LC-EQ STB** Local Clifford Equivalence Problem for stabilizer codes.
- LC-EQ CSS** Local Clifford Equivalence-to-CSS Problem.
- $\mathcal{O}$ Big-O asymptotic notation.

Acronyms

QECC Quantum Error-Correction Code.

QEC Quantum Error Correction.

CSS Calderbank-Shor-Steane.

LC local Clifford.

RREF Row Reduced Echelon Form.

KLS Khesin-Lu-Shor.

HK Hu-Khesin.

RQ Research Question.

LSE Linear System of Equations.

OOM Out-of-Memory.

BB Bivariate Bicycle.

Bibliography

- [1] Scott Aaronson and Daniel Gottesman. Improved simulation of stabilizer circuits. *Physical Review A*, 70(5), November 2004.
- [2] Jeremy C. Adcock, Sam Morley-Short, Axel Dahlberg, and Joshua W. Silverstone. Mapping graph state orbits under local complementation. *Quantum*, 4:305, August 2020.
- [3] Aisling Mac Aree and Mark Howard. Exhaustive optimisation of automorphism groups for stabiliser codes, 2026.
- [4] László Babai. Graph isomorphism in quasipolynomial time, 2016.
- [5] Huck Bennett and Kaung Win. Relating code equivalence to other isomorphism problems. *Designs, Codes and Cryptography*, 93:701–723, 12 2024.
- [6] H. Bombin and M. A. Martin-Delgado. Topological quantum distillation. *Physical Review Letters*, 97(18), October 2006.
- [7] André Bouchet. An efficient algorithm to recognize locally equivalent graphs. *Combinatorica*, 11:315–329, 1991.
- [8] André Bouchet. Recognizing locally equivalent graphs. *Discrete mathematics*, 1993.
- [9] Iliya Bouyukliev and Stefka Bouyuklieva. About code equivalence – a geometric approach, 2022.
- [10] Sergey Bravyi, Andrew W. Cross, Jay M. Gambetta, Dmitri Maslov, Patrick Rall, and Theodore J. Yoder. High-threshold and low-overhead fault-tolerant quantum memory. *Nature*, 627(8005):778–782, March 2024.
- [11] Adán Cabello, Lars Eirik Danielsen, Antonio J. López-Tarrida, and José R. Portillo. Optimal preparation of graph states. *Physical Review A*, 83(4), April 2011.
- [12] A. R. Calderbank, E. M Rains, P. W. Shor, and N. J. A. Sloane. Quantum error correction via codes over $\text{GF}(4)$, 1997.
- [13] Kai Chen and Hoi-Kwong Lo. Conference key agreement and quantum sharing of classical secrets with noisy GHZ states. In *Proceedings. International Symposium on Information Theory, 2005. ISIT 2005.*, page 1607–1611. IEEE, 2005.
- [14] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158. ACM, 1971.
- [15] Andrew Cross and Drew Vandeth. Small binary stabilizer subsystem codes, 2025.

- [16] Axel Dahlberg, Jonas Helsen, and Stephanie Wehner. The complexity of the vertex-minor problem, 2019.
- [17] Axel Dahlberg, Jonas Helsen, and Stephanie Wehner. Counting single-qubit Clifford equivalent graph states is $\#P$ -complete. *Journal of Mathematical Physics*, 61(2), February 2020.
- [18] Lars Eirik Danielsen and Matthew G. Parker. On the classification of all self-dual additive codes over $GF(4)$ of length up to 12. *Journal of Combinatorial Theory, Series A*, 113(7):1351–1367, October 2006.
- [19] Shival Dasu and Simon Burton. A classification of transversal Clifford gates for qubit stabilizer codes, 2025.
- [20] Ross Duncan, Aleks Kissinger, Simon Perdrix, and John van de Wetering. Graph-theoretic simplification of quantum circuits with the ZX-calculus. *Quantum*, 4:279, June 2020.
- [21] Matthew B. Elliott, Bryan Eastin, and Carlton M. Caves. Graphical description of the action of Clifford operators on stabilizer states. *Physical Review A*, 77(4), April 2008.
- [22] Thomas Feulner. The automorphism groups of linear codes and canonical representatives of their semilinear isometry classes. *Advances in Mathematics of Communications*, 3(4):363–383, 2009.
- [23] Daniel Gottesman. Stabilizer codes and quantum error correction, 1997.
- [24] Daniel Gottesman. Surviving as a quantum computer in a classical world, May 2024. Textbook manuscript preprint.
- [25] Markus Grassl, Martin Rötteler, and Thomas Beth. Computing local invariants of quantum-bit systems. *Physical Review A*, 58(3):1833–1839, September 1998.
- [26] Jeongwan Haah and John Preskill. Logical-operator tradeoff for local quantum codes. *Physical Review A*, 86(3), September 2012.
- [27] Hanson Hao. Investigations on automorphism groups of quantum stabilizer codes, 2021.
- [28] Alexander Tianlin Hu and Andrey Boris Khesin. Improved graph formalism for quantum circuit simulation. *Physical Review A*, 105(2), February 2022.
- [29] Andrey Boris Khesin and Alexander Li. Equivalence classes of quantum error-correcting codes, 2024.
- [30] Andrey Boris Khesin, Jonathan Z. Lu, and Peter W. Shor. Graphical quantum Clifford-encoder compilers from the ZX calculus, 2025.
- [31] Andrey Boris Khesin, Jonathan Z. Lu, and Peter W. Shor. Universal graph representation of stabilizer codes. *PRX Quantum*, 6(4), November 2025.
- [32] Aleks Kissinger. Phase-free ZX diagrams are CSS codes (...or how to graphically grok the surface code), 2022.

- [33] Mithilesh Kumar, Håvard Raddum, and Srimathi Varadharajan. Graphs and self-dual additive codes over $\text{GF}(4)$, 03 2019.
- [34] J. Leon. Computing automorphism groups of error-correcting codes. *IEEE Transactions on Information Theory*, 28(3):496–511, 1982.
- [35] Brendan D. McKay and Adolfo Piperno. Practical graph isomorphism, II, 2013.
- [36] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.
- [37] James Oxley. *Matroid Theory*. Oxford University Press, 2 edition, 2011.
- [38] Erez Petrank and Ron M. Roth. Is code equivalence easy to decide? *IEEE Trans. Inf. Theory*, 43:1602–1604, 1997.
- [39] Joschka Roffe. Quantum error correction: an introductory guide. *Contemporary Physics*, 60(3):226–245, July 2019.
- [40] Pradeep Sarvepalli. Quantum codes and symplectic matroids, 2011.
- [41] Hasan Sayginel, Stergios Koutsoumpas, Mark Webster, Abhishek Rajput, and Dan E Browne. Fault-tolerant logical Clifford gates from code automorphisms, 2025.
- [42] D. Schlingemann. Stabilizer codes can be realized as graph codes, 2001.
- [43] Nicolas Sendrier. Finding the permutation between equivalent linear codes: The support splitting algorithm. *IEEE Trans. Inf. Theory*, 46:1193–1203, 2000.
- [44] Lorenzo Traldi. Binary matroids and local complementation. *European Journal of Combinatorics*, 45:21–40, April 2015.
- [45] Raghavendra Rao B. V. and Jayalal M. N. Sarma. On the complexity of matroid isomorphism problem, 2008.
- [46] John van de Wetering. ZX-calculus for the working quantum computer scientist, 2020.
- [47] Maarten Van den Nest, Jeroen Dehaene, and Bart De Moor. Efficient algorithm to recognize the local Clifford equivalence of graph states. *Physical Review A*, 70(3), September 2004.
- [48] Maarten Van den Nest, Jeroen Dehaene, and Bart De Moor. Graphical description of the action of local Clifford transformations on graph states. *Physical Review A*, 69(2), February 2004.
- [49] Maarten Van den Nest, Jeroen Dehaene, and Bart De Moor. Local equivalence of stabilizer states. *Proceedings of the 16th international symposium on mathematical theory of networks and systems (MTNS)*. KU Leuven, Belgium, May 2004.
- [50] Maarten Van den Nest, Jeroen Dehaene, and Bart De Moor. Local invariants of stabilizer codes. *Physical Review A*, 70(3), September 2004.

- [51] Maarten Van den Nest, Jeroen Dehaene, and Bart De Moor. Finite set of invariants to characterize local Clifford equivalence of stabilizer states. *Physical Review A*, 72(1), July 2005.
- [52] Mark Webster, Stergios Koutsoumpas, and Dan E Browne. Heuristic and optimal synthesis of CNOT and Clifford circuits, 2025.
- [53] Zipeng Wu, Song Cheng, and Bei Zeng. A ZX-calculus approach for the construction of graph codes, 2024.